МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 8

Тема занятия введение в desktop-автоматизацию. создание прототипа и базовая автоматизация формы.
Вариант: 11

Выполнил студент 3 курса группы
ЭФБО-09-23 Калинин Н.В.

Проверил Громов Е.В.

Москва
2026 г.

Цель работы: сформировать у обучающихся профессиональные компетенций по созданию сложных RPA-сценариев для взаимодействия с десктопными приложениями, включая обработку табличных данных, управление динамически и состояниями интерфейса и корректную обработку модальных окон.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Основные понятия

В первой практической работе мы освоили базовые операции: открытие приложений, клики, ввод текста, чтение значений. Это фундамент, на котором строятся любые RPA-решения. Однако реальные бизнес-задачи редко ограничиваются простыми линейными сценариями.

В современных RPA-проектах робот сталкивается с более сложными вызовами:

- Табличные данные — извлечение информации из отчётов, прайс-листов, реестров
- Динамические интерфейсы — элементы, которые появляются, исчезают и меняют состояние

- Модальные окна — неожиданные диалоги, блокирующие выполнение
- Нештатные ситуации — ошибки, сбои, исключения, требующие интеллектуальной обработки

Эти вызовы и составляют содержание нашей второй практической работы. Мы переходим от уровня «базового пользователя» к уровню «профессионального разработчика», способного создавать надёжные, отказоустойчивые и масштабируемые RPA-решения.

Мы изучим архитектуру сложных RPA-решений — научимся проектировать модульных, масштабируемых роботов, которые легко поддерживать и развивать. Разберём жизненный цикл Desktop-элементов в динамическом интерфейсе: поймём, как элементы появляются, изменяются и исчезают, и научимся создавать устойчивые селекторы даже для самых нестабильных интерфейсов. Освоим обработку модальных окон — научимся обнаруживать диалоги, определять их тип и принимать правильные решения (OK, Cancel, Yes, No) без потери управления. Погрузимся в обработку исключительных ситуаций — создадим отказоустойчивые сценарии с механизмами повторных попыток, логированием и корректным завершением. И наконец, детально разберём продвинутые блоки Puzzle RPA: работу с таблицами («Получить табличные данные», множественный поиск), точное позиционирование (клики со смещением, по координатам, drag-and-drop), управление прокруткой и фокусом, а также автоматизацию Java-приложений.

В результате вы будете не просто использовать базовые блоки Desktop-автоматизации, а осознанно проектировать сложные RPA-решения, способные:

- Обрабатывать табличные данные любой сложности
- Адаптироваться к динамически меняющемуся интерфейсу
- Корректно реагировать на модальные окна
- Восстанавливаться после сбоев
- Взаимодействовать с приложениями на разных технологиях (включая Java)

Вы перейдёте от уровня «базового пользователя» к уровню «профессионального разработчика», способного создавать надёжные, масштабируемые и отказоустойчивые RPA-решения для любых десктопных задач.

Архитектура сложных RPA-решений для Desctop

Когда мы только начинали, наши сценарии были линейными: открыл приложение → кликнул → ввёл текст → сохранил. Это работало для маленьких задач. Но в реальном бизнесе роботы обрабатывают тысячи операций, работают с разными системами, сталкиваются с ошибками, и линейный подход превращается в кошмар поддержки.

Плохая архитектура – это когда:

- Чтобы понять логику, нужно пролистывать 500 блоков
- Исправление одной ошибки ломает три других сценария
- Нельзя переиспользовать уже написанный код
- Робот падает при любом неожиданном повороте

Хорошая архитектура – это когда:

- Сценарий читается как оглавление книги
- Каждый компонент можно заменить, не трогая остальные
- Общие функции написаны один раз и используются везде
- Робот устойчив к сбоям и понятно сообщает о проблемах

Разберём переход от линейных сценариев к модульным.

Линейный подход – это когда все действия выстроены в одну длинную цепочку. Сто блоков идут один за другим, и каждый следующий зависит от предыдущего. Такой сценарий легко написать, но очень трудно поддерживать. Любое изменение требует вмешательства в самую середину этой цепочки, что часто ломает всё вокруг.

Модульный подход предлагает совершенно иную философию. Вместо одного длинного сценария мы создаём несколько небольших, независимых модулей, каждый из которых отвечает за свою конкретную задачу. Один модуль отвечает за авторизацию, другой – за загрузку данных, третий – за формирование отчётов. А главный сценарий только управляет ими, как дирижёр оркестром: вызывает нужные модули в правильной последовательности и передаёт данные между ними.

Что нам даёт модульность:

- Модуль авторизации, написанный один раз, можно использовать во всех сценариях, где нужен вход в систему
- Каждый модуль можно проверить отдельно, не запуская всего робота
- Если меняется логика входа, правим только модуль авторизации, остальные сценарии не страдают
- Главный сценарий читается как инструкция: что делаем, но не как именно

Профессиональные RPA-решения строятся по принципу разделения ответственности. Каждый уровень занимается только своими задачами и не лезет в чужие.

- Уровень оркестрации – это главный управляющий сценарий. Он не знает, как именно авторизоваться в программе или как сохранить документ. Он знает только что нужно сделать и в каком порядке. Оркестратор похож на дирижёра: он не играет на инструментах, но именно он создаёт музыку, управляя музыкантами. Он вызывает модули, передаёт им данные, получает результаты и решает, что делать дальше.
- Уровень бизнес-логики – это мозг робота. Здесь принимаются все решения, выполняются расчёты, проверяются условия. Именно на этом уровне живут все «если» и «иначе». Бизнес-логика не знает, как выглядит окно программы и где там кнопка «Сохранить». Она работает только с данными: проверяет, корректна ли сумма счёта, не просрочена ли дата, достаточно ли у клиента средств. Если данные валидны, бизнес-логика говорит: «Всё хорошо, можно сохранять». Если нет – «Ошибка, нужна проверка оператором».
- Уровень взаимодействия с интерфейсом – это руки и глаза робота. Здесь выполняются все базовые операции: открытие приложений, клики, ввод текста,

чтение значений с экрана. Этот уровень не принимает решений – он только исполняет команды. Ему сказали «введи сумму 1000 рублей в поле Сумма» – он вводит. Сказали «нажми кнопку Сохранить» – нажимает. Он не проверяет, правильная ли это сумма и нужно ли вообще сохранять. Это уровень чистого исполнителя.

- Уровень обработки ошибок – это система безопасности робота. Он следит за тем, чтобы при первой же проблеме робот не разбился, а попытался исправить ситуацию или хотя бы корректно завершился, сохранив все промежуточные результаты. Этот уровень перехватывает исключения, логирует ошибки с полным контекстом, пробует повторить неудачные действия и, если ничего не помогает, зовёт на помощь человека.

Один из главных принципов хорошей архитектуры – не повторяйся. Если вы написали один и тот же код дважды, значит, что-то пошло не так.

В Puzzle RPA есть прекрасный инструмент для повторного использования – Мастер UI. С его помощью можно сохранять элементы интерфейса в библиотеку. Один раз настроили селектор для кнопки «Сохранить» в 1С, дали ей осмысленное имя, и теперь можете использовать эту кнопку в любом сценарии. Более того, если в следующей версии 1С разработчики изменят идентификатор этой кнопки, вам не нужно править все сценарии – достаточно исправить элемент в одном месте.

То же самое с модулями. Модуль авторизации, написанный для одного проекта, можно с минимальными доработками использовать в другом. Модуль выгрузки отчёта в Excel пригодится в десятках разных роботов. Создавая каждый модуль, думайте о нём как о кирпичике, который будет использоваться снова и снова.

Важный приём – параметризация. Хороший модуль не должен содержать жёстко зашитых данных. Он должен принимать всё необходимое через параметры. Модуль создания документа не должен знать, что сумма сегодня равна 1000 рублей. Ему передают сумму как параметр, и он работает с тем, что получил. Такой модуль можно использовать для создания любых документов с любыми суммами.

Рассмотрим с Вами момент масштабирования робота. Когда робот обрабатывает десять записей, всё работает идеально. Когда записей становится десять тысяч, начинаются проблемы. Масштабирование – это умение работать с большими объёмами данных без потери производительности и надёжности.

Пакетная обработка – первый друг масштабирования. Вместо того чтобы пытаться обработать десять тысяч записей за один присест, разбиваем их на пакеты по сто или двести. Обработали пакет – сохранили промежуточный результат, дали системе небольшую передышку, обработали следующий. Это не только снижает нагрузку на приложение, но и позволяет в случае сбоя продолжить с того места, где остановились, а не начинать всё заново.

Контроль ресурсов – важный аспект длительной работы. Приложения, с которыми работает робот, нужно закрывать, когда они больше не нужны. Большие

переменные, хранящие тысячи записей, нужно очищать, когда данные уже обработаны и сохранены. Иначе через несколько часов работы робот начнёт потреблять гигабайты памяти и либо упадёт сам, либо положит всю систему.

Подведём итоги: хорошая архитектура – это как хороший фундамент, её не видно, но на ней держится всё здание. Потратив время на проектирование сейчас, вы сэкономите дни и недели поддержки в будущем.

В мире RPA, где роботы работают 24/7 и обрабатывают миллионы операций, архитектура становится не просто вопросом удобства, а вопросом выживания проекта. Плохо спроектированный робот развалится при первой же нагрузке. Хорошо спроектированный будет работать годами, радуя заказчиков стабильностью и лёгкостью изменений.

И помните: архитектура – это не про то, как красиво разложить блоки. Это про то, как сделать так, чтобы робот решал бизнес-задачи сегодня, завтра и через год, не требуя постоянных вливаний на поддержку и доработку.

Жизненный цикл Desktop-элемента в динамическом интерфейсе

Каждый элемент интерфейса проходит через определённые стадии существования. Их понимание помогает роботу выбрать правильную стратегию взаимодействия.

Стадия 1. Рождение (появление в иерархии)

Элемент появляется в дереве элементов приложения. Это происходит, когда:

- Открывается новое окно
- Раскрывается вкладка или раздел
- Динамически подгружается содержимое

На этой стадии элемент уже существует, но он ещё не готов к работе. Это как новорождённый ребёнок – он уже есть, но говорить и ходить пока не умеет. Элемент может быть невидимым, иметь нулевые размеры, не реагировать на действия.

Как робот может это обнаружить: элемент уже находится по своему селектору, но его свойства состояния (видимость, активность) ещё не соответствуют рабочему режиму.

Стадия 2. Инициализация (загрузка данных)

Элемент заполняется данными, настраивается, приобретает окончательный вид. В зависимости от сложности приложения, инициализация может занимать от миллисекунд до нескольких секунд.

Признаки инициализации:

- Элемент может мигать, менять цвет
- На нём может отображаться индикатор загрузки
- Текст может появляться постепенно
- Размеры могут изменяться

Коварство этой стадии: элемент формально существует, его можно найти, но данные внутри него ещё не те, которые ожидает робот. Например, таблица уже отобразилась, но строки в неё ещё загружаются.

Стадия 3. Готовность к взаимодействию

Элемент полностью загружен, видим на экране, доступен для действий. Это состояние, в котором робот может с ним работать.

Признаки готовности:

- IsEnabled = True (доступен)
- Элемент видим на экране
- Содержит актуальные данные
- Реагирует на действия

Золотое правило: Никогда не предполагайте, что элемент готов. Всегда проверяйте его состояние перед взаимодействием.

Стадия 4. Изменение состояния

В процессе работы элемент может многократно менять своё состояние. Это нормально и ожидаемо.

Виды изменений:

- Фокус – элемент получает или теряет фокус ввода
- Значение – меняется текст, число, состояние чекбокса
- Доступность – элемент становится недоступным, а потом снова доступным
- Видимость – элемент может временно скрываться и появляться
- Положение – при изменении размеров окна элементы могут перемещаться

Важно понимать: изменение состояния – это не ошибка, а нормальное поведение интерфейса. Робот должен быть к этому готов.

Стадия 5. Трансформация

- Элемент изменяет свои фундаментальные характеристики:
- Меняется тип элемента (редко, но бывает)
- Полностью перестраиваются дочерние элементы
- Изменяются идентификационные свойства (например, Name)

Трансформация часто происходит при обновлении данных. Например, таблица может полностью перерисоваться с новыми строками. Формально это тот же элемент, но фактически – уже другой.

Опасность: селекторы, которые полагались на внутреннюю структуру, могут перестать работать после трансформации.

Стадия 6. Исчезновение

Элемент удаляется из интерфейса. Причины:

- Закрытие окна
- Переход на другую вкладку
- Обновление содержимого
- Скрытие по условию

После исчезновения любые попытки взаимодействия с элементом приведут к ошибке «Элемент не найден». Это нормальная ситуация, которую нужно обрабатывать.

Практические приёмы для работы с динамическими интерфейсами:

Приём 1. Путь через стабильных родителей

Если сам элемент динамический, найдите его стабильного родителя и стройте путь от него.

Плохо: искать непосредственно кнопку с названием «Удалить запись №12345».

Хорошо: найти панель действий (она стабильна), а внутри неё искать кнопку по её типу и позиции.

Приём 2. Комбинация свойств

Используйте несколько свойств для уникальной идентификации. Одно свойство может быть ненадёжным, но их комбинация часто даёт нужную уникальность.

Приём 3. Поиск по индексу в коллекции

Для элементов в списках и таблицах используйте поиск по индексу.

Как работает: находите все элементы определённого типа в контейнере, затем берёте нужный по порядковому номеру.

Плюсы: не зависит от названий и идентификаторов.

Минусы: если порядок изменится, робот возьмёт не тот элемент.

Приём 4. Обработка динамических списков

Для списков, которые подгружаются при прокрутке, используйте алгоритм:

- Получить текущее количество элементов
- Прокрутить список
- Подождать
- Снова получить количество
- Если количество выросло – появились новые элементы, можно продолжать
- Если количество не меняется несколько раз подряд – достигнут конец списка

Приём 5. Умные таймауты

Не используйте одинаковые таймауты для всех операций. Для быстрых действий ставьте маленькие таймауты (1-2 секунды), для длительных – большие (30-60 секунд). И всегда имейте запас на случай, если система тормозит.

Подведём итоги философии работы с динамическим интерфейсом. Работа с динамическим интерфейсом требует особого мышления. Это не механика («нажать кнопку»), а биология («взаимодействовать с живым организмом»).

Главные принципы:

- Никогда не предполагайте – всегда проверяйте. Элемент может не быть готовым, даже если он виден.
- Ожидание – не слабость, а мудрость. Лучше подождать лишнюю секунду, чем упасть с ошибкой и начинать всё заново.
- Ищите стабильное в нестабильном. Даже в самом динамичном интерфейсе есть точки опоры – стабильные контейнеры, неизменные свойства, предсказуемые паттерны поведения.

- Проектируйте на вырост. Ваши селекторы должны пережить хотя бы пару обновлений приложения без полной переделки.
- Будьте готовы к неожиданностям. Интерфейс может повести себя не так, как вчера. Робот должен это пережить.

Понимание жизненного цикла элемента превращает робота из слепого исполнителя в адаптивного агента, способного работать в реальных, постоянно меняющихся условиях. А это именно то, что требуется в промышленной автоматизации.

Модальные окна: природа, типы и стратегии обработки

Опасность модальных окон

Модальное окно – это элемент интерфейса, который блокирует взаимодействие пользователя (и робота) со всеми остальными окнами приложения до тех пор, пока оно не будет закрыто. Оно требует обязательного действия: подтверждения, отмены, ввода данных или просто нажатия кнопки «ОК».

Для робота модальное окно – это как внезапное препятствие на дороге. Робот ехал по своему маршруту, и вдруг – стена. Дальше ехать нельзя, нужно разобраться с препятствием и только потом продолжать путь.

Почему модальные окна так опасны для автоматизации:

- Они появляются неожиданно, в самый неподходящий момент
- Блокируют доступ к основному окну, с которым работает робот
- Могут быть разных типов, требующих разных решений
- Если их не обработать, робот «зависает» или падает с ошибкой
- Часто имеют таймауты – если не ответить вовремя, окно закрывается с нежелательным результатом

С технической точки зрения модальное окно – это отдельное окно (или диалог), которое:

- Создаётся поверх основного окна
- Захватывает фокус ввода
- Блокирует передачу сообщений основному окну
- Обычно имеет свой набор элементов управления (кнопки, поля, надписи)
- Закрывается только после явного действия пользователя

В терминах UI Automation модальное окно – это отдельный элемент с ControlType = WindowControl, но с особым свойством: IsDialog = True. Оно находится на верхнем уровне иерархии, но при этом блокирует доступ к нижележащим окнам.

Модальные окна – это способ приложения привлечь внимание пользователя к важному событию. Разработчики приложений используют их, чтобы:

- Запросить подтверждение опасного действия
- Сообщить об ошибке
- Предупредить о последствиях
- Запросить дополнительные данные
- Убедиться, что пользователь увидел важную информацию

Для человека модальное окно – это нормально. Он видит его, обрабатывает и идёт дальше. Для робота это нарушение запланированного сценария, которое требует отдельной логики обработки.

Рассмотрим, как модальные окна влияют на робота. Чтобы понять опасность модальных окон, нужно представить, что происходит с роботом, когда они появляются.

Сценарий без обработки модального окна:

- Робот выполняет действие – кликает по кнопке «Удалить запись»
- Приложение показывает модальное окно «Вы уверены?» и блокирует основное окно
- Робот пытается продолжить работу – например, хочет кликнуть по кнопке «Сохранить» в основном окне
- Основное окно заблокировано, клик не проходит
- Робот ждёт какое-то время (таймаут), но окно не отвечает
- Робот падает с ошибкой «Элемент не найден» или «Таймаут ожидания»
- Модальное окно так и висит, заблокировав приложение

Что должно происходить на самом деле:

- Робот выполняет действие – кликает по кнопке «Удалить запись»
- Робот переключается в режим ожидания модального окна
- Обнаруживает появление окна подтверждения
- Анализирует окно: читает заголовок, текст сообщения, определяет доступные кнопки
- Принимает решение: например, нажать «Да»
- Выполняет клик по нужной кнопке
- Дожидается закрытия модального окна
- Возвращается к основному сценарию

Стратегии обработки модальных окон

Стратегия 1. Упреждающая обработка

Мы заранее знаем, что после определённого действия появится модальное окно, и сразу после действия переключаемся на его ожидание и обработку.

Когда применять: для предсказуемых сценариев, где точно известно, что появится модальное окно определённого типа.

Алгоритм:

- Выполнить действие, которое вызывает модальное окно
- Сразу переключиться на ожидание появления модального окна
- Обработать окно
- Продолжить основной сценарий

Плюсы: надёжно, предсказуемо, легко реализовать.

Минусы: работает только для известных заранее окон.

Стратегия 2. Реактивная обработка (фоновый мониторинг)

Робот постоянно, в фоновом режиме, проверяет, не появилось ли модальное окно. Это особенно важно, когда окно может возникнуть в любой момент, не связанный с конкретным действием робота.

Когда применять: для непредсказуемых окон (ошибки соединения, предупреждения системы, сообщения от других процессов).

Алгоритм:

- В ключевых точках сценария (перед важными действиями) проверять наличие модальных окон
- Если окно есть – приостановить основной сценарий и обработать окно
- После обработки продолжить с того же места

Плюсы: защита от неожиданных окон.

Минусы: постоянные проверки создают небольшую нагрузку.

Стратегия 3. Циклическая обработка с таймаутом

Робот в цикле проверяет наличие модального окна в течение определённого времени. Если окно появляется – обрабатывает. Если не появляется за отведённое время – считает, что его не было, и продолжает работу.

Когда применять: когда точно известно, что окно может появиться, но неизвестно, через сколько секунд.

Алгоритм:

- Засечь время начала ожидания
- Пока время ожидания не истекло:

  Проверить наличие модального окна

  ✓ Если есть – обработать и выйти из цикла

  ✓ Если нет – подождать 0.5-1 секунду
- Продолжить основной сценарий

Плюсы: гибкость, не пропустит окно, если оно появится с задержкой.

Минусы: может долго ждать, если окно так и не появилось.

Стратегия 4. Обработка цепочек модальных окон

Иногда одно модальное окно порождает другое. Например, после нажатия «Да» может появиться следующее окно с предупреждением.

Когда применять: для многоступенчатых диалогов, мастеров, сложных подтверждений.

Алгоритм:

- Обработать первое модальное окно
- Сразу после его закрытия переключиться на ожидание следующего
- Повторять, пока есть окна
- Вернуться к основному сценарию

Плюсы: полнота обработки.

Минусы: сложность, нужно точно знать последовательность.

Заключение: модальные окна как часть жизни робота. Модальные окна – не враги, а просто особенность интерфейса, с которой нужно научиться работать.

Хороший робот не пытается их игнорировать или надеяться, что они не появятся. Он готов к ним, знает их в лицо и умеет с ними обращаться.

Главные принципы работы с модальными окнами:

- Предсказуемость – знайте, какие окна могут появиться и после каких действий
- Наблюдательность – всегда следите за появлением новых окон
- Анализ – читайте текст окон, не полагайтесь только на заголовки
- Решительность – принимайте чёткие решения и выполняйте их
- Восстановление – после обработки возвращайтесь к основному сценарию

Робот, который умеет работать с модальными окнами, становится по-настоящему промышленным. Он не упадёт от неожиданного предупреждения, не зависнет на диалоге подтверждения, не пропустит важное сообщение об ошибке. А это именно то, что требуется от профессионального RPA-решения.

Обработка исключительных ситуаций в сложных сценариях

Исключительные ситуации

Исключительная ситуация (исключение, ошибка, сбой) – это любое отклонение от нормального хода выполнения сценария, которое робот не предусмотрел в своей основной логике.

Важно понять главное: исключения – это не признак плохого кода, а признак реальной жизни. Хороший робот отличается от плохого не отсутствием ошибок, а умением их обрабатывать.

Плохой робот при первой же проблеме падает и оставляет после себя хаос. Хороший робот замечает проблему, пытается её исправить, логирует детали и, если ничего не помогает, корректно завершается, сохраняя всё, что можно было сохранить.

Чтобы научиться обрабатывать исключения, нужно понимать, откуда они берутся. Источники проблем можно разделить на несколько категорий.

Внешние источники (не зависят от робота)

Проблемы с приложением:

- Приложение зависло или не отвечает
- Приложение вылетело (упало)
- Приложение работает медленнее обычного
- Изменился интерфейс (новая версия, обновление)
- Появилось неожиданное модальное окно

Проблемы с окружением:

- Пропало сетевое соединение
- Отключили питание
- Закончилось место на диске
- Сработала антивирусная защита
- Другой пользователь вмешался в работу

Проблемы с данными:

- Входные данные в неверном формате

- Пустые файлы
- Отсутствие обязательных полей
- Несоответствие типов данных
- Недопустимые значения (отрицательные суммы, даты из будущего)
- Внутренние источники (ошибки в сценарии)

Ошибки проектирования:

- Неверно выбран селектор элемента
- Недостаточные таймауты
- Отсутствие проверок существования
- Игнорирование возможности модальных окон

Ошибки реализации:

- Неправильная последовательность действий
- Потеря контекста (не то окно активно)
- Ошибки в логике ветвления
- Некорректная обработка типов данных

Пять уровней защиты

Философия обработки ошибок: пять уровней защиты

Хорошая стратегия обработки исключений строится как многоуровневая система защиты. Если не сработал один уровень, подключается следующий.

Уровень 1. Предотвращение

Лучшая ошибка – та, которой не случилось. На этом уровне мы делаем всё, чтобы исключительные ситуации не возникали.

Что делаем:

- Тщательно проектируем селекторы
- Закладываем достаточные таймауты
- Проверяем существование элементов перед действием
- Валидируем входные данные
- Изучаем приложение на предмет "узких мест"

Например:

Перед кликом по кнопке проверяем, что она существует, видима и активна.

Если нет – ждём или ищем альтернативу, но не кликаем вслепую.

Уровень 2. Обнаружение

Если ошибка всё же возникла, нужно её заметить как можно раньше.

Что делаем:

- Используем блоки с таймаутами
- Проверяем результаты действий
- Следим за появлением неожиданных окон
- Контролируем состояние приложения

Например:

После ввода данных проверяем, что они действительно появились в поле.

Если поле пустое – что-то пошло не так.

Уровень 3. Локальное восстановление

Пытаемся исправить ситуацию на месте, не прерывая основной сценарий.

Что делаем:

- Повторяем действие
- Пробуем альтернативный способ
- Ждём и пробуем снова
- Адаптируемся к изменениям

Например:

Элемент не нашёлся? Ждём секунду и пробуем ещё раз.

Не помогло? Пробуем другой селектор.

Всё равно нет? Возможно, элемент в другом фрейме – ищем там.

Уровень 4. Логирование и контекст

Если восстановиться не удалось, фиксируем всё, что может помочь при разборе.

Что делаем:

- Записываем точное сообщение об ошибке
- Сохраняем скриншот экрана
- Фиксируем состояние приложения
- Запоминаем, на каком шаге произошёл сбой
- Сохраняем входные данные текущей операции

Например:

Ошибка: Не найден элемент "Кнопка Сохранить"

Контекст: Шаг 42, запись №15, ИНН контрагента 7712345678

Скриншот: сохранён в logs\screenshots\2025-03-18_15-30-22.png

Логирование – это не просто запись "ошибка". Это искусство оставлять достаточно информации, чтобы потом можно было восстановить картину произошедшего.

Обязательная информация:

- Время события (с точностью до секунды)
- Уровень важности (INFO, WARNING, ERROR, CRITICAL)
- Имя модуля или сценария
- Конкретное действие, которое выполнялось
- Текст ошибки (если есть)

Контекстная информация:

- Номер обрабатываемой записи
- Входные данные текущей операции
- Состояние приложения (активное окно, видимые элементы)
- Путь к сохранённому скриншоту

Техническая информация:

- Версия робота
- Имя компьютера
- Имя пользователя, под которым запущен робот

- Время работы с момента старта

Пример хорошего лога

```
2025-03-18 15:30:22 | ERROR | Модуль обработки счетов | Шаг 42 | Запись №15
Действие: клик по кнопке "Сохранить"
Ошибка: элемент не найден (таймаут 10 сек)
Контекст: активное окно "1С:Предприятие-Документ №12345"
Скриншот: logs\screenshots\2025-03-18_15-30-22.png
Входные данные: ИНН=7712345678, Сумма=15000.00
```

Чего в логе быть не должно:

- Просто "Ошибка" без пояснений
- Только "Что-то пошло не так"
- Бессмысленных технических кодов без расшифровки
- Избыточной информации, засоряющей лог

В хорошем RPA-проекте обработка исключений – это не приложение к основному сценарию, а его неотъемлемая часть. Она проектируется вместе с основной логикой, реализуется параллельно с ней и тестируется так же тщательно.

Главные принципы зрелого подхода:

- Ошибки ожидаемы – мы знаем, что они будут, и готовы к ним
- Ошибки управляемы – у нас есть стратегия на каждый тип проблем
- Ошибки прозрачны – мы всегда знаем, что произошло и почему
- Ошибки не фатальны – робот либо восстанавливается, либо останавливается, не навредив

Робот, который умеет обрабатывать исключения, – это уже не просто программа, выполняющая последовательность действий. Это интеллектуальный агент, способный работать в реальном, несовершенном мире. А именно такие роботы и нужны бизнесу.

Продвинутые блоки десктопной автоматизации в Puzzle RPA

Первая практическая работа заложила фундамент. Мы научились открывать приложения, активировать окна, ожидать элементы, проверять их существование, читать значения и свойства, вводить текст, нажимать кнопки и использовать горячие клавиши. Это база, без которой невозможна никакая серьёзная автоматизация.

Но реальные бизнес-задачи редко ограничиваются работой с отдельными кнопками и полями. В жизни робот сталкивается с более сложными вызовами:

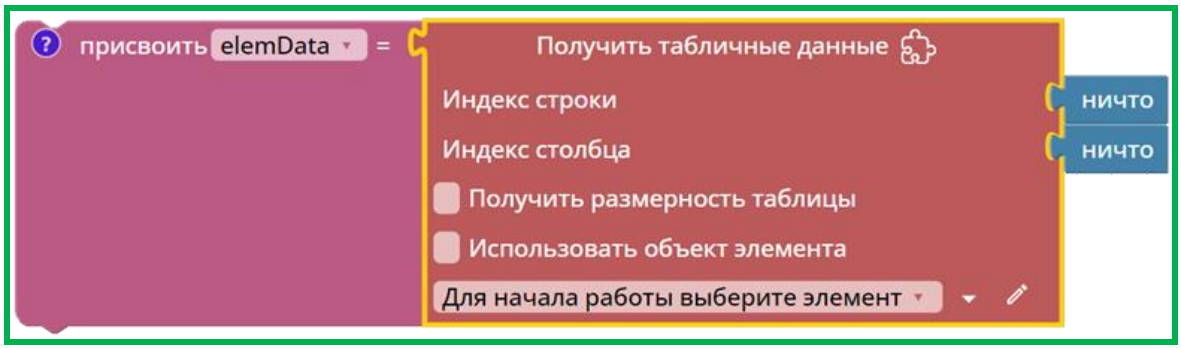
- Таблицы, из которых нужно извлечь сотни строк с данными
- Списки, где элементы нужно перебрать и обработать по одному
- Интерфейсы, где нельзя просто кликнуть по кнопке – нужно точно позиционировать курсор
- Объекты, которые нужно перетащить из одной части экрана в другую
- Длинные списки, которые нужно прокручивать, чтобы добраться до нужного элемента

- Java-приложения, которые живут по своим собственным законам и требуют особого подхода

Именно для решения таких задач в Puzzle RPA существует набор продвинутых блоков десктопной автоматизации. Эти инструменты расширяют возможности робота за пределы простых кликов и вводов, позволяя ему работать со сложными интерфейсами так же ловко, как это делает человек.

Рассмотрим подробно продвинутые блоки десктопной автоматизации в Puzzle RPA:

1. Блок «Получить табличные данные»



Блок «Получить табличные данные» – это интеллектуальный инструмент, который позволяет роботу извлекать структурированную информацию из таблиц, отображаемых в десктопных приложениях. Вместо того чтобы вручную перебирать каждую ячейку, определять, где заканчивается строка и начинается следующая, робот получает готовую таблицу с чёткой структурой: строки, столбцы, заголовки и данные.

Это один из самых мощных блоков в арсенале разработчика RPA, поскольку таблицы – основной способ представления структурированных данных в корпоративных приложениях: отчёты, реестры, списки, прайс-листы, выгрузки – всё это таблицы.

Блок выполняет сложную последовательность действий, скрытую от разработчика:

- Поиск табличного элемента – находит указанный элемент и определяет, является ли он таблицей (или контейнером, содержащим табличные данные).
- Анализ структуры – определяет, где в иерархии находятся строки, где столбцы, а где ячейки. Разные фреймворки (WinForms, WPF, Qt) по-разному реализуют таблицы, и блок умеет адаптироваться к этим различиям.
- Определение границ – вычисляет, сколько в таблице строк и столбцов, есть ли заголовки, объединённые ячейки.
- Извлечение данных – проходит по всем ячейкам и считывает их содержимое (текст, числа, состояния чекбоксов).
- Формирование результата – упаковывает извлечённые данные в структурированный формат, готовый для дальнейшей обработки.

Параметры блока

1. Индекс строки и Индекс столбца

Эти параметры определяют, что именно извлекать из таблицы:

- Если указаны оба индекса – блок возвращает значение только из одной ячейки на пересечении указанных строки и столбца.
- Если оставить оба параметра пустыми (значение "ничто") – блок возвращает всю таблицу целиком.
- Если указан только один индекс – поведение зависит от реализации (обычно возвращается строка или столбец).

Важно: Индексы в таблицах обычно начинаются с 0 (нулевая строка – первая строка данных, нулевой столбец – первый столбец). Если таблица имеет заголовки, они часто не входят в данные и имеют отдельную индексацию.

2. Получить размерность таблицы

Когда этот чекбокс включён, блок не извлекает данные из ячеек, а только определяет структуру таблицы:

- Количество строк – сколько строк данных содержит таблица
- Количество столбцов – сколько столбцов содержит таблица

Это полезно, когда нужно:

- Оценить объём данных перед обработкой
- Проверить, что таблица не пустая
- Организовать цикл по всем строкам

3. Использовать объект элемента

Позволяет передать в блок не просто селектор, а уже готовый объект элемента, полученный ранее:

- Через Мастер UI
- Из переменной
- Как результат другого блока (например, поиска элементов)

4. Выбор элемента через Мастер UI

Как и все блоки десктопной автоматизации, этот блок интегрирован с Мастером UI. Достаточно один раз указать на таблицу мышкой, и Мастер UI сохранит точный путь к ней, который можно использовать в любом сценарии.

Профессиональные советы:

1.Всегда проверяйте размерность перед обработкой. Это защитит от ситуаций, когда таблица вдруг оказалась пустой.

2. Для очень больших таблиц используйте пакетную обработку. Например, получайте по 100 строк, обрабатывайте, потом следующие 100.

3. Сохраняйте заголовки отдельно. Если нужны заголовки столбцов, получите их как отдельную строку (обычно это строка с индексом 0).

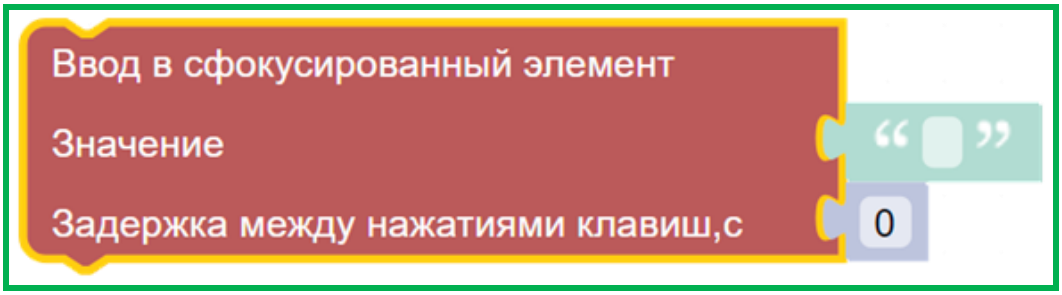
4. Учитывайте типы данных. Блок возвращает текст, даже если в ячейке число. При необходимости преобразуйте строки в числа.

5. Тестируйте на разных объёмах. Убедитесь, что блок одинаково хорошо работает и с одной строкой, и с тысячей.

Блок «Получить табличные данные» – это мост между визуальным табличным интерфейсом и структурированными данными, с которыми умеет работать

робот. Он превращает хаотичное нагромождение строк и столбцов на экране в аккуратную, готовую к обработке структуру.

2. Блок «Ввод в сфокусированный элемент»



Блок «Ввод в сфокусированный элемент» осуществляет ввод указанного текста в элемент интерфейса, который в данный момент уже имеет фокус ввода. Это принципиально иной подход к вводу данных, чем у обычного блока «Ввод в приложение».

Если обычный ввод можно сравнить с ситуацией, когда вы говорите человеку: «Возьми ручку, подойди к нужному полю в анкете и напиши там текст», то ввод в сфокусированный элемент – это: «Видишь, где сейчас мигает курсор? Просто печатай».

Ключевое отличие: поиск vs доверие

Обычный «Ввод в приложение»	«Ввод в сфокусированный элемент»
Находит указанный элемент	Не ищет никакой элемент
Проверяет видимость и доступность	Не проверяет ничего
При необходимости устанавливает фокус	Доверяет, что фокус уже установлен
Гарантирует, что текст попадёт в нужное место	Текст попадёт туда, где сейчас курсор
Безопаснее, но медленнее	Быстрее, но требует уверенности в фокусе

Блок выполняет только одно действие – эмулирует набор текста на клавиатуре. Он не пытается:

- Найти элемент
- Проверить его существование
- Активировать окно
- Установить фокус

Он просто «печатает» текст в текущую позицию курсора.

Параметры блока

1. Текст для ввода

Строка, которую нужно ввести. Может содержать:

- Обычные символы (буквы, цифры)
- Специальные символы
- Пробелы и знаки препинания

2. Задержка между нажатиями клавиш

Этот параметр определяет паузу (в секундах) между вводом каждого символа:

- 0 – все символы вводятся максимально быстро (рекомендуется для большинства случаев)

- >0 – имитация медленного набора текста (полезно, если приложение не успевает обрабатывать быстрый ввод)

Преимущества использования:

- Скорость – не тратится время на поиск элемента и проверки
- Простота – не нужно настраивать сложные селекторы
- Надёжность в контексте – если фокус точно установлен, текст попадёт куда нужно
- Работа с "невидимыми" элементами – если элемент не ловится инспектором, но фокус в него можно поставить

Профессиональные советы:

- Используйте этот блок только там, где вы на 100% уверены в фокусе. Лучше потратить секунду на явную установку фокуса, чем потом час искать, почему текст ушёл не туда.
- Для критических данных всегда делайте проверку. После ввода прочитайте значение и убедитесь, что оно совпадает с отправленным.
- Комбинируйте с перемещением фокуса для навигации. Это быстрее, чем искать каждое поле.
- В длинных сценариях периодически "подстраховывайтесь" явной активацией окна. Фокус может потеряться из-за системных событий.
- При отладке временно включайте задержку, чтобы видеть, куда реально вводится текст.

Блок «Ввод в сфокусированный элемент» – это инструмент для опытных разработчиков. Он даёт прирост в скорости и упрощает код, но требует глубокого понимания того, как управляется фокус в приложении.

3. Блок «Перемещение фокуса»



Блок «Перемещение фокуса» предназначен для программного перевода фокуса ввода на указанный UI-элемент десктопного приложения. В отличие от навигации с помощью клавиши Tab (которая перемещает фокус последовательно), этот блок позволяет установить фокус сразу на нужный элемент, минуя промежуточные.

Прежде чем углубляться в работу блока, важно понять, что такое фокус ввода.

Фокус ввода – это состояние элемента интерфейса, означающее, что он готов принимать ввод с клавиатуры. Когда поле ввода имеет фокус:

- В нём мигает курсор

- Нажатия клавиш отправляются именно в это поле
- Обычно элемент визуально выделяется (рамкой, подсветкой)
- Фокус может быть только у одного элемента в каждый момент времени. Это как микрофон на конференции – говорить может только тот, у кого микрофон.

Блок выполняет две потенциальные задачи:

- Переводит фокус ввода на указанный элемент (основная задача)
- Опционально перемещает курсор мыши к этому элементу

Параметры блока

1. Фокус на элементе

Этот чекбокс определяет, будет ли элемент активирован и получит ли он фокус ввода.

- Включен (True) – элемент получает фокус ввода. Курсор начинает мигать в поле (если это поле ввода), элемент становится активным.
- Выключен (False) – фокус не меняется. Этот режим используется, если нужно только переместить мышь, но не менять фокус.

2. Переместить курсор мыши

Этот чекбокс определяет, будет ли курсор мыши перемещён к указанному элементу.

- Включен (True) – мышь перемещается к элементу (обычно в его центр)
- Выключен (False) – мышь остаётся на месте, меняется только фокус ввода

3. Использовать объект элемента

Позволяет передать в блок не селектор, а готовый объект элемента, полученный ранее:

- Через Мастер UI
- Из переменной
- Как результат другого блока (например, поиска элементов)

Преимущества использования:

- Скорость – мгновенный переход к нужному элементу без последовательной навигации
- Точность – всегда попадаем точно в целевой элемент
- Гибкость – можно отделить установку фокуса от перемещения мыши
- Неинвазивность – в отличие от клика, не вызывает нежелательных действий (не нажимает кнопки, не меняет состояние чекбоксов)
- Экономия действий – не нужно делать лишний клик перед вводом

Профессиональные советы:

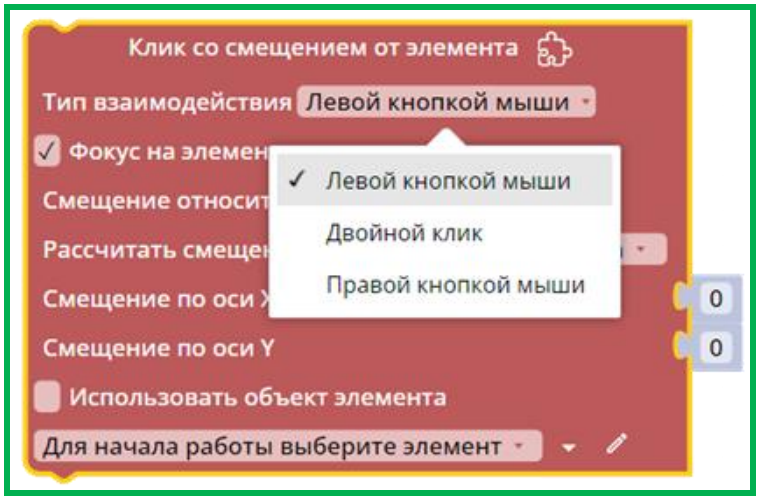
- Для полей ввода используйте комбинацию: перемещение фокуса + ввод в сфокусированный элемент. Это быстрее и чище, чем клик + ввод.
- Не перемещайте мышь без необходимости. Лишние движения могут вызвать нежелательные эффекты и замедляют работу.
- Перед перемещением фокуса убедитесь, что окно активно. Фокус нельзя установить в окно, которое неактивно.

- Для элементов, которые требуют клика для активации, используйте клик, а не фокус. Некоторые элементы (например, кнопки) активируются только по клику, фокус им не поможет.
- При работе с таблицами и списками проверяйте, что элемент видим. На невидимый элемент фокус не перевести.

Блок «Перемещение фокуса» – это элегантный инструмент для управления активным элементом интерфейса. Он занимает нишу между навигацией по Tab (слишком медленной) и кликами (избыточными, когда нужно просто подготовить поле).

Главное преимущество блока – разделение понятий: "сделать элемент активным" и "выполнить над ним действие". Это позволяет писать более чистые и эффективные сценарии, где подготовительные операции отделены от основных.

4. Блок «Клик со смещением от элемента»



Блок «Клик со смещением от элемента» выполняет клик не по самому элементу, а в точке, сдвинутой относительно него на заданное расстояние. Это как если бы вы хотели нажать не на дверной звонок, а на стену рядом с ним – возможно, именно там спрятана кнопка вызова лифта.

Стандартный блок «Клик по элементу» обычно нажимает в центр элемента. Этого достаточно для 90% задач. Но остаются 10%, где требуется ювелирная точность:

- Выпадающие списки – иногда нужно кликнуть не на само поле, а на иконку стрелочки справа
- Ползунки – нужно схватить ползунок не за центр, а за край, чтобы перетащить
- Календари – нужно попасть в конкретную дату, а кнопка "сегодня" может быть в углу
- Чекбоксы с подписями – иногда клик по тексту не отмечает чекбокс, нужно кликнуть именно в квадратик
- Нестандартные интерфейсы – игры, графические редакторы, CAD-системы

Блок выполняет последовательность действий:

- Находит указанный элемент на экране
- Определяет его координаты и размеры

- Вычисляет опорную точку согласно выбранному типу смещения (центр, угол)
- Смещает эту точку на заданное расстояние по X и Y
- Выполняет клик в полученных координатах

Параметры блока

1. Тип взаимодействия

Как и в обычном клике, можно выбрать:

Тип	Когда использовать
Клик левой кнопкой мыши	Стандартное нажатие (по умолчанию)
Двойной клик	Открытие файлов, выделение слова, активация ячеек
Правой кнопкой мыши	Вызов контекстного меню

2. Фокус на элементе

Определяет, нужно ли активировать элемент перед кликом:

- Включен – перед кликом элемент получит фокус
- Выключен – клик произойдёт без изменения фокуса

Для кликов со смещением эта опция особенно полезна, если элемент должен быть активным для корректной обработки клика.

3. Тип смещения (откуда отсчитываем)

Задаёт опорную точку, относительно которой будет выполняться смещение:

Тип смещения	Описание	Пример использования
Центр	От центра элемента	Нужно кликнуть чуть выше центра
Левый верхний угол	От левого верхнего угла	Иконка в правом верхнем углу элемента
Правый верхний угол	От правого верхнего угла	Кнопка закрытия в углу окна
Левый нижний угол	От левого нижнего угла	Статусная строка внизу элемента
Правый нижний угол	От правого нижнего угла	Ползунок изменения размера

4. Расчёт смещения (в чём измеряем)

Определяет, как интерпретировать значения смещения:

Режим	Описание	Когда использовать
Пиксели	Смещение в абсолютных единицах (точках экрана)	Когда нужно точное расстояние независимо от размера элемента
Проценты	Смещение в процентах от размера элемента	Когда элемент может менять размер, а пропорция важна

5. Смещение по оси X и Y

Числовые значения, определяющие, на сколько сдвинуться от опорной точки:

- Положительные значения – вправо (для X) или вниз (для Y)
- Отрицательные значения – влево (для X) или вверх (для Y)

Типичные ошибки и их решения:

Ошибка	Причина	Решение
Клик происходит не там	Неправильно выбран тип смещения	Перепроверить, от какой точки считаем
На разных размерах окна клик смещается	Использованы пиксели для адаптивного интерфейса	Переключиться на проценты
Клик не срабатывает	Сместились слишком далеко от активной зоны	Уменьшить смещение

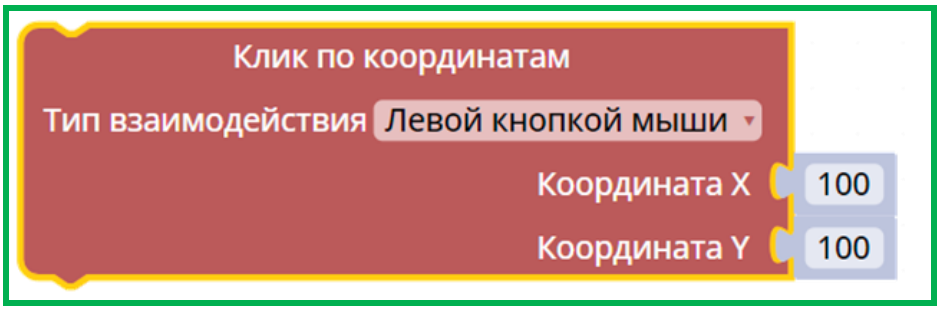
Ошибка	Причина	Решение
Элемент не находится	Забыли указать элемент или он изменился	Проверить селектор
Двойной клик не работает как двойной	Слишком большой интервал	Уменьшить задержку между кликами

Блок «Клик со смещением от элемента» – это инструмент для ювелирной работы. Там, где обычный клик работает как молоток, этот блок работает как скальпель.

Он позволяет:

- Взаимодействовать с нестандартными элементами
- Обходить ограничения приложений
- Достигать точности, недоступной обычным кликам
- Адаптироваться к изменяющимся размерам интерфейса

5. Блок «Клик по координатам»



Блок «Клик по координатам» выполняет клик в абсолютной точке на экране, заданной координатами X и Y. В отличие от всех предыдущих блоков, которые искали элементы и кликали по ним, этот блок ничего не ищет – он просто посылает сигнал мыши в указанную точку экрана, независимо от того, что там находится.

Важно понимать, как устроена система координат в экране компьютера:

- Начало координат (0,0) – левый верхний угол экрана
- Ось X (горизонтальная) – увеличивается при движении вправо
- Ось Y (вертикальная) – увеличивается при движении вниз

Таким образом:

- Точка (0,0) – левый верхний угол
- Точка (1920, 1080) – правый нижний угол при разрешении Full HD

Параметры блока

1. Координата X

Горизонтальная позиция в пикселях от левого края экрана.

- 0 – левый край
- Положительные значения – смещение вправо

2. Координата Y

Вертикальная позиция в пикселях от верхнего края экрана.

- 0 – верхний край
- Положительные значения – смещение вниз

3. Тип взаимодействия

Как и в других кликах, можно выбрать:

Тип	Когда использовать
Клик левой кнопкой мыши	Стандартное нажатие (по умолчанию)
Двойной клик	Открытие файлов, активация
Правой кнопкой мыши	Вызов контекстного меню

Прежде чем углубляться в возможности этого блока, нужно чётко понять одно важное правило:

Клик по координатам – это крайнее средство. Используйте его только тогда, когда все остальные способы исчерпаны.

Координаты ненадёжны, потому что:

- Окно можно передвинуть
- Разрешение экрана может измениться
- Элементы могут сместиться при обновлении приложения
- Размеры окон могут быть разными
- DPI-масштабирование меняет всё

Профессиональные советы:

- Всегда комментируйте, почему вы используете координаты. Напишите прямо в названии блока или в комментарии: "Клик по кнопке, которая не ловится инспектором".
- Используйте переменные для координат. Вместо жёстко заданных чисел вычислите их в начале сценария.
- Делайте несколько попыток с небольшими смещениями. Если элемент может быть чуть в другом месте, попробуйте кликнуть в (X, Y), потом в (X+5, Y), потом в (X-5, Y).
- Логируйте координаты перед кликом. При отладке это поможет понять, что пошло не так.

Блок «Клик по координатам» – это как аварийный люк в самолёте. Вы должны знать, что он есть, уметь им пользоваться, но надеяться, что никогда не придётся.

В идеальном мире RPA этот блок не нужен. Все элементы прекрасно находятся, все кнопки имеют уникальные идентификаторы, все приложения дружелюбны к автоматизации. Но мы живём не в идеальном мире.

6. Блок «Перемещение drag-and-drop»



Блок «Перемещение drag-and-drop» позволяет роботу выполнять перетаскивание объектов – одно из самых сложных для автоматизации действий пользователя. Он имитирует полный цикл: захват объекта в исходной точке, перемещение его в целевую позицию и отпускание.

В компьютерном мире drag-and-drop (перетащить и бросить) встречается повсюду:

- Перемещение файлов между папками
- Сортировка элементов в списках
- Изменение порядка в конструкторах
- Перетаскивание объектов в графических редакторах
- Настройка интерфейсов (перемещение панелей)
- Игры (например, пасьянс «Косынка»)

Блок выполняет последовательность действий, невидимую для разработчика:

- Перемещение к исходной точке – курсор мыши перемещается в начальные координаты
- Захват – нажатие и удержание левой кнопки мыши
- Перемещение к целевой точке – курсор с зажатой кнопкой перемещается в конечные координаты (по прямой или с заданной траекторией)
- Отпускание – кнопка мыши отпускается, объект «бросается» в новом месте

Параметры блока

1. Начальные координаты (X1, Y1)

Точка, где начинается перетаскивание. Обычно это координаты объекта, который нужно захватить.

- X1 – горизонтальная позиция от левого края экрана
- Y1 – вертикальная позиция от верхнего края экрана

2. Конечные координаты (X2, Y2)

Точка, куда нужно переместить объект.

- X2 – горизонтальная позиция целевой точки
- Y2 – вертикальная позиция целевой точки

3. Тип данных

Координаты указываются как целые числа (пиксели).

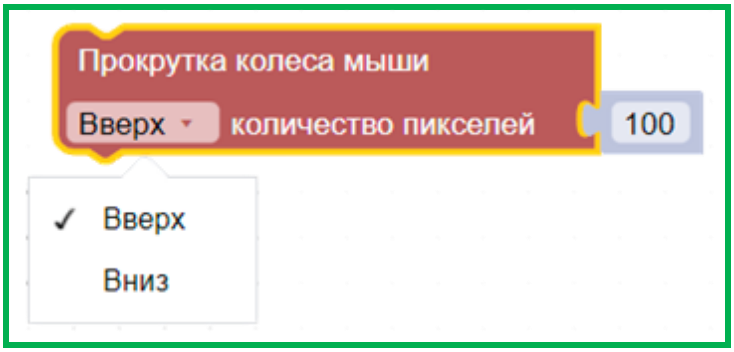
Профессиональные советы:

- Всегда захватывайте объект за центр, если нет особых причин поступать иначе. Это наиболее предсказуемый способ.
- Для точных перемещений используйте несколько этапов. Сначала переместите объект близко к цели, потом точно позиционируйте.
- Проверяйте результат. После перетаскивания убедитесь, что объект действительно оказался там, где нужно.
- Учитывайте анимацию. Некоторые приложения после перетаскивания показывают анимацию, во время которой нельзя ничего делать.
- Для сложных случаев разбивайте на действия. Вместо одного блока выполните: клик, задержка, перемещение, задержка, отпускание.

Блок «Перемещение drag-and-drop» – это один из самых сложных, но и самых мощных инструментов в арсенале RPA-разработчика. Он позволяет автоматизировать действия, которые обычно доступны только человеку: сортировку, организацию, компоновку.

Этот блок превращает робота из простого «нажимателя кнопок» в полноценного оператора, способного управлять интерфейсом наравне с человеком. С его помощью можно автоматизировать даже те задачи, которые кажутся слишком сложными для программной реализации.

7. Блок «Прокрутка колеса мыши»



Блок «Прокрутка колеса мыши» имитирует вращение колёсика мыши, позволяя роботу прокручивать содержимое окон, списков, документов и веб-страниц. Это базовое действие, которое каждый пользователь выполняет десятки раз в день, но для робота оно открывает доступ к контенту, скрытому за границами экрана.

Блок эмулирует вращение колеса мыши, что вызывает стандартную реакцию приложения:

- Прокрутка вверх – содержимое движется вверх, показывая то, что было выше
- Прокрутка вниз – содержимое движется вниз, показывая то, что было ниже

Важное требование. Перед прокруткой необходимо установить фокус в элемент, внутри которого будет осуществляться прокрутка.

Это критически важно! Если фокус будет в другом месте, прокручиваться будет совсем не то, что нужно. Например, если вы хотите прокрутить список клиентов, но фокус в поле поиска, то прокручиваться будет поле поиска (если оно вообще поддерживает прокрутку).

Параметры блока:

1. Направление прокрутки

- Вверх – колесо вращается от себя (содержимое движется вверх)
- Вниз – колесо вращается на себя (содержимое движется вниз)

2. Количество пикселей или "щелчков"

Блок позволяет указать, на сколько прокрутить. Обычно это количество "щелчков" колеса:

- 1 щелчок = стандартный шаг прокрутки (обычно 3 строки текста или около 30-50 пикселей)
- Можно указать любое целое число: 5, 10, 100 щелчков

3. Фокус на элементе (подготовительный шаг)

Хотя это не параметр самого блока, перед прокруткой нужно явно установить фокус.

Профессиональные советы:

- Всегда начинайте с проверки, есть ли куда прокручивать. Если список короткий, лишняя прокрутка не повредит, но потеряет время.
- Используйте прокрутку маленькими шагами. 5 щелчков, проверка, ещё 5 щелчков – так надёжнее, чем 50 щелчков сразу.
- Для бесконечных лент используйте цикл с условием. Прокручивайте, пока не найдёте нужное или пока не перестанет появляться новый контент.
- Логируйте, сколько раз прокрутили. Это поможет при отладке.
- Помните про "эффект аккордеона". Некоторые списки при прокрутке могут сжиматься или разворачивать элементы – учитывайте это в расчётах.

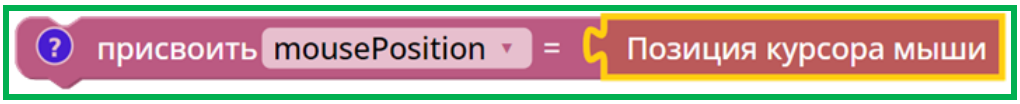
Блок «Прокрутка колеса мыши» – это ключ к контенту, который не помещается на экране. Он позволяет роботу видеть больше, чем открыто в данный момент, и добираться до элементов, скрытых в глубине интерфейса.

В сочетании с поиском элементов и циклами этот блок превращается в мощный инструмент навигации, способный:

- Прочёсывать длинные списки в поисках нужной записи
- Собирать данные из многостраничных документов
- Работать с современными интерфейсами, построенными на бесконечной прокрутке

Это не самый сложный, но один из самых полезных блоков в повседневной работе RPA-разработчика. Умение грамотно им пользоваться отличает профессионала от новичка.

8. Блок «Позиция курсора мыши»



Блок «Позиция курсора мыши» – это уникальный инструмент, который не управляет мышью, а получает от неё информацию. Он возвращает текущие координаты курсора на экране, позволяя роботу «знать», где в данный момент находится мышь.

Что возвращает блок

Блок возвращает словарь (объект) с двумя ключами:

```
{
  "X": 500,
  "Y": 300
}
```

- X – горизонтальная координата в пикселях от левого края экрана
- Y – вертикальная координата в пикселях от верхнего края экрана

Эти значения можно сохранить в переменную и использовать в дальнейшем.

Профессиональные советы:

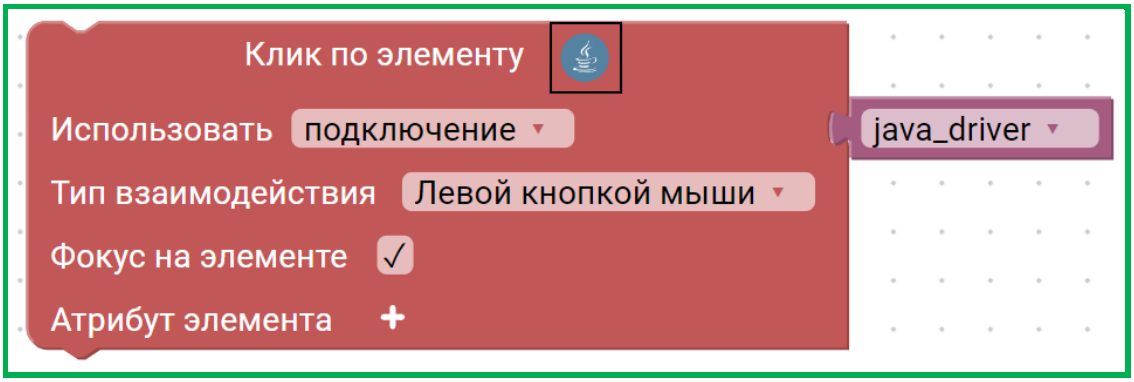
- Используйте этот блок для отладки. Вставляйте его после сложных перемещений и записывайте координаты в лог.
- Никогда не предполагайте, что мышь осталась там, где вы её оставили. Пользователь или другой процесс могли её сдвинуть.
- Сохраняйте позицию в начале и восстанавливайте в конце. Это вежливо по отношению к пользователю.
- Для относительных перемещений всегда запрашивайте текущую позицию. Не надейтесь, что помните её из прошлого шага.
- Учитывайте многомониторные конфигурации. Ваш робот может работать на разных компьютерах.

Блок «Позиция курсора мыши» – это редкий пример инструмента, который не управляет, а наблюдает. Он даёт роботу обратную связь, позволяя:

- Контролировать результаты своих действий
- Адаптироваться к изменяющимся условиям
- Обнаруживать аномалии
- Вежливо возвращать мышь на место после работы

В программировании это называется "сенсор" – устройство, которое получает информацию из внешнего мира. Без сенсоров робот слеп и вынужден действовать наугад. С этим блоком он обретает способность видеть, пусть и в очень узком смысле – видеть положение собственной мыши.

9. Блок «Java-инспектор»



Java-инспектор – это специализированный инструмент для анализа и исследования Java-приложений, к которым невозможно применить стандартные средства десктопной автоматизации. Он позволяет «заглянуть внутрь» Java-интерфейса и увидеть его структуру так же, как Мастер UI показывает структуру обычных Windows-приложений.

Java-приложения живут по своим законам. Они не используют стандартные механизмы Windows для отрисовки интерфейсов, поэтому обычные инструменты автоматизации часто видят только одно большое окно, но не видят кнопок, полей и таблиц внутри него.

Java-инспектор решает эту проблему, подключаясь к Java-приложению через специальный мост (Java Access Bridge) и показывая:

- Иерархию элементов – какие окна, панели, кнопки и поля есть в приложении
- Свойства каждого элемента – имя, тип, состояние, доступность

- Отношения между элементами – кто чей родитель, кто внутри кого находится

Java-инспектор открывается кнопкой на блоках из раздела «Java-приложения» в палитре компонентов Puzzle RPA.

Важное отличие: расширение

Java-инспектор не входит в стандартную поставку Puzzle RPA. Это расширение, которое нужно установить отдельно.

Как работать с Java-инспектором

Шаг 1. Запустите Java-приложение

Откройте приложение, которое нужно автоматизировать. Убедитесь, что оно работает и видно на экране.

Шаг 2. Откройте Java-инспектор

Нажмите кнопку Java-инспектора в студии Puzzle RPA. Откроется окно инспектора.

Шаг 3. Подключитесь к приложению

Инспектор обычно автоматически обнаруживает запущенные Java-приложения. Если приложение не появилось в списке, нажмите Refresh.

Шаг 4. Исследуйте структуру

Изучите дерево элементов, чтобы понять, как устроен интерфейс:

- Где находится нужная кнопка?
- Какой путь к ней?
- Какие у неё атрибуты?

Шаг 5. Найдите нужный элемент

Используйте кнопку Find Component:

- Нажмите и удерживайте кнопку
- Наведите курсор на интересующий элемент в приложении
- Элемент подсветится зелёным
- Отпустите кнопку – в дереве автоматически выделится соответствующий узел

Шаг 6. Сохраните информацию

Запишите:

- Путь к элементу в иерархии
- Уникальные атрибуты (Name, Role, индекс)
- Тип элемента

Эти данные понадобятся для настройки блоков автоматизации Java-приложений

Java-инспектор – это мост между миром Java-приложений и миром RPA. Без него Java остаётся «чёрным ящиком», с которым невозможно работать. С ним Java-приложения становятся такими же доступными для автоматизации, как и обычные Windows-программы.

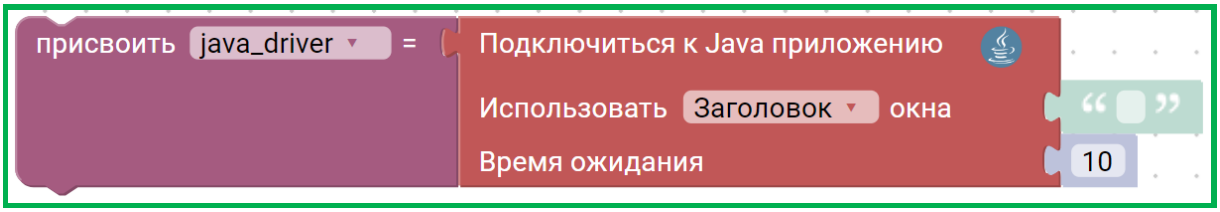
Этот инструмент незаменим, когда сталкиваешься с:

- SAP-клиентами на Java

- Старыми корпоративными системами, написанными на Java
- Специализированным ПО, которое использует Java-интерфейсы

Установите расширение, включите Java Access Bridge, и мир Java-автоматизации откроется перед вами.

10. Блок «Подключиться к Java-приложению»



Блок «Подключиться к Java-приложению» выполняет установку соединения с работающим Java-приложением, делая возможным дальнейшее взаимодействие с его элементами. Это обязательный первый шаг перед использованием любых других блоков для Java-автоматизации.

Без этого блока Java-приложение остаётся «невидимым» для робота. С ним между роботом и Java-приложением устанавливается мост, по которому пойдут все команды и запросы.

Как работает:

- Указываем цель – идентификатор приложения (заголовок окна, HWND или VMID)
- Задаём таймаут – сколько секунд ждать подключения
- Блок ищет приложение – среди запущенных Java-программ
- Устанавливает соединение – робот получает доступ к дереву элементов

Что получаем на выходе:

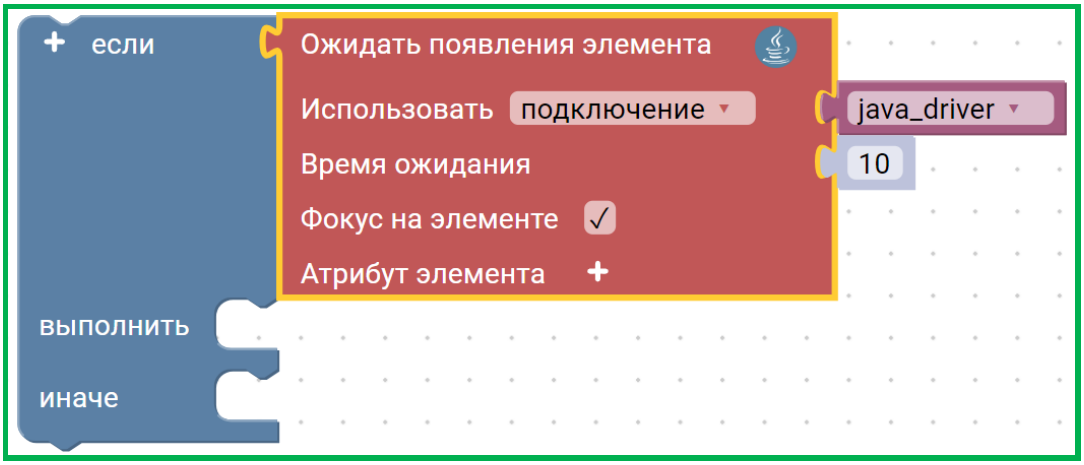
- Постоянное соединение с приложением
- Возможность использовать все остальные Java-блоки (клики, ввод, ожидание, поиск)
- Доступ к иерархии элементов интерфейса

Важно:

- Блок устанавливается как расширение (не входит в базовую поставку)
- Требует включённого Java Access Bridge в системе
- Выполняется строго первым перед любыми другими Java-блоками
- При работе с несколькими Java-приложениями нужно переподключаться к каждому

Блок «Подключиться к Java-приложению» – это входной билет в мир Java-автоматизации. Без него все остальные Java-блоки бесполезны. С ним Java-приложения становятся доступными для робота так же, как обычные Windows-приложения.

11. Блок «Ожидать появление элемента (Java)»



Специализированный блок для Java-приложений, который приостанавливает выполнение сценария до момента появления указанного элемента на экране. Аналог обычного «Ожидать появления элемента», но для Java-мира.

Как работает:

- Указываем элемент – задаём один или несколько атрибутов (name, role, index и др.), по которым будет найден элемент
- Задаём таймаут – сколько секунд ждать появления
- Выбираем фокус – нужно ли активировать элемент после появления
- Блок ждёт – пока элемент не появится или не истечёт время

Важно:

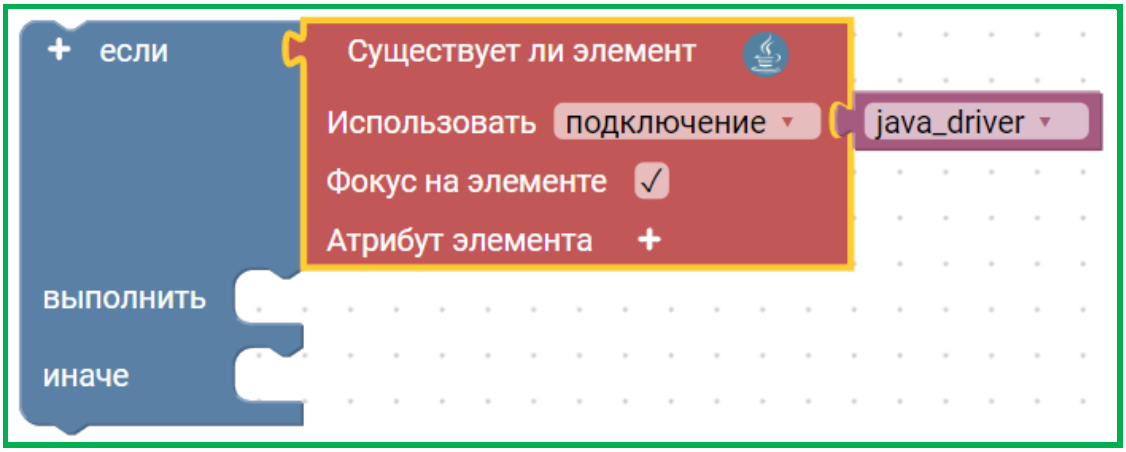
- Работает только после подключения к Java-приложению
- Атрибуты для поиска удобно получать через Java-инспектор
- Можно комбинировать несколько атрибутов для точной идентификации

Зачем нужен:

- Синхронизация робота с загрузкой Java-интерфейса
- Ожидание появления модальных окон
- Гарантия, что элемент готов к взаимодействию

Блок «Ожидать появление элемента (Java)» – это специализированный инструмент для синхронизации робота с Java-приложением, который приостанавливает выполнение сценария до момента появления указанного элемента на экране. Он работает только после подключения к Java-приложению, позволяет задавать несколько атрибутов для точной идентификации элемента (полученных через Java-инспектор) и настраивать таймаут ожидания, гарантируя, что элемент готов к взаимодействию, прежде чем робот продолжит работу.

12. Блок «Существует ли элемент (Java)»



Блок «Существует ли элемент (Java)» выполняет мгновенную проверку наличия указанного элемента в интерфейсе Java-приложения. В отличие от блока ожидания, он не приостанавливает сценарий, а сразу возвращает результат – True (элемент есть) или False (элемент отсутствует).

Как работает:

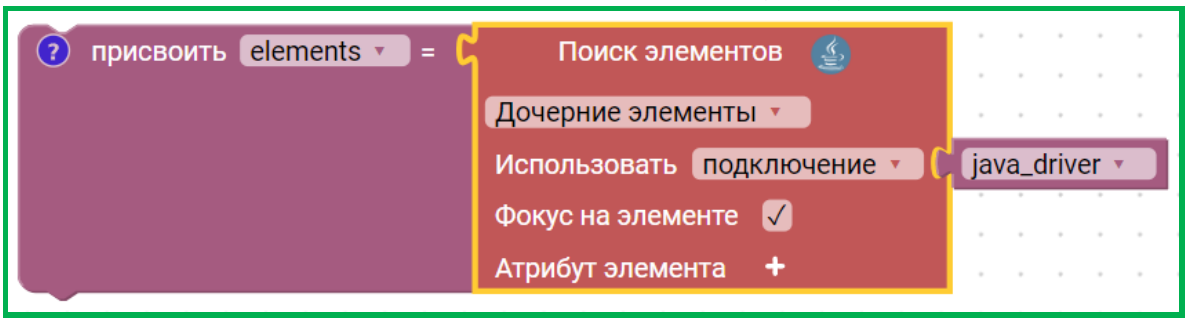
- Указываем элемент – задаём один или несколько атрибутов (name, role, index) для идентификации
- Выбираем фокус – опционально активируем элемент, если он найден
- Блок проверяет – ищет элемент в текущем состоянии интерфейса
- Возвращает результат – мгновенно, без ожидания

Важно:

- Работает только после подключения к Java-приложению
- Не ждёт появления элемента – проверяет здесь и сейчас
- Идеален для ветвления логики (конструкции «Если – Иначе»)
- Атрибуты удобно получать через Java-инспектор

Блок «Существует ли элемент (Java)» – это инструмент для мгновенной проверки наличия элемента в Java-приложении без ожидания. Он работает только после подключения к приложению и возвращает логический результат: True (элемент найден) или False (отсутствует). Блок незаменим для построения ветвящейся логики – например, чтобы проверить, появилось ли сообщение об ошибке, активна ли нужная форма или загрузились ли данные, и на основе этого принять решение о дальнейших действиях. Атрибуты для точной идентификации элемента удобно получать через Java-инспектор.

13. Блок «Поиск элементов (Java)»



Блок «Поиск элементов (Java)» предназначен для получения коллекции UI-элементов Java-приложения, соответствующих заданным критериям. В отличие от обычного поиска, который находит один элемент, этот блок возвращает список всех найденных элементов, открывая возможности для массовой обработки.

Как работает:

- Выбираем режим иерархии:
  - ✓ Дочерние элементы – все элементы внутри указанного родителя
  - ✓ Родительский элемент – родитель для указанного элемента
  - ✓ Объект текущего элемента – сам элемент как объект
- Указываем атрибуты для идентификации (name, role, index и др.)
- Блок ищет все подходящие элементы
- Возвращает список объектов-элементов

Что получаем на выходе:

```
text
[
    <Element object at 0x000001A388544C50>,  # первый элемент
    <Element object at 0x000001A381248544>,  # второй элемент
    ...                                      # и так далее
]
```

Каждый элемент в списке – полноценный объект, с которым можно работать:

- Передавать в другие Java-блоки
- Извлекать свойства
- Выполнять клики
- Читать значения

Важно:

- Работает только после подключения к Java-приложению
- Может вернуть пустой список, если элементы не найдены (не ошибка!)
- Атрибуты удобно получать через Java-инспектор

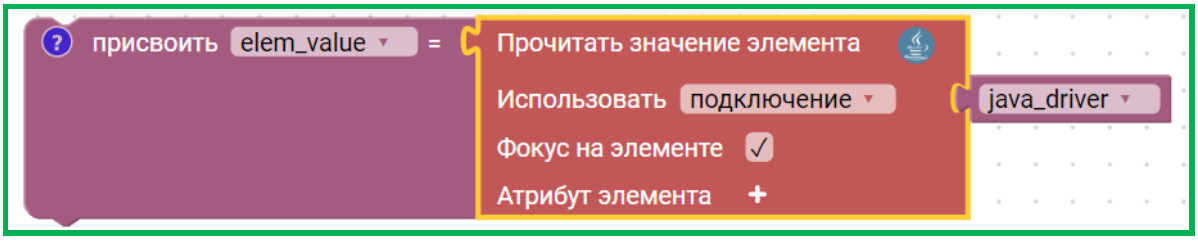
Где применяется:

- Получение всех строк таблицы для обработки
- Сбор всех кнопок на форме
- Работа со списками элементов
- Массовые операции (отметить все чекбоксы)
- Навигация по иерархии интерфейса

Блок «Поиск элементов (Java)» – это инструмент для получения коллекции UI-элементов Java-приложения, возвращающий список объектов, соответствующих заданным критериям поиска. Он позволяет работать не с одним элементом, а с группами: получать все дочерние элементы контейнера, находить родителя для указанного элемента или сохранять сам элемент как объект для дальнейшего использования. Результат работы блока – список объектов, которые можно перебирать в цикле, подсчитывать их количество, фильтровать или применять к каждому отдельные действия (клики, чтение свойств, ввод текста). Это незаменимый ин-

струмент при работе с таблицами, списками, коллекциями однотипных элементов и при необходимости массовой обработки интерфейса.

14. Блок «Прочитать значение элемента (Java)»



Блок «Прочитать значение элемента (Java)» извлекает текстовое содержимое из указанного элемента Java-приложения – то, что пользователь видит как текст или введённые данные. Это основной инструмент для получения информации из Java-интерфейсов.

Как работает:

- Указываем элемент – задаём один или несколько атрибутов (name, role, index) для идентификации
- Выбираем фокус – опционально активируем элемент перед чтением
- Блок находит элемент и извлекает его значение
- Возвращает строку с прочитанным текстом

Что можно читать:

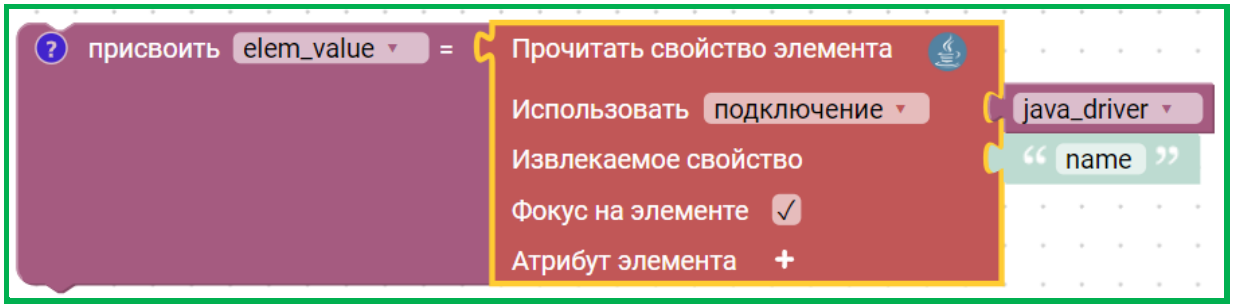
Тип элемента	Что возвращается
Текстовое поле	Введённый текст
Надпись (Label)	Текст надписи
Кнопка	Текст на кнопке
Заголовок окна	Текст заголовка
Ячейка таблицы	Содержимое ячейки
Список	Выбранный элемент

Важно:

- Работает только после подключения к Java-приложению
- Для некоторых элементов может потребоваться фокус для корректного чтения
- Атрибуты удобно получать через Java-инспектор

Блок «Прочитать значение элемента (Java)» – это инструмент для извлечения текстовой информации из элементов Java-приложений, возвращающий строковое содержимое поля ввода, надписи, кнопки, ячейки таблицы или любого другого элемента, отображающего текст. Он работает только после установки соединения с Java-приложением и позволяет точно идентифицировать целевой элемент по одному или нескольким атрибутам, полученным через Java-инспектор. Блок незаменим для сбора данных из форм, проверки введённых значений, чтения статусов и сообщений, а также для извлечения информации из таблиц и списков с целью дальнейшей обработки, анализа или передачи в другие системы.

15. Блок «Прочитать свойство элемента (Java)»



Блок «Прочитать свойство элемента (Java)» извлекает указанное техническое свойство элемента Java-приложения. В отличие от блока «Прочитать значение», который получает текстовое содержимое, этот блок даёт доступ ко всем атрибутам элемента – от его имени и роли до координат и состояния.

Как работает:

- Указываем элемент – задаём атрибуты для идентификации
- Выбираем свойство – указываем, какое именно свойство нужно прочитать (name, role, enabled, visible и т.д.)
- Блок находит элемент и извлекает значение запрошенного свойства
- Возвращает результат – строку, число, булево значение или объект в зависимости от свойства

Доступные свойства:

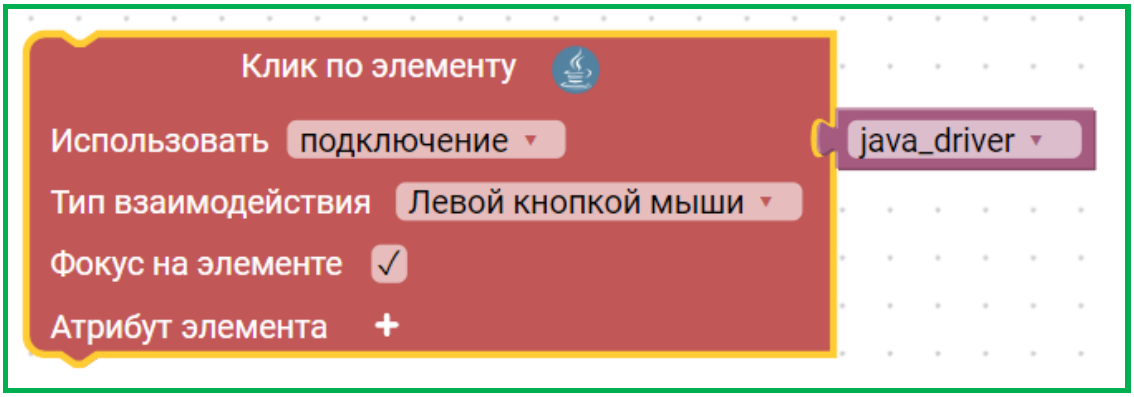
Категория	Свойства
Идентификация	name, role, control_type, description
Состояние	enabled, visible, showing, selected, checked, editable
Позиция	bounds, rectangle, element_center, bounding_rectangle
Иерархия	object_depth, children_count, index_in_parent
Технические	hwnd, vmid, accessible_context
Специальные	accessible_component, accessible_action, accessible_selection, accessible_text

Важно:

- Работает только после подключения к Java-приложению
- Позволяет читать любое доступное свойство элемента
- Атрибуты для поиска и свойства для чтения удобно получать через Java-инспектор

Блок «Прочитать свойство элемента (Java)» – это инструмент для получения технических атрибутов элементов Java-приложений, таких как имя, роль, состояние (активен/видим), координаты, положение в иерархии и многие другие характеристики. В отличие от чтения значения, этот блок даёт доступ к «паспортным данным» элемента – его идентификационным, пространственным и статусным свойствам. Он работает только после подключения к Java-приложению и позволяет гибко настраивать как сам искомый элемент (через атрибуты), так и извлекаемое свойство (из обширного списка доступных). Блок незаменим для проверки готовности элемента к взаимодействию, получения координат для позиционирования, определения типа элемента в универсальных сценариях и диагностики проблем при отладке.

16. Блок «Выполнить клик (Java)»



Блок «Выполнить клик (Java)» предназначен для программного нажатия на элементы Java-приложений. Он имитирует различные типы взаимодействия: от простого щелчка мышью до специальных действий, таких как вызов элемента (Invoke) или управление раскрытием/сворачиванием компонентов.

Как работает:

- Указываем элемент – задаём один или несколько атрибутов для идентификации (name, role, index и др.)
- Выбираем тип взаимодействия – определяем, какое действие выполнить
- Указываем фокус – нужно ли активировать элемент перед кликом
- Блок находит элемент и выполняет выбранное действие

Типы взаимодействия:

Тип	Описание	Когда использовать
Левой кнопкой мыши	Обычный щелчок	Кнопки, поля, чекбоксы
Двойной клик	Два быстрых нажатия	Открытие файлов, активация ячеек
Правой кнопкой мыши	Вызов контекстного меню	Дополнительные опции
Средней кнопкой мыши	Нажатие колесика	Специфические действия
Вызвать элемент (Invoke)	Программный вызов без мыши	Максимальная скорость и надёжность
Развернуть элемент	Раскрытие свернутого узла	Деревья, списки, группы
Свернуть элемент	Сворачивание развёрнутого узла	Скрытие дочерних элементов
Выполнить действие по умолчанию	Основное действие элемента	Универсальный вариант

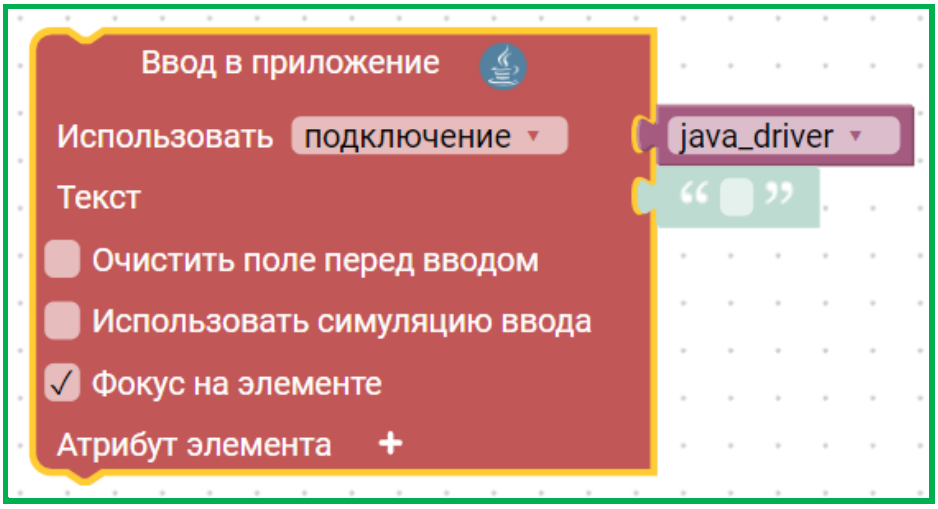
Важно:

- Работает только после подключения к Java-приложению
- Invoke – самый быстрый и надёжный способ для кнопок (не требует эмуляции мыши)
- Для контекстных меню обязателен правый клик
- Атрибуты удобно получать через Java-инспектор

Блок «Выполнить клик (Java)» – это универсальный инструмент для программного взаимодействия с элементами Java-приложений, поддерживающий широкий спектр действий: от стандартных кликов левой, правой и средней кнопкой мыши до специализированных операций, таких как двойной клик, программный вызов (Invoke), разворачивание и сворачивание элементов. Работает только

после установки соединения с Java-приложением и позволяет точно идентифицировать целевой элемент по одному или нескольким атрибутам, полученным через Java-инспектор. Блок незаменим для навигации по интерфейсу, активации кнопок и пунктов меню, работы с деревьями и списками, причём режим Invoke обеспечивает максимальную скорость и надёжность, не требуя эмуляции движений мыши.

17. Блок «Ввод в приложение (Java)»



Блок «Ввод в приложение (Java)» предназначен для передачи текстовых данных в поля ввода Java-приложений. Это основной инструмент для заполнения форм, ввода поисковых запросов и любых других действий, требующих набора текста.

Как работает:

- Указываем элемент – задаём атрибуты для идентификации поля ввода
- Задаём текст – строку, которую нужно ввести
- Настраиваем параметры:

Очистить поле перед вводом – удалить существующий текст

Использовать симуляцию ввода – эмулировать набор с клавиатуры (вместо прямой установки значения)

Фокус на элементе – активировать поле перед вводом

- Блок выполняет ввод – текст появляется в указанном поле

Параметры ввода:

- Очистить поле перед вводом

✓ True – поле будет очищено перед вводом нового текста

✓ False – новый текст добавится к существующему (или заменит выделенное)

- Использовать симуляцию ввода

✓ True – эмуляция набора с клавиатуры (посимвольно)

Нужно для полей с масками, автодополнением, триггерами на ввод

Медленнее, но реалистичнее

✓ False – прямая установка значения (программно)

Быстрее, но может не работать в сложных полях

- Фокус на элементе

True – поле активируется перед вводом

False – ввод без изменения фокуса (только для прямой установки)

Важно:

- Работает только после подключения к Java-приложению
- Атрибуты для поиска поля удобно получать через Java-инспектор
- Выбор режима ввода зависит от типа поля и требований приложения

Блок «Ввод в приложение (Java)» – это инструмент для передачи текстовых данных в поля ввода Java-приложений с гибкой настройкой режима ввода. Он позволяет как быстро устанавливать значение программно (прямая установка), так и реалистично имитировать набор текста с клавиатуры (симуляция) для полей с масками, автодополнением или специальными триггерами. Дополнительные параметры – очистка поля перед вводом и установка фокуса – обеспечивают полный контроль над процессом. Блок работает только после подключения к Java-приложению и требует точной идентификации целевого поля через атрибуты, полученные с помощью Java-инспектора. Это основной инструмент для заполнения форм, авторизации, поиска и любых других задач, связанных с вводом данных в Java-интерфейсах.

Подведение итогов практической работы

Мы завершили вторую практическую работу, которая стала логическим продолжением первого погружения в Desktop-автоматизацию. Если первая работа закладывала фундамент – базовые блоки, понимание принципов, знакомство с Мастером UI, – то вторая работа подняла нас на принципиально новый уровень.

Мы перешли от простых линейных сценариев к решению реальных бизнес-задач со всей их сложностью и непредсказуемостью. Мы научились не просто «нажимать кнопки», а проектировать архитектуру, адаптироваться к изменяющемуся интерфейсу, обрабатывать неожиданные ситуации и работать с приложениями на разных технологиях.

Что мы изучили:

1. Архитектура сложных RPA-решений

Мы поняли, что профессиональный робот – это не длинная цепочка блоков, а продуманная система с чёткими уровнями ответственности:

Мы научились проектировать модульных роботов, которые легко поддерживать, масштабировать и переиспользовать в разных проектах.

2. Жизненный цикл Desktop-элемента

Мы осознали, что элементы интерфейса не статичны – они рождаются, инициализируются, меняют состояние и исчезают.

3. Модальные окна

Мы изучили природу модальных окон – главного врага неопытных разработчиков. Мы научились создавать универсальные обработчики, которые не дают роботу «зависнуть» при появлении неожиданных диалогов.

4. Обработка исключительных ситуаций

Мы поняли, что ошибки – это не признак плохого кода, а неизбежная часть реальной жизни. Главное – уметь их обрабатывать.Мы научились строить отказоустойчивые сценарии, которые не падают при первом же сбое.

5. Продвинутые блоки Puzzle RPA

Мы освоили целый арсенал новых инструментов:

- Работа с таблицами
- Множественный поиск элементов – работа с коллекциями
- Продвинутое взаимодействие
- Java-автоматизация

Ключевые принципы, которые стали нашей основой:

- Модульность – любой сложный процесс можно разбить на простые компоненты
- Адаптивность – интерфейс живёт своей жизнью, робот должен к этому адаптироваться
- Бдительность – модальные окна могут появиться в любой момент
- Отказоустойчивость – не падать, а восстанавливаться
- Правильный выбор инструмента – для каждой задачи свой блок и свой режим работы

После выполнения практической работы мы умеем:

- Проектировать архитектуру сложных RPA-решений
- Работать с динамическими интерфейсами
- Обрабатывать модальные окна любых типов
- Строить отказоустойчивые сценарии
- Извлекать данные из таблиц
- Использовать продвинутые техники позиционирования
- Автоматизировать Java-приложения
- Комбинировать все изученные блоки для решения комплексных задач

Практическая работа № 8 завершает большой блок Desktop-автоматизации. Мы прошли путь от новичка, который учится нажимать кнопки, до профессионала, способного автоматизировать любые десктопные приложения – от старых учётных систем до современных Java-интерфейсов.

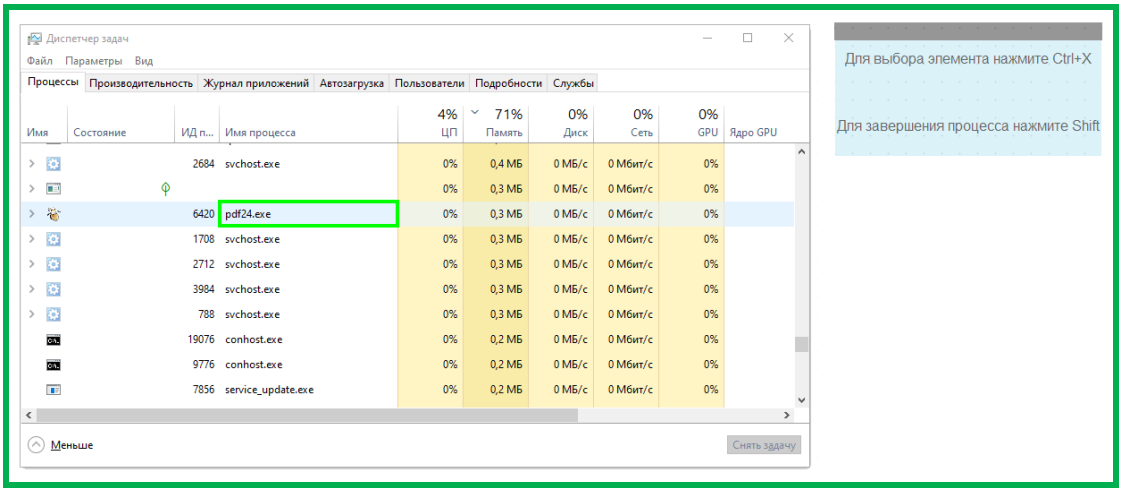
ПРИМЕРЫ ЗАДАЧ

Пример 1 – Получить табличные данные

В данном примере открывается **Диспетчер задач** и производится считывание табличных данных из него.

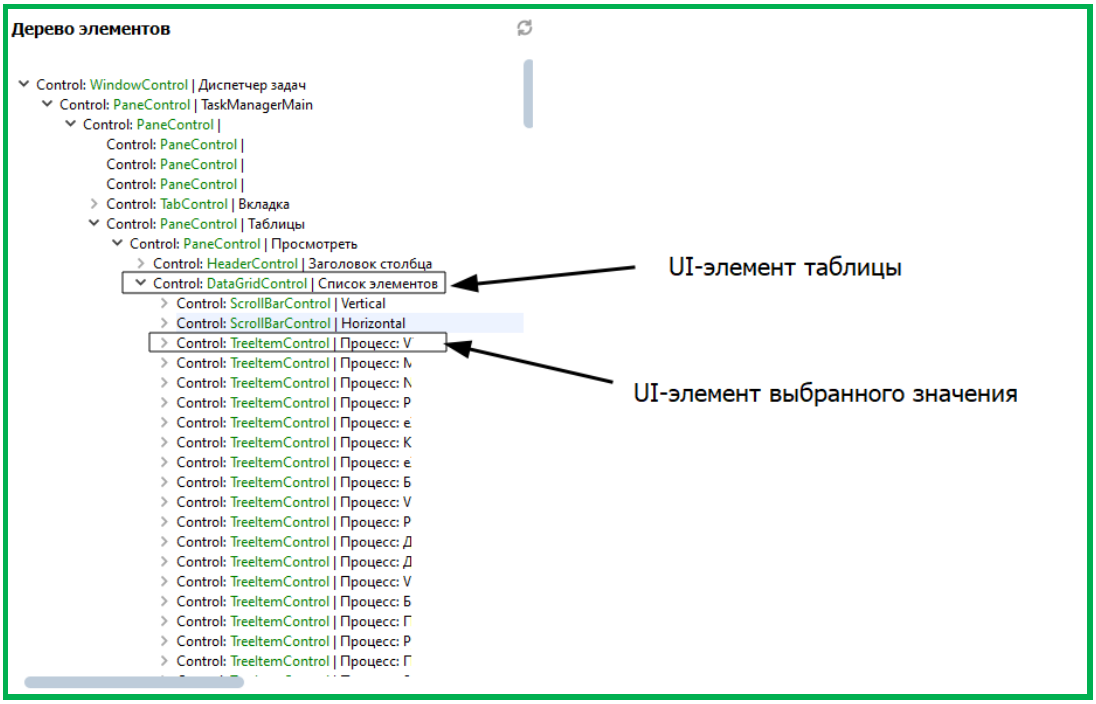
Для выбора элемента в блоке **Получить табличные данные** используется инструмент Master UI для десктопных приложений. Для этого:

1. Открывается инструмент **Master UI для десктопных приложений**;
2. Выполняется выбор любого элемента таблицы;



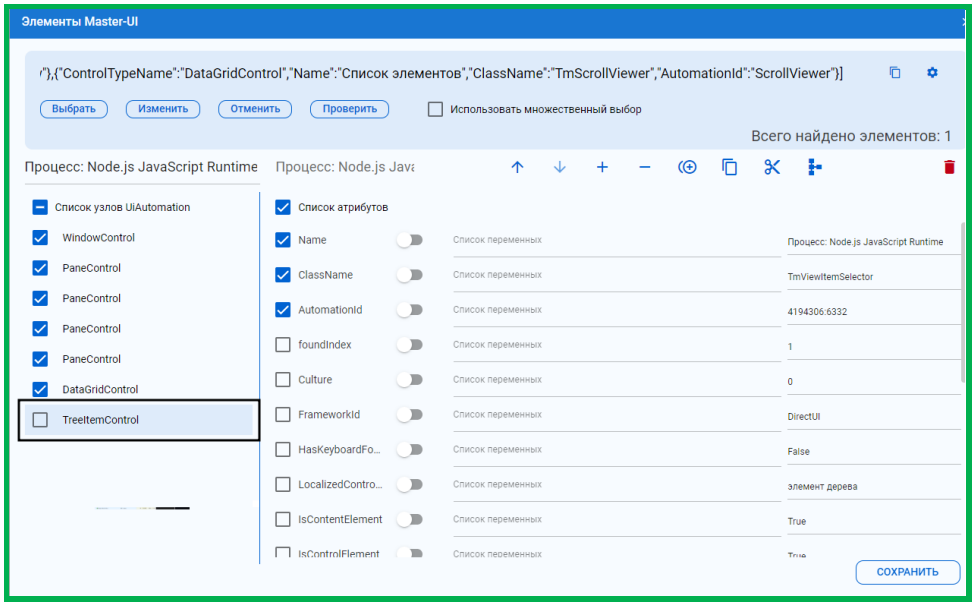
В данном примере выбран не весь UI-элемент таблицы (который необходим для работы блока), а лишь один из её внутренних элементов. Этот элемент является дочерним по отношению к родительскому элементу, представляющему всю таблицу.

Дерево элементов приложения имеет вид:



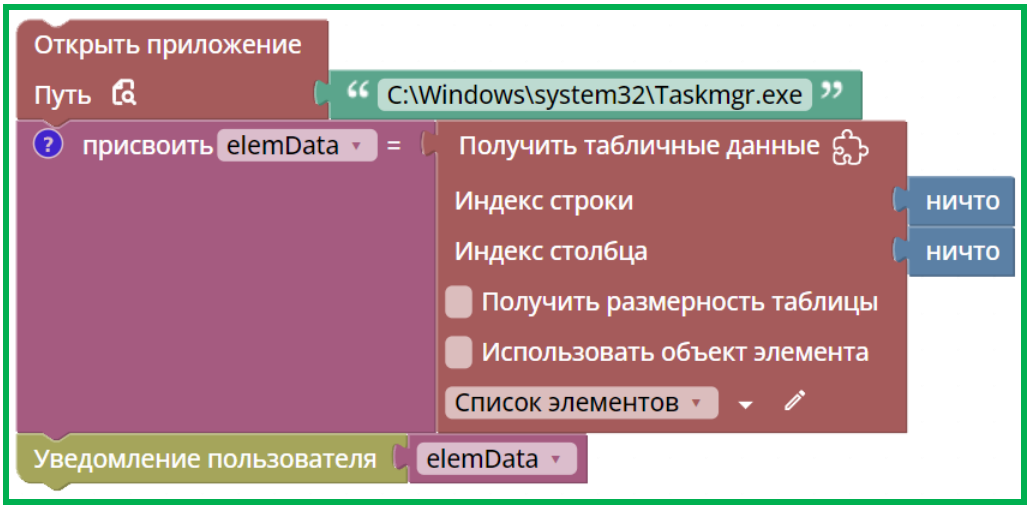
Поэтому для использования UI-элемента таблицы необходимо исключить конечный (вложенный) узел из выбранного элемента.

3. В инструменте Master UI, выполняется отключение последнего узла выбранного элемента.

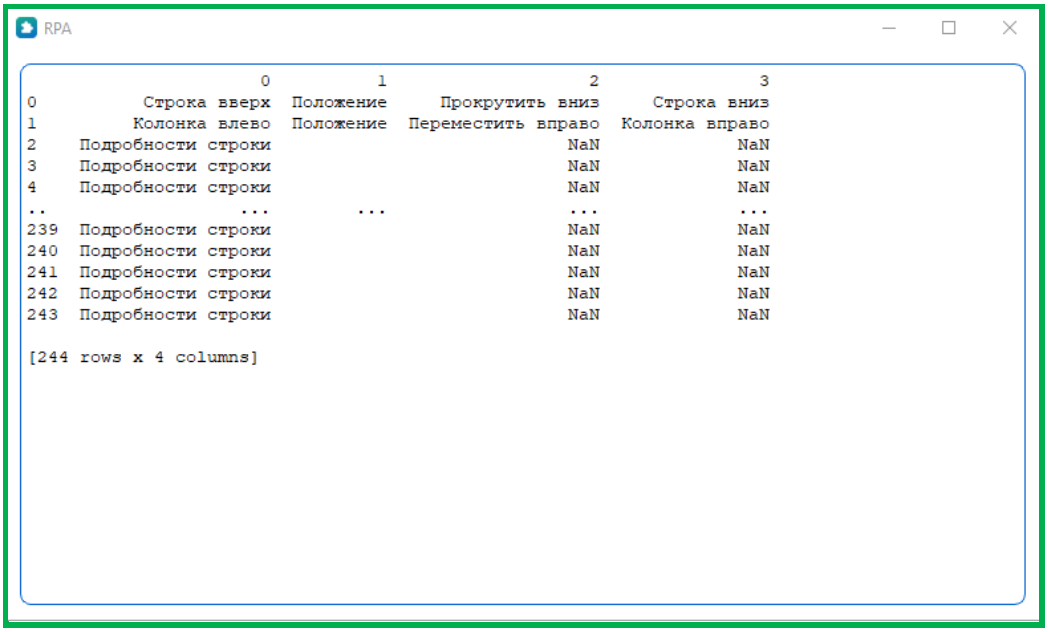


Алгоритм состоит из блоков:

- Открыть приложение **Диспетчер задач**;
- Получить табличные данные;
- Уведомление пользователя с результатом.

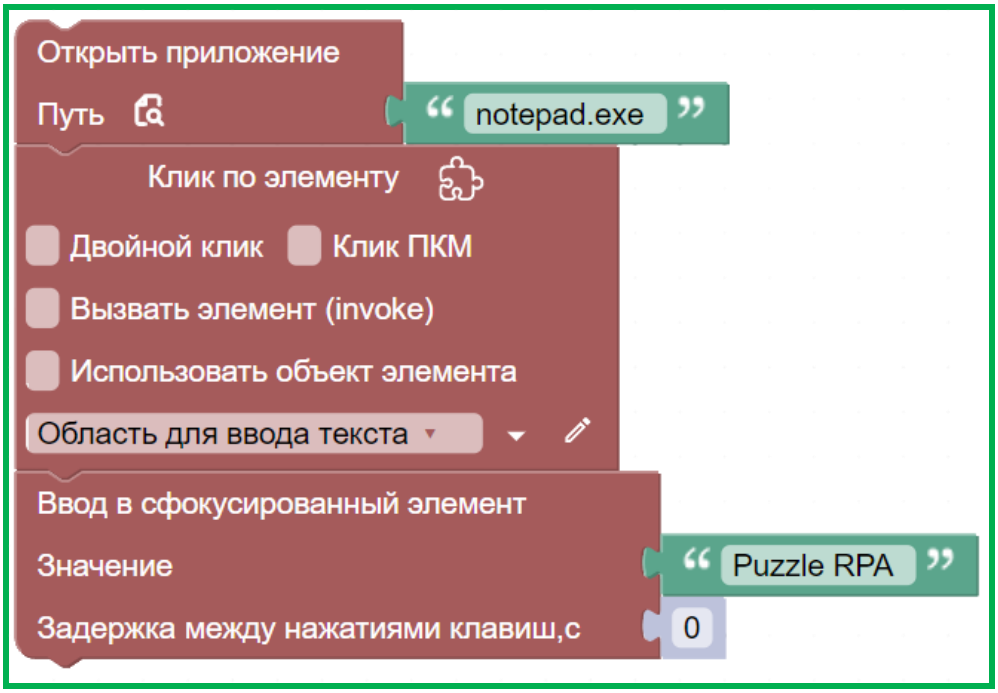


Результат считывания таблицы:

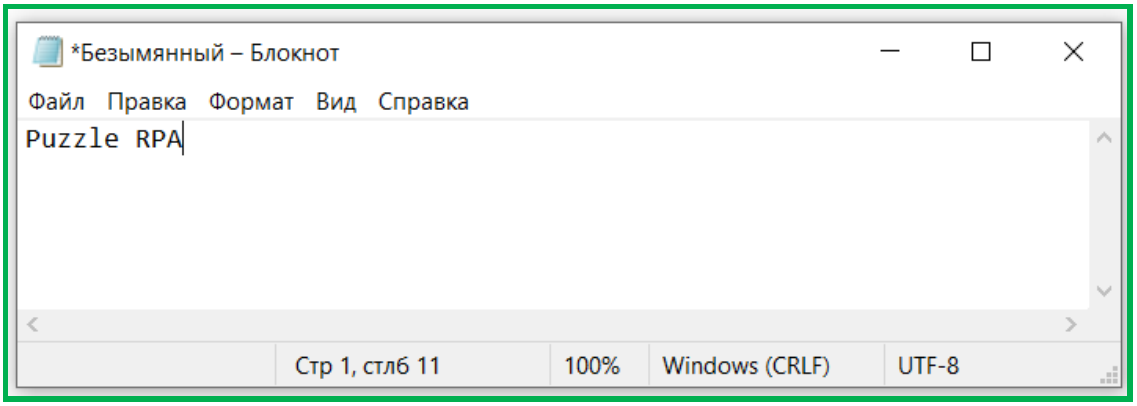


Пример 2 – Ввод в сфокусированный элемент

В данном примере открывается приложение “Блокнот”, и в него записывается текст “Puzzle RPA”:



Результат ввода текста в “Блокнот”:

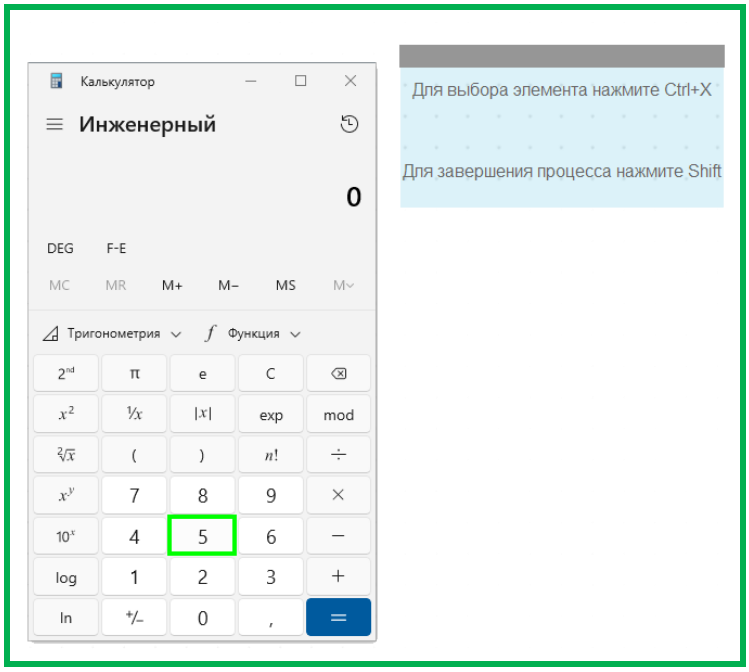


Пример 3 – Перемещение фокуса

В данном примере открывается приложение **Калькулятор** и выполняется перемещение курсора мыши на кнопку “5”.

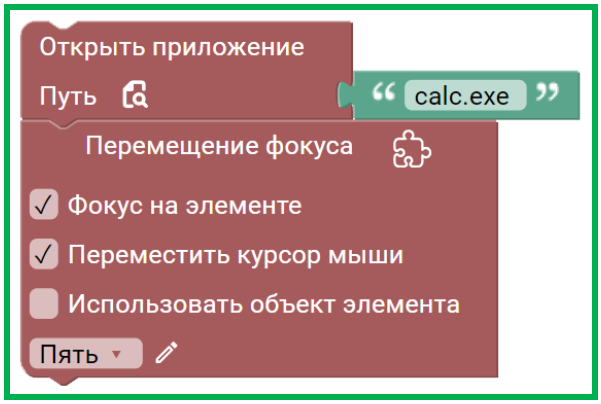
Для выбора элемента в блоке **Перемещение фокуса** используется инструмент Master UI для десктопных приложений. Для этого:

- Открывается инструмент **Master UI для десктопных приложений**;
- Выполняется выбор элемента, соответствующий кнопке 5 в приложение калькулятор.

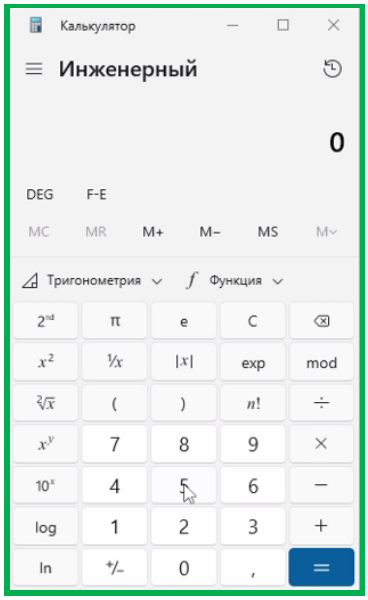


Алгоритм состоит из блоков:

- Открыть приложение **calc.exe**;
- Перемещение фокуса.



Приложение калькулятор после клика:



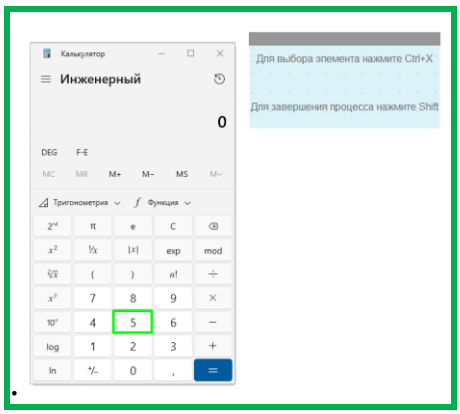
Пример 4 – Клик со смещением от элемента

В данном примере блок открывает приложения **Калькулятор** и выполняется последовательный клик по клавишам “5” (через клик по элементу), ”+” (через

клик со смещением в процентах относительно центра), “5” (через клик по элементу), ”=” (через клик со смещением в пикселях относительно левого верхнего угла).

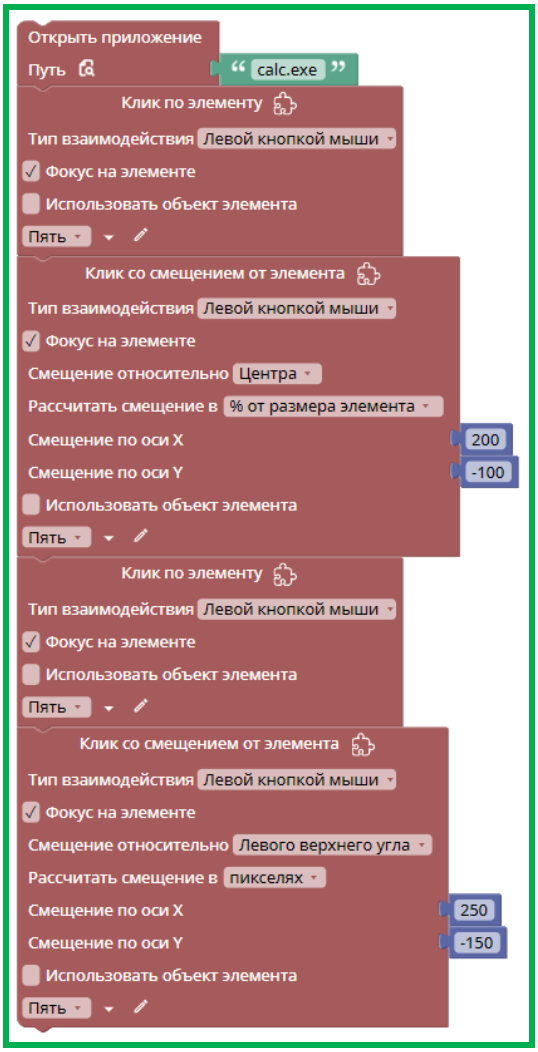
Для выбора элемента в блоке **Клик со смещением от элемента** используется инструмент Master UI для десктопных приложений. Для этого:

- Открывается инструмент **Master UI для десктопных приложений**;
- Выполняется выбор элемента, соответствующий кнопке 5 в приложение калькулятор



Алгоритм состоит из блоков:

- Открыть приложение **calc.exe**;
- Клик по элементу;
- Клик со смещением от элемента.



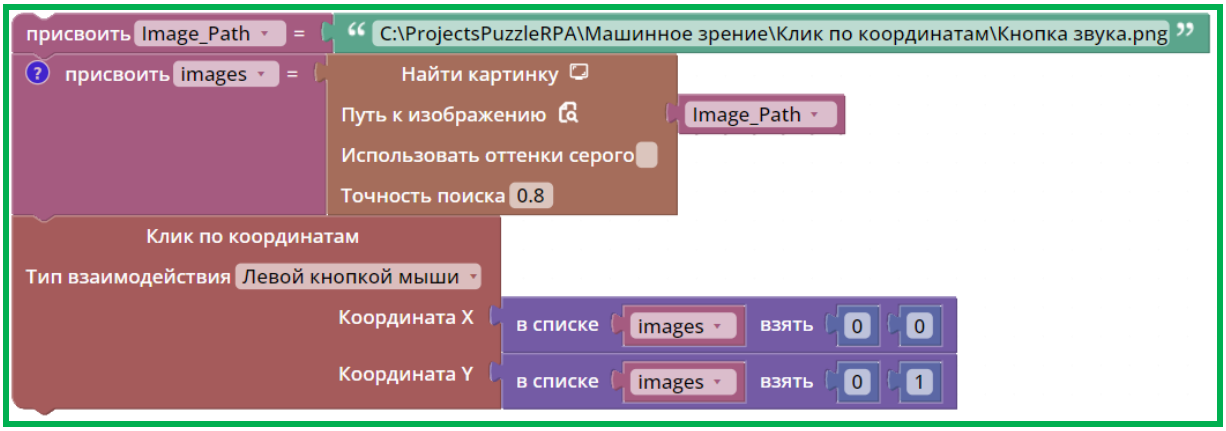
Приложение калькулятор после клика:



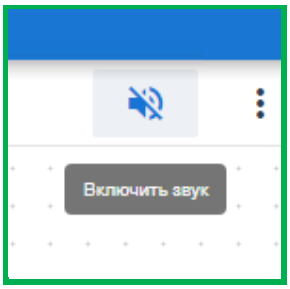
Пример 5 –Клик по координатам

В данном примере осуществляется клик по координатам кнопки звука в Платформе Puzzle RPA:

- Указывается путь до изображения со значком звука;
- Осуществляется поиск координат указанного изображения на экране;
- Выполняется клик по найденным координатам на экране.

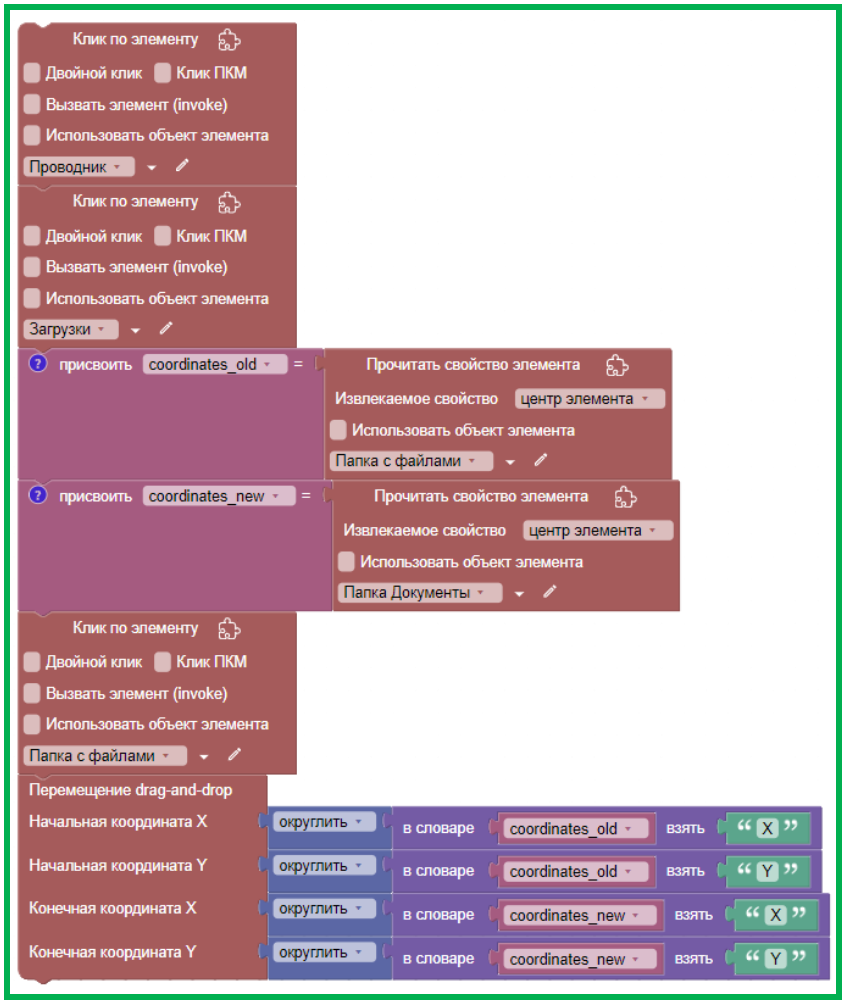


Клик на кнопку звука в Платформе Puzzle RPA:

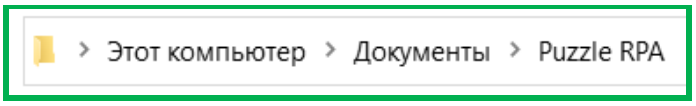


Пример 6 – Перемещение drag-and-drop

В данном примере производится перемещение папки из каталога “Загрузки” в каталог “Документы” с помощью drag-and-drop:

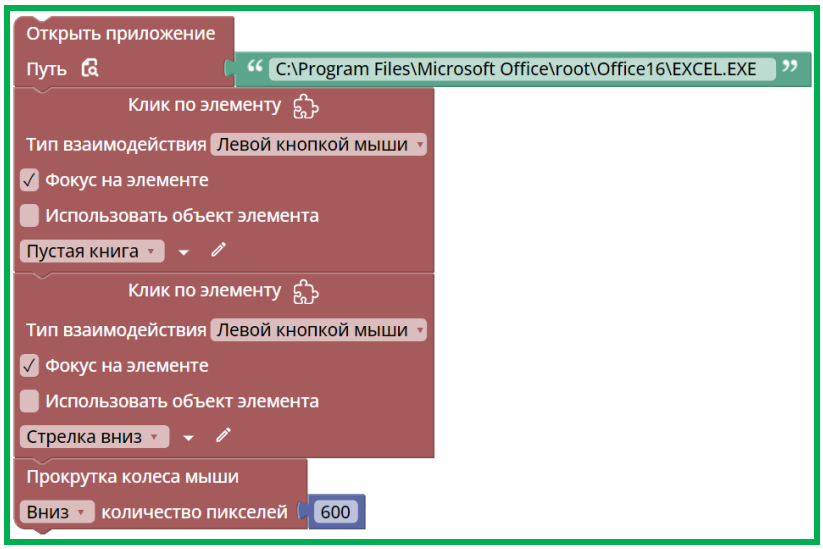


Расположение перемещенной папки:

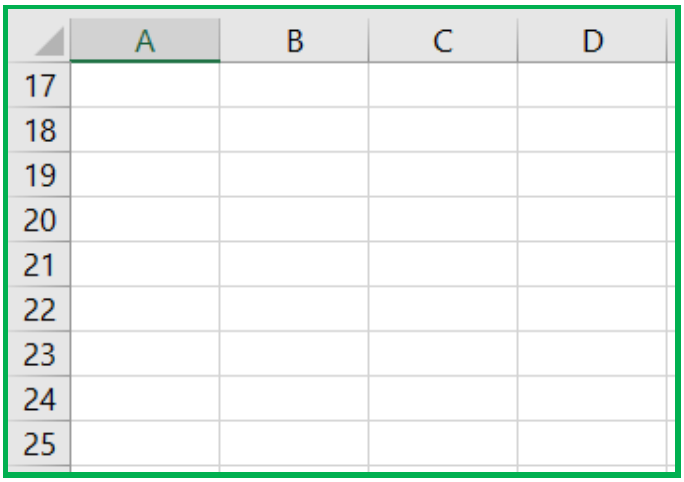


Пример 7 – Прокрутка колеса мыши

В данном примере блок открывает приложение Excel, создает пустую книгу, с помощью клика по стрелочке вниз устанавливает фокус в поле листа и прокручивает колесо мыши вниз на 600 пикселей.



В результате прокрутки колеса мыши произошло перемещение рабочей области Excel-листа с 1-ой на 17-ую строку:



Кейс 2. «Автоматизация обработки заявок в корпоративной системе»

После внедрения прототипа системы регистрации заявок руководство компании TechFlow решило расширить её возможности.

Теперь сотрудники должны иметь возможность:

- изменять статус заявок
- удалять устаревшие или ошибочно созданные записи
- обрабатывать заявки из внешних источников данных

Кроме того, планируется автоматизировать массовую обработку заявок с помощью программных роботов, которые смогут добавлять записи из таблиц, изменять их статус и выполнять стандартные операции в интерфейсе системы.

В рамках данной практической работы необходимо доработать созданное на предыдущей практической работе приложение и разработать более сложный сценарий автоматизации.

Задание 1. Расширение функциональности приложения

Необходимо доработать созданное на предыдущей практической работе приложение и добавить новые элементы управления.

Новые кнопки интерфейса:

- `Mark` as `Done`

  Изменяет статус выбранной заявки на

```
{ "status": "Done" }
```

- `Delete Selected`

  Удаляет выбранную заявку из таблицы.

- Подтверждение удаления

  Перед удалением записи должно отображаться окно подтверждения действия.

  Пример сообщения:

```
Are you sure you want to delete this request?
```

- Управление активностью элементов интерфейса

  Необходимо реализовать управление доступностью кнопок:

- Кнопки `Mark as Done` и `Delete Selected` должны быть неактивными (`IsEnabled = false`), если ни одна строка таблицы не выбрана.
- После выбора строки таблицы кнопки должны становиться активными (`IsEnabled = true`).
- После удаления записи кнопки снова должны становиться неактивными, если строка больше не выбрана.

Задание 2. Разработка расширенного RPA-сценария

После доработки приложения необходимо создать более сложный процесс автоматизации.

Алгоритм работы робота

1.Запустить приложение.

2.Считать список заявок из Excel-файла.

3.Добавить в систему не менее десяти заявок.

4.Найти заявку с приоритетом `High`.

5.Выбрать соответствующую строку таблицы.

6.Проверить, что кнопка `Mark as Done` активна (`IsEnabled = true`).

7.Изменить статус выбранной заявки на `Done`.

8.Выбрать одну из заявок для удаления.

9.Проверить, что кнопка `Delete Selected` активна (`IsEnabled = true`).

10. Удалить запись и обработать всплывающее окно подтверждения.

Логирование выполнения процесса

В расширенном сценарии необходимо реализовать подробное логирование работы робота.

В журнале должны фиксироваться:

- начало выполнения процесса
- загрузка данных из Excel
- добавление каждой заявки
- изменение статуса заявки
- удаление записи
- обработка окна подтверждения
- завершение процесса

Модульность RPA-процесса

Для повышения устойчивости и удобства сопровождения необходимо реализовать модульную структуру процесса.

Рекомендуется выделить отдельные функции или подпроцессы:

- запуск приложения
- чтение данных из Excel
- добавление заявки
- изменение статуса заявки
- удаление записи
- обработка всплывающих окон

- запись событий в журнал

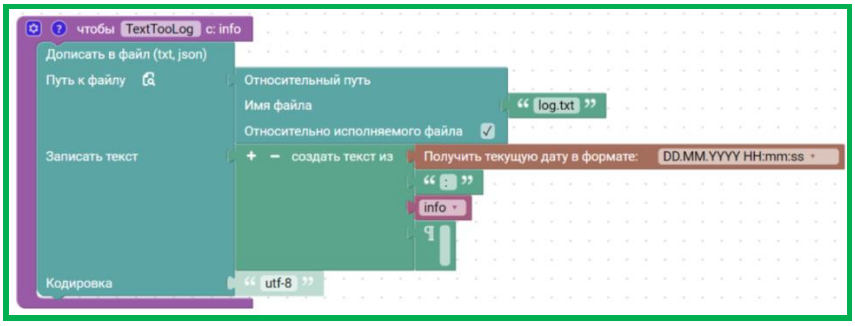
Использование модульной архитектуры позволяет повторно использовать блоки автоматизации и упрощает поддержку RPA-решения.

Теперь перейдём к реализации RPA-сценария в среде Puzzle RPA.

Реализации RPA-сценария

Для выполнения поставленной задачи был разработан модульный робот, состоящий из пяти основных функций. Давайте последовательно рассмотрим каждый блок автоматизации.

Функция `TextTooLog`



Шаг 1. Блок Получить текущую дату

Форматируем время: `дд.мм.гггг чч:мм:сс`

Шаг 2. Собрать строку

Склеиваем: `[дата]: [info] + перевод строки`

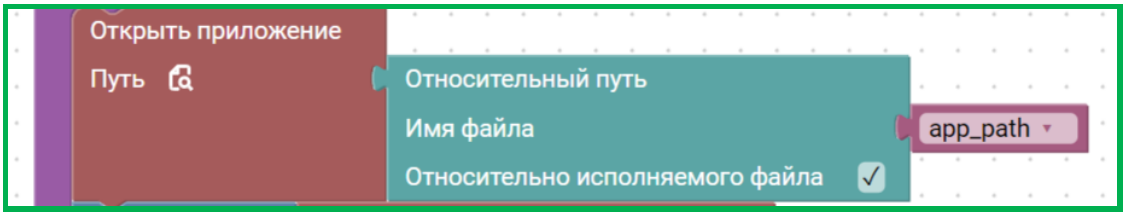
Шаг 3. Дописать в файл

Сохраняем в log.txt (в папке с программой)

Функция `start_exe`

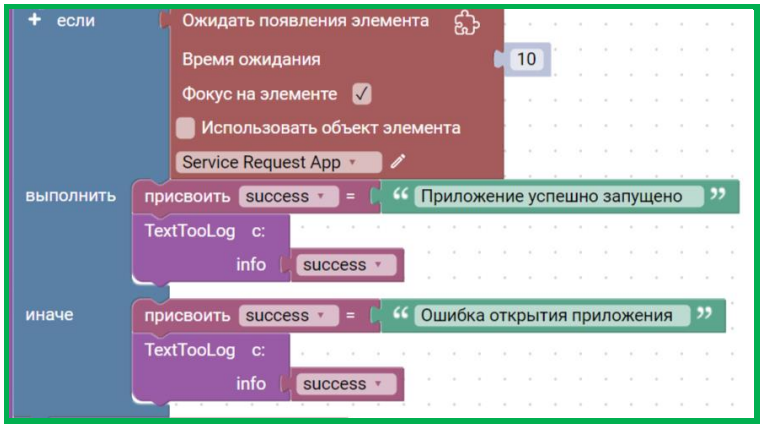
Шаг 1. Блок Открыть приложение

Запускаем .exe файл по указанному пути



Шаг 2. Блок Ожидать появления элемента

- Ждём, пока появится окно "Service Request App" (таймаут 10 сек)
- Устанавливаем фокус на элементе



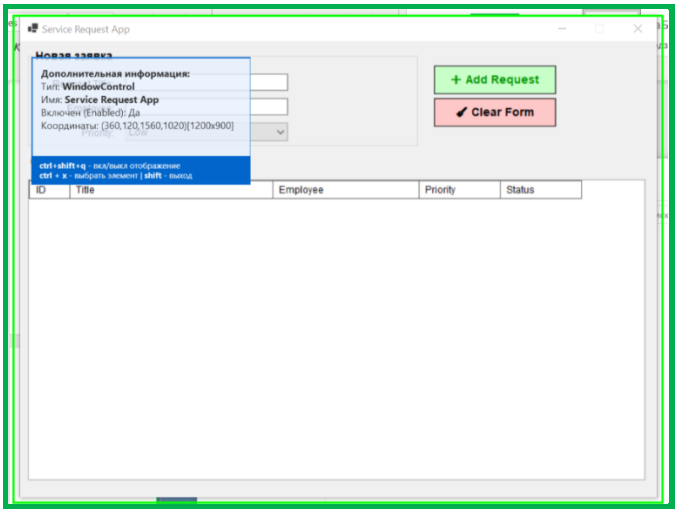
- Выбрать элемент через Мастер UI

2.1 В окне Master UI нажмите кнопку "Выбрать"

2.2 Наведите курсор мыши на главное окно приложения "Service Request App"

2.3 Нажмите комбинацию клавиш Ctrl + X (по умолчанию) – элемент подсветится зелёной рамкой

2.4 Для подтверждения выбора нажмите клавишу Shift

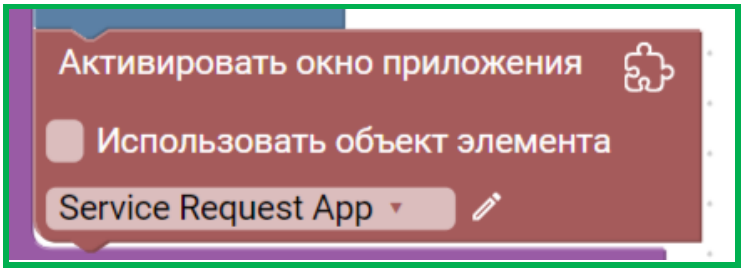


Шаг 3.Если окно появилось:

- Устанавливаем `success = "Приложение успешно запущено"`
- Логируем успех через `TextTooLog`
- Если окно НЕ появилось: Устанавливаем `success = "Ошибка открытия приложения"`
- Логируем ошибку

Шаг 4. Блок Активировать окно приложения

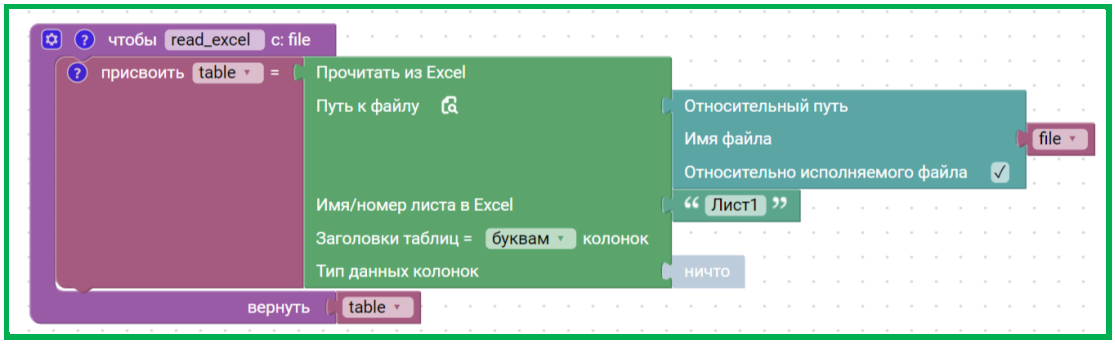
Устанавливаем фокус на окно приложения



Функция чтение `read_excel`

Шаг 1. Блок Прочитать из Excel

- Читаем файл requests.xlsx
- лист "Лист1"
- Сохраняем все данные в переменную `table`



Шаг 2. Вернуть результат

Возвращаем `table` в основной процесс

Функция `fill_form`

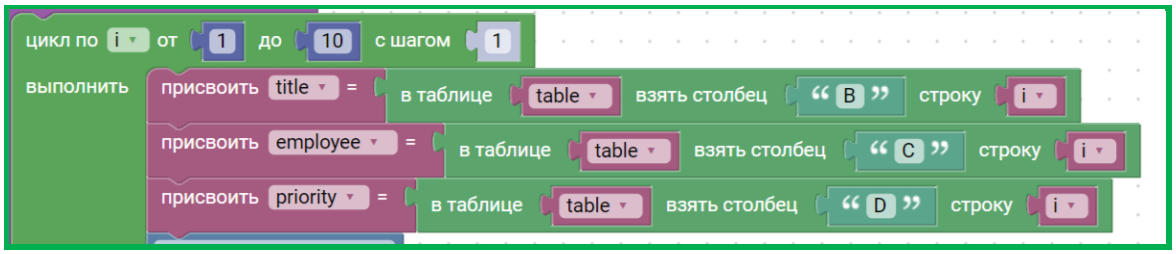
Шаг 1. Блок Цикл Повторяем 10 раз (для i от 1 до 1

Шаг 2. Внутри цикла для каждой заявки:

Получить данные из Excel

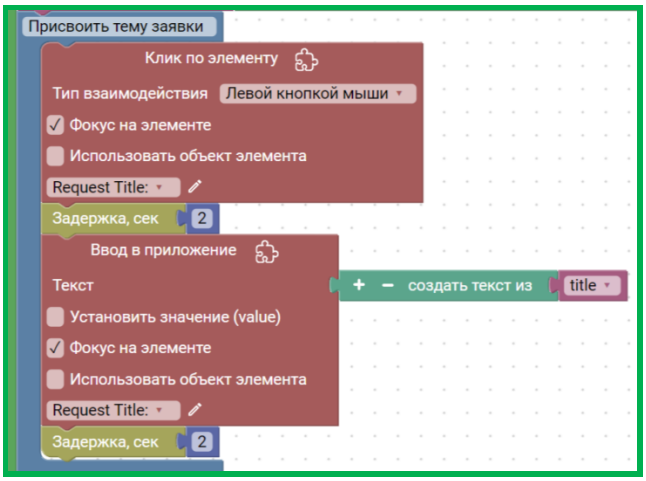
Из таблицы `table` берём:

- Колонка B → `title` (тема заявки)
- Колонка C → `employee` (сотрудник)
- Колонка D → `priority` (приоритет)



Шаг 3. Заполнить Тему заявки

- Клик по полю `Request Title`
- Задержка 2 сек
- Ввод текста из переменной `title`



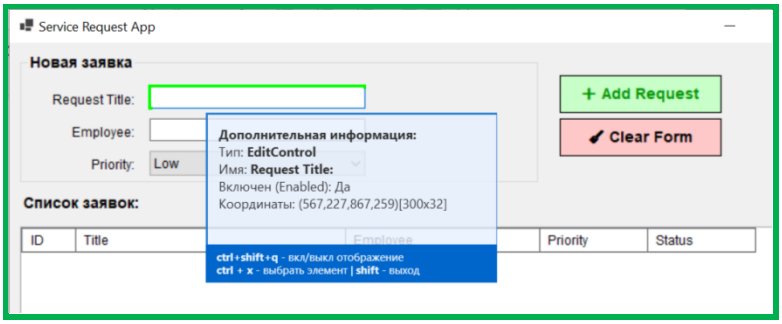
- Выбрать элемент через Мастер UI (поле Request Title)

2.1 В окне Master UI нажмите кнопку "`Выбрать`"

2.2 Наведите курсор мыши на поле `Request Title` в вашем приложении

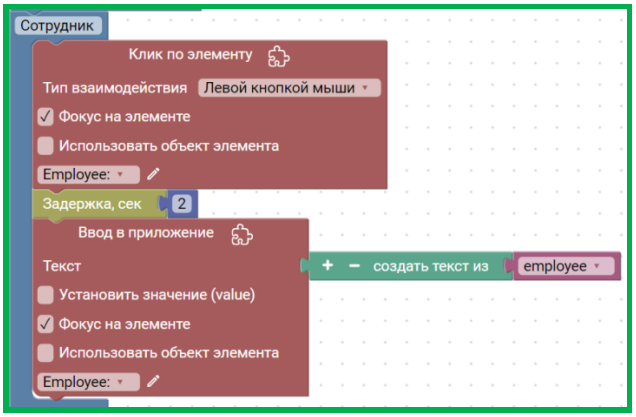
2.3 Нажмите комбинацию клавиш Ctrl + X – элемент подсветится зелёной рамкой

2.4 Для подтверждения выбора нажмите клавишу Shift

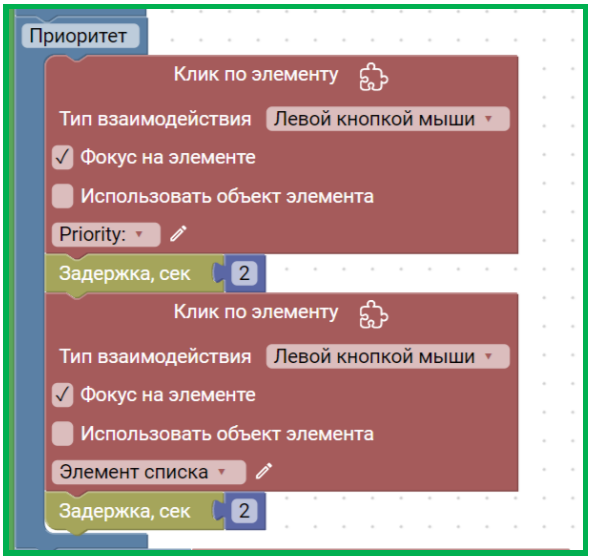


Шаг 4. Заполнить поле Сотрудник

- Клик по полю Employee
- Задержка 2 сек
- Ввод текста из переменной `employee`



Шаг 5. Выбрать приоритет



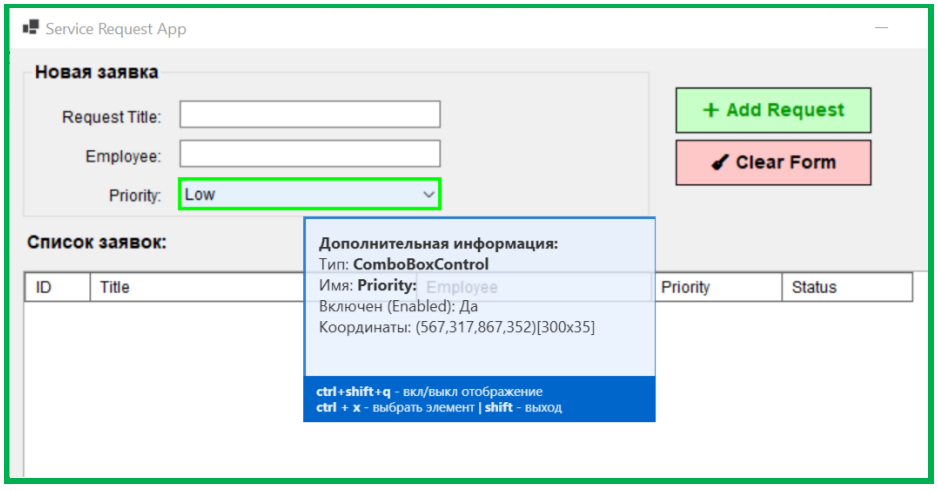
- Клик по выпадающему списку `Priority`
- Выбрать элемент через Мастер UI (сам выпадающий список `Priority`)

2.1 В окне Master UI нажмите кнопку "Выбрать"

2.2 Наведите курсор мыши на поле `Priority`

2.3 Нажмите комбинацию клавиш Ctrl + X (по умолчанию) – элемент подсветится зелёной рамкой

2.4 Для подтверждения выбора нажмите клавишу Shift



- Задержка 2 сек
- Клик по нужному пункту списка (Low/Medium/High)
- Пошаговая работа с Master UI для выбора пункта списка

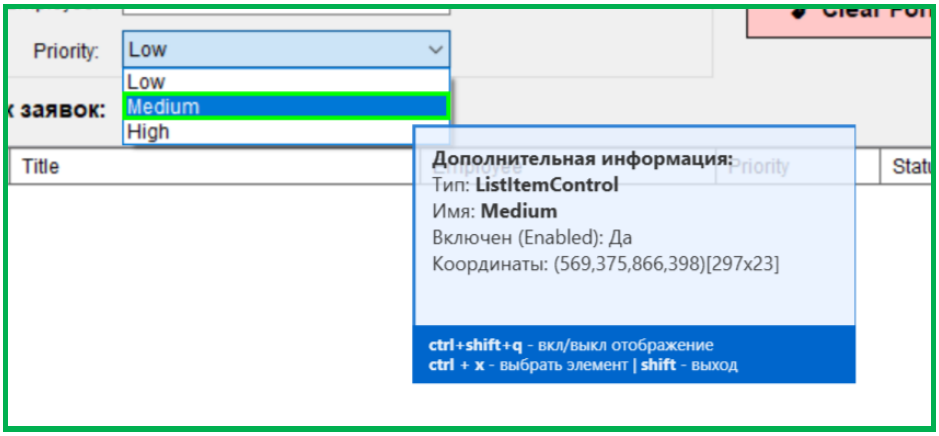
Шаг 1: Подготовка – раскрыть список вручную

Важный момент! Чтобы Master UI мог "увидеть" пункты списка, список должен быть раскрыт в момент выбора элемента.

1.1 Временно сделайте это вручную: кликните по полю Priority в вашем приложении, чтобы список раскрылся

1.2 Убедитесь, что пункты Low, Medium, High видны на экране

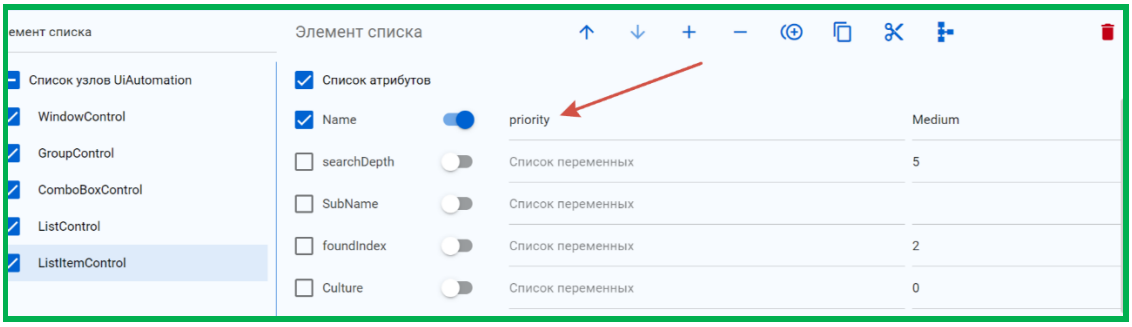
1.3 Не закрывайте список – переходите к Master UI



- После выбора в Master UI отобразится список узлов

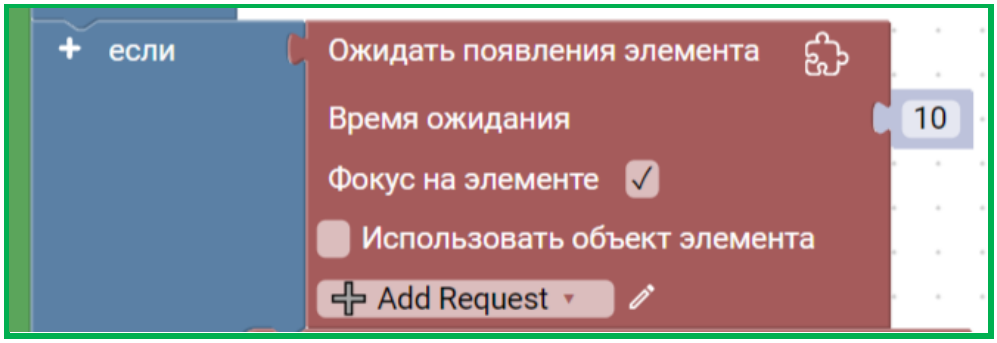
Обратите внимание на последний узел:

Именно здесь нам нужно, чтобы робот выбирал разные значения в зависимости от переменной [priority] (Low, Medium, High).



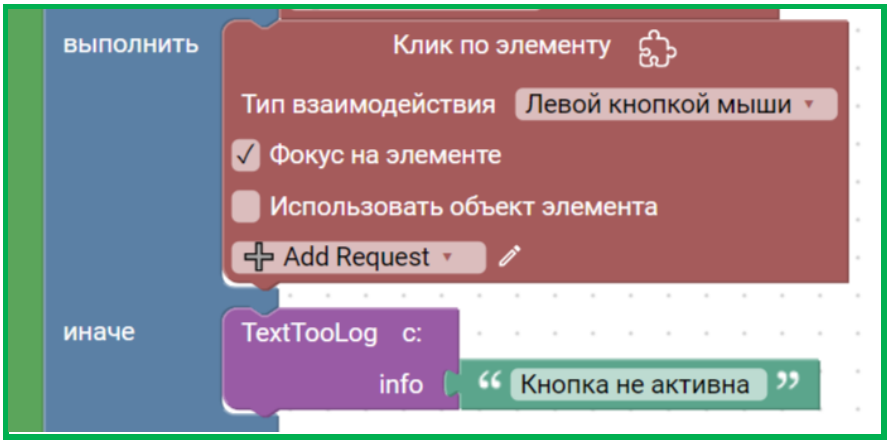
Шаг 6. Проверить кнопку `Add Request`

Ждём появления кнопки (таймаут 10 сек)



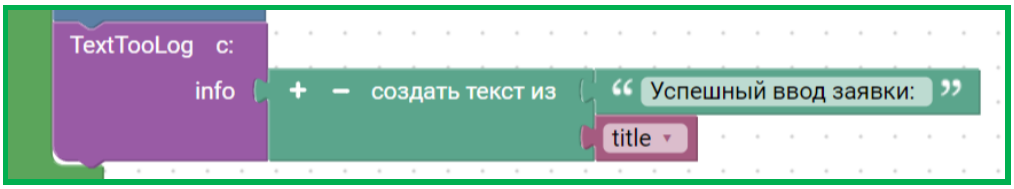
Шаг 7. Нажать `Add Request`

- Если кнопка появилась → кликаем
- Если нет → логируем "Кнопка не активна"



Шаг 8. Логирование

Записываем "Успешный ввод заявки: " + title (наименование заявки)



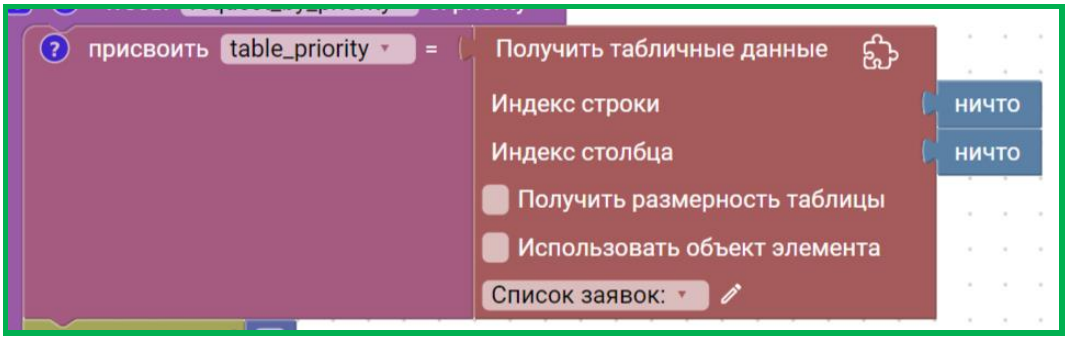
Функция `request_by_priority`

Функция `request_by_priority` выполняет поиск заявки с заданным приоритетом в таблице, выделяет найденную строку и изменяет её статус на "Done".

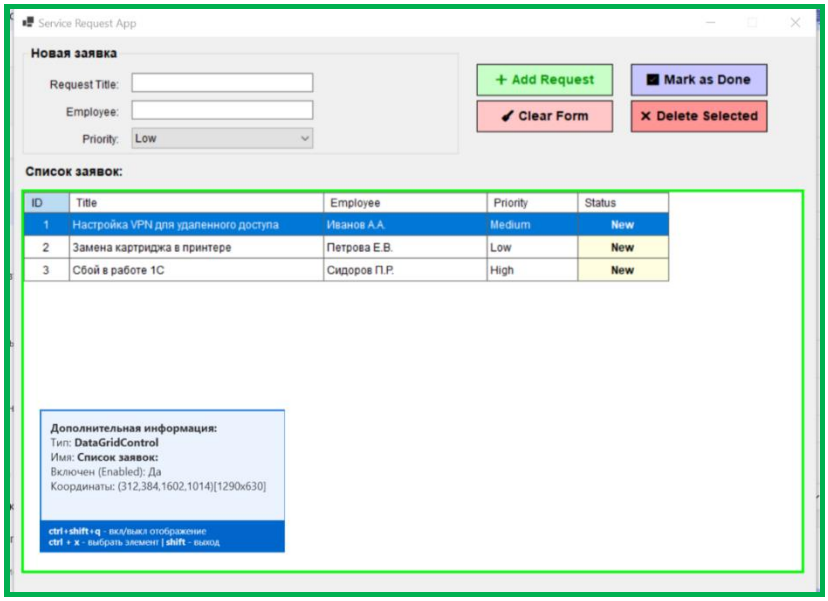
- Входной параметр: `priority` – строка с названием приоритета (например, "High")
- Выходное значение: `index_line` – индекс найденной строки в таблице

Шаг 1. Блок Получить табличные данные

- Элемент: Таблица dataGridRequests
- Результат: Переменная table_priority (таблица со всеми заявками)



- Настройка через Мастер UI:
- Нажать карандашик □ в блоке
- Навести курсор на таблицу "Список заявок"
- Нажать Ctrl+X → Shift
- Сохранить селектор



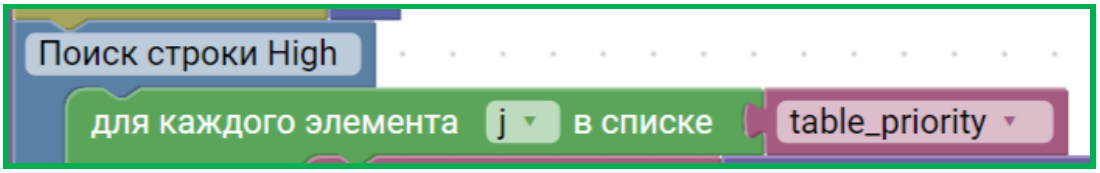
Шаг 2. Блок Задержка

- Длительность: 2 секунды
- Назначение: Дать время на полную загрузку данных в таблицу.

Шаг 3. Коллекция

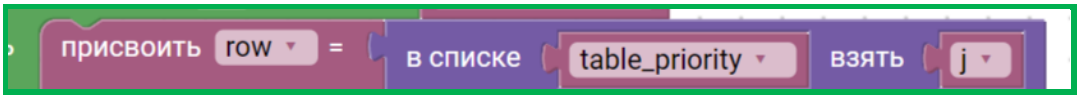
- Переменная `table_priority` (таблица)
- Текущий элемент: `j` (индекс строки)

Особенность: В данном коде цикл перебирает индексы строк таблицы, а не сами строки. Переменная `j` принимает значения 0, 1, 2... (номера строк).



Шаг 4. Блок Получить значение из строки таблицы (внутри цикла)

- Переменная: `row`
- Значение: `table_priority[j]` (строка таблицы по индексу)

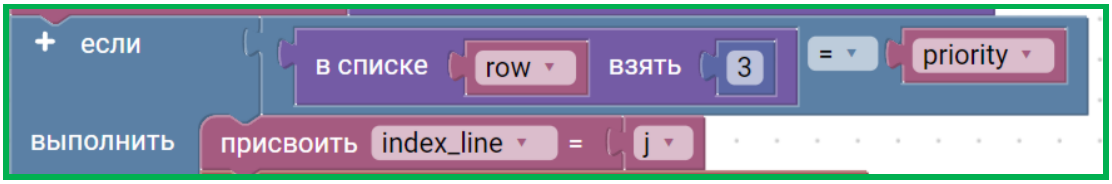


Шаг 5. Блок Условный оператор (проверка приоритета)

- Условие: `row[3] == priority`
- Пояснение: `row[3]` – это четвертая колонка таблицы (индекс 3), где хранится приоритет заявки.

Шаг 6. Блок Присвоить значение переменной (сохранение индекса)

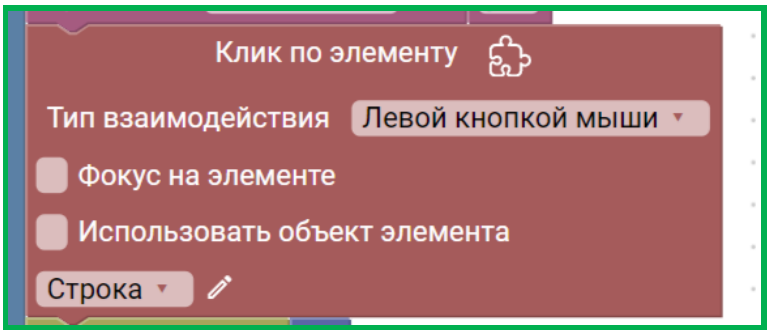
- Переменная: index_line
- Значение: j (текущий индекс строки)



Шаг 7. Блок Клик по UI-элементу (выбор найденной строки)

- Элемент: Строка таблицы с динамическим индексом

- Тип: Левая кнопка
- Фокус: FALSE

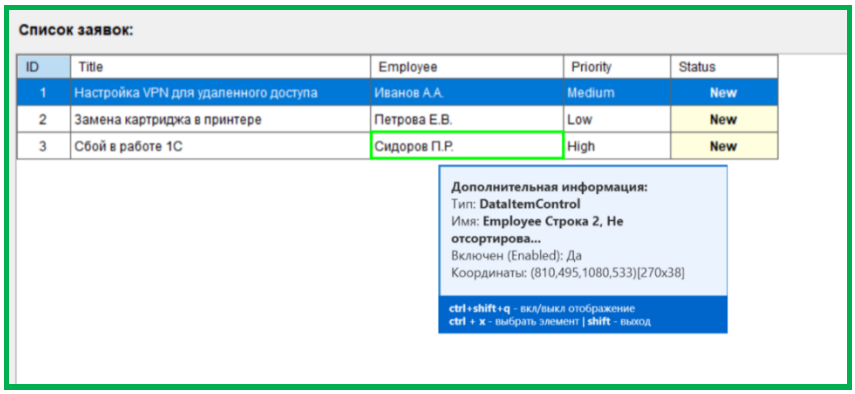


Настройка селектора:

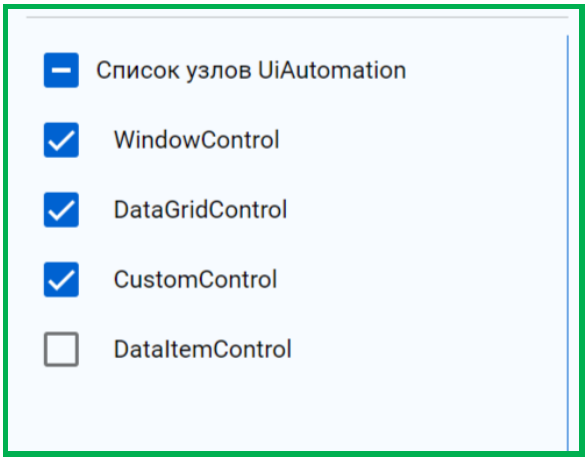
```
[
  {"ControlTypeName":"WindowControl","Name":"Service Request App"},
  {"ControlType-
Name":"DataGridControl","AutomationId":"dataGridRequests"},
  {"ControlTypeName":"CustomControl","foundIndex": index_line }
]
```

Внимание: В данном коде используется индекс переменной `index_line`.

- Открыть Мастер UI
- Нажмите кнопку карандашик ✎ в блоке "Клик по элементу"
- Откроется окно Мастер UI для десктопных приложений
- Нажмите кнопку «Выбрать»
- Наведите курсор мыши на любую строку в таблице приложения
- Нажмите комбинацию клавиш Ctrl+X – элемент подсветится зеленой рамкой
- Нажмите Shift – подтвердить выбор



В левой части окна «Список узлов» отобразится дерево элементов:



- Выберите узел CustomControl (это строка таблицы)

Узлы ниже (например, DataItemControl) можно отключить (снять галочку), так как они не нужны для выбора строки

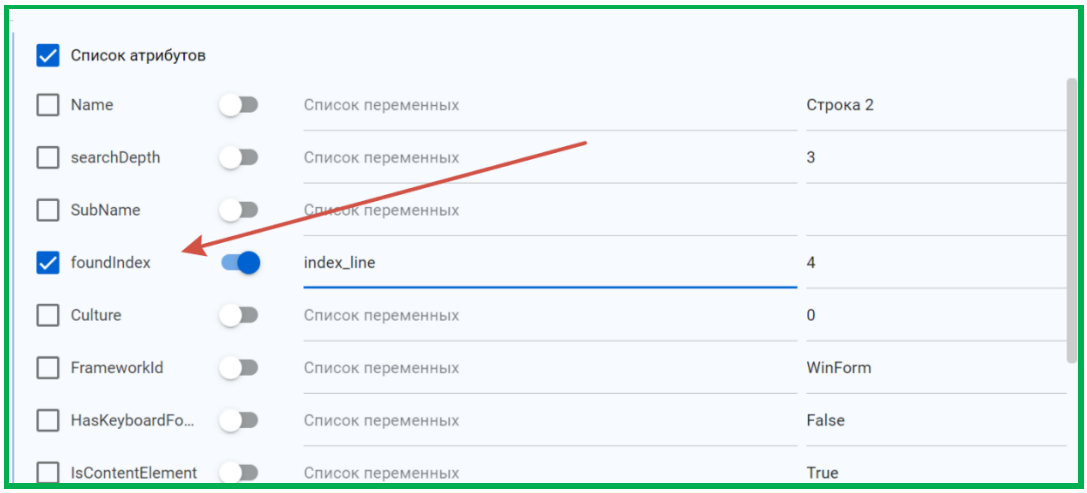
- Убедитесь, что узел WindowControl и DataGridControl остаются активными

В правой части окна откроется «Список атрибутов»

- Найдите атрибут, отвечающий за номер строки. Возможные варианты:

✓ `foundIndex`
✓ `RowIndex`
✓ `index`

1. Найдите атрибут `foundIndex` (или другой, отвечающий за номер строки)
2. Активируйте переключатель (тумблер) рядом с названием атрибута
3. Справа от переключателя появится поле для выбора переменной
4. Нажмите на выпадающий список и выберите переменную index_line
5. Обязательно снимите отметку с атрибута Name, если пробуете искать по индексу строки а не по имени
6. Нажмите кнопку «Сохранить» в правом нижнем углу
7. Селектор автоматически применится к блоку "Клик по элементу"

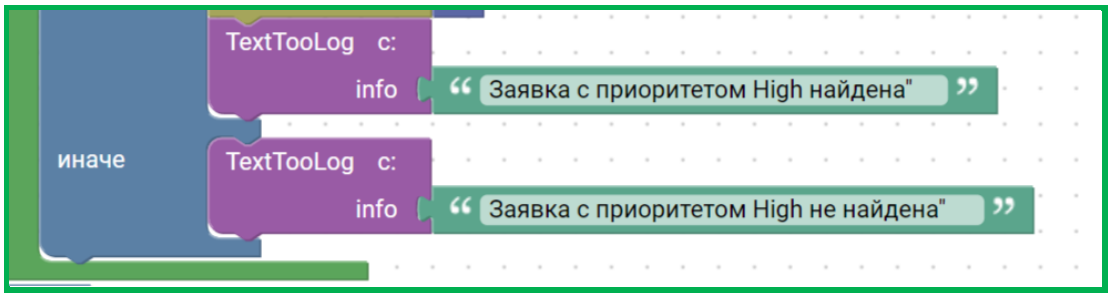


Шаг 8. Блок Задержка (delay)

Длительность: 2 секунды

Шаг 9. Блок Исполнить функцию (логирование) по результату поиска заявки.

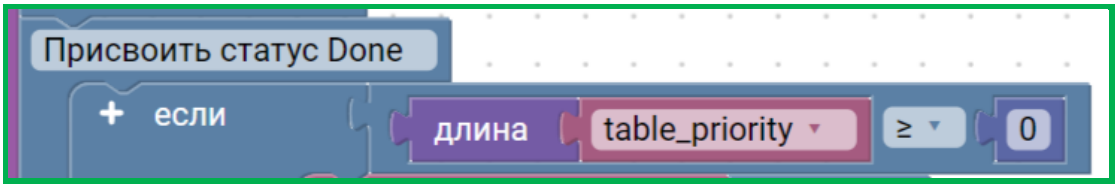
Функция: TextTooLog



Шаг 10. Блок Если – выполнить (проверка наличия данных)

Условие `len(table_priority) >= 0`

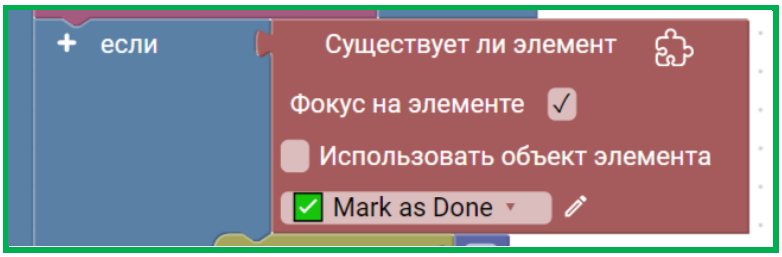
Пояснение: Это условие всегда истинно, так как длина таблицы всегда ≥ 0. В реальном сценарии здесь должна быть проверка, что таблица не пуста и строка с нужным приоритетом найдена.



Шаг 11. Блок Существует ли элемент (проверка кнопки Mark as Done)

Элемент: Кнопка ☑ Mark as Done (btnMarkAsDone)

Результат: Логическое значение

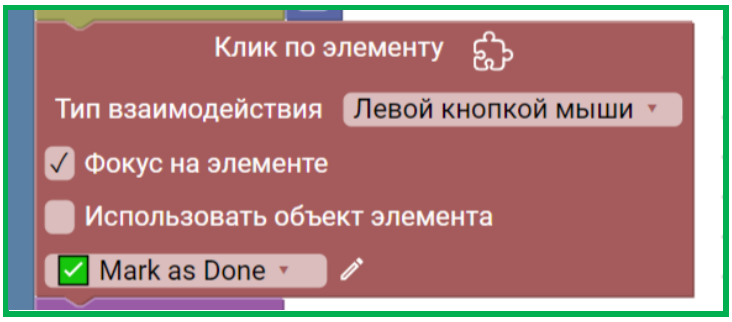


Шаг 11. Блок Задержка (delay)

Длительность: 2 секунды

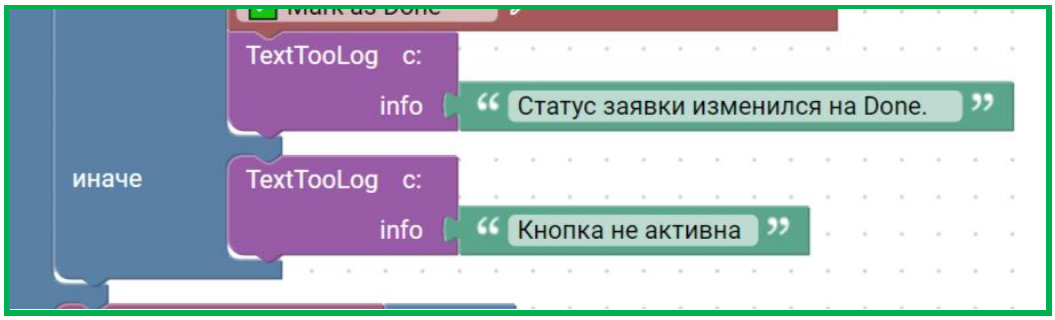
Шаг 12. Блок Клик по UI-элементу (нажатие Mark as Done)

- Элемент: Кнопка ☑ Mark as Done
- Тип: Левая кнопка
- Фокус: TRUE



Шаг 13. Блок Исполнить функцию (логировать действия с кнопкой)

Функция: TextTooLog



Шаг 14. Возврат значения

Значение: index_line (индекс найденной строки)

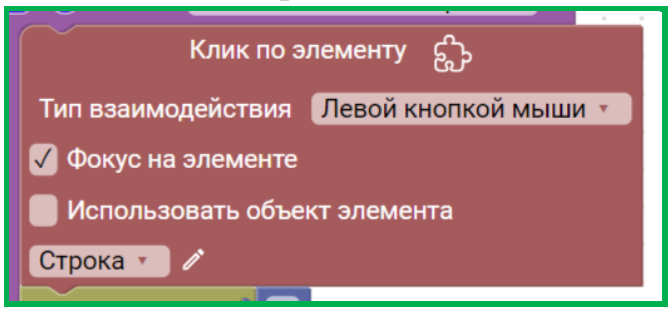
Функция select_and_delete_request

Выполняет удаление выбранной заявки из таблицы. Она автоматически выбирает строку для удаления, проверяет активность кнопки удаления и обрабатывает все модальные окна подтверждения.

Шаг 1. Блок Клик по UI-элементу (выбор строки для удаления)

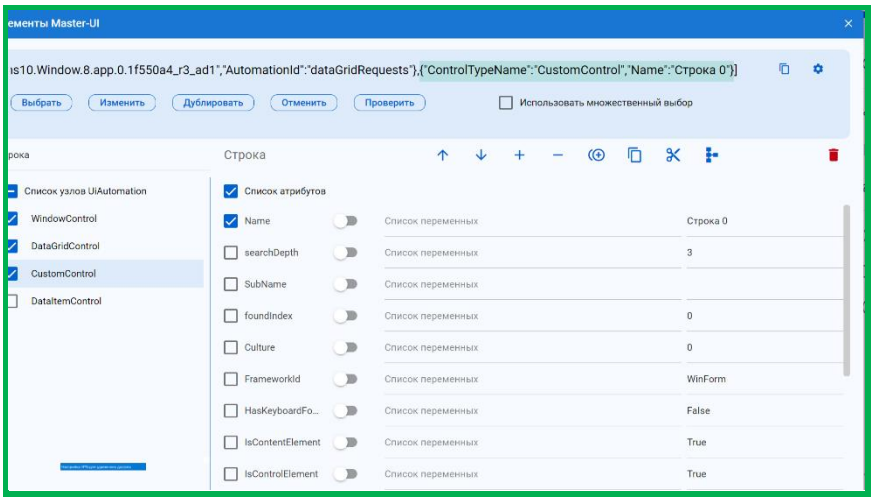
- Элемент: Строка таблицы (первая строка данных)

- Тип взаимодействия: Левая кнопка мыши
- Фокус на элементеTRUE (активировать элемент)



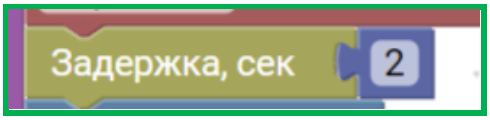
- Настройка через Мастер UI:

✓ Нажать карандашик в блоке
✓ Навести курсор на первую строку таблицы
✓ Нажать Ctrl+X → Shift
✓ В списке узлов оставить: Window → DataGrid → CustomControl
✓ Для атрибута Name установить значение или использовать foundIndex: 0



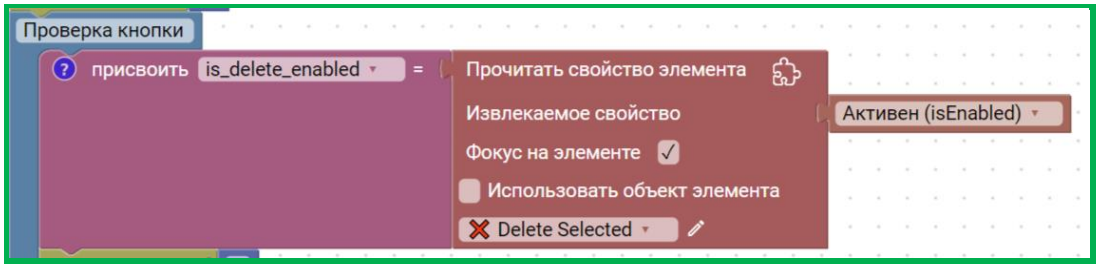
Шаг 2. Блок Задержка (ожидание выделения строки)

- Длительность: 2 секунды
- Назначение: Дать время интерфейсу на выделение выбранной строки перед проверкой кнопки.

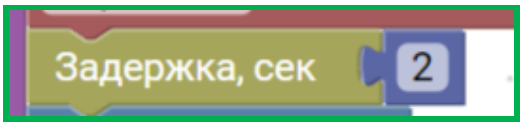


Шаг 3. Блок Прочитать свойство (проверка активности кнопки Delete Selected)

- Элемент: Кнопка ✕ `Delete Selected (btnDeleteSelected)`
- Свойство: `enabled (IsEnabled)`
- Результат: Переменная `is_delete_enabled`

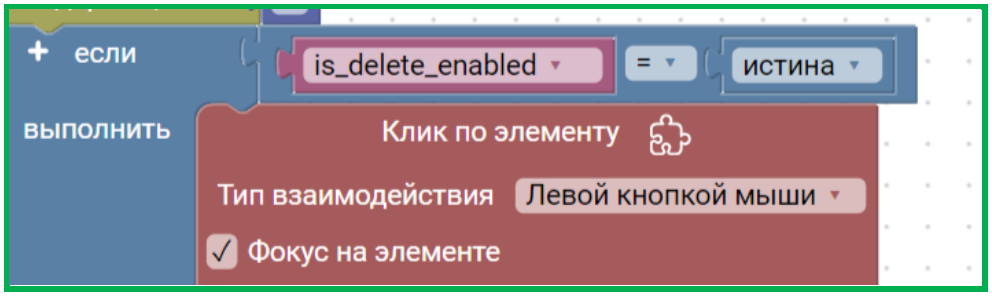


Шаг 4. Блок Задержка (пауза после чтения свойства)
Длительность: 2 секунды



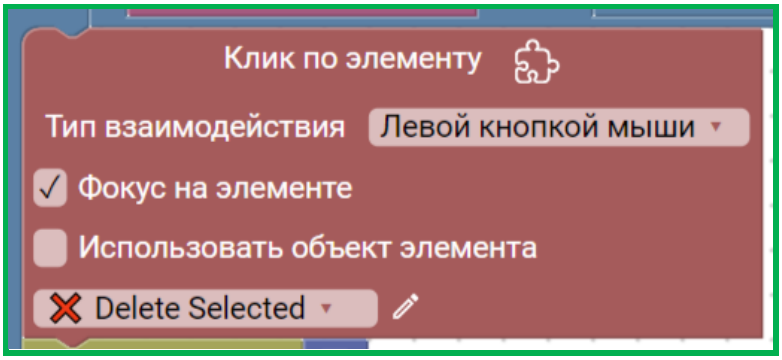
Шаг 5. Блок Условный оператор (проверка активности кнопки)
Условие: `is_delete_enabled == True`

- Ветка "Да": продолжается удаление
- Ветка "Нет": записывается лог "Кнопка Delete Selected не активна"

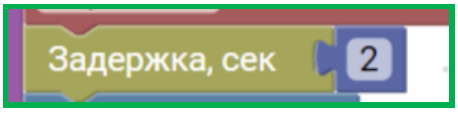


Шаг 6. Блок Клик по UI-элементу (нажатие `Delete Selected`)

- Элемент: Кнопка ✕ `Delete Selected`
- Тип: Левая кнопка
- Фокус:TRUE

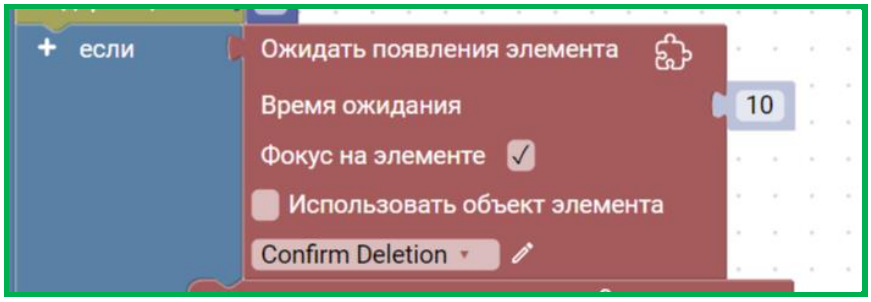


Шаг 7. Блок Задержка (ожидание появления окна)
Длительность: 2 секунды



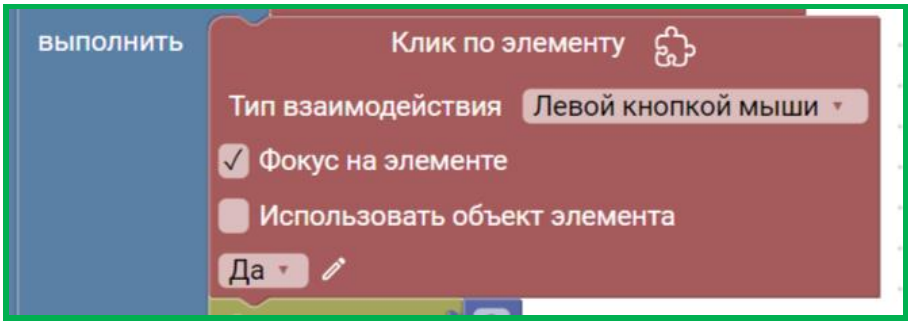
Шаг 8. Блок Условный оператор (проверка появления окна)

- Блок Ожидать появления элемента (окно подтверждения)
- Элемент: Окно с заголовком "Confirm Deletion"
- Таймаут: 10 секунд
- Фокус: TRUE



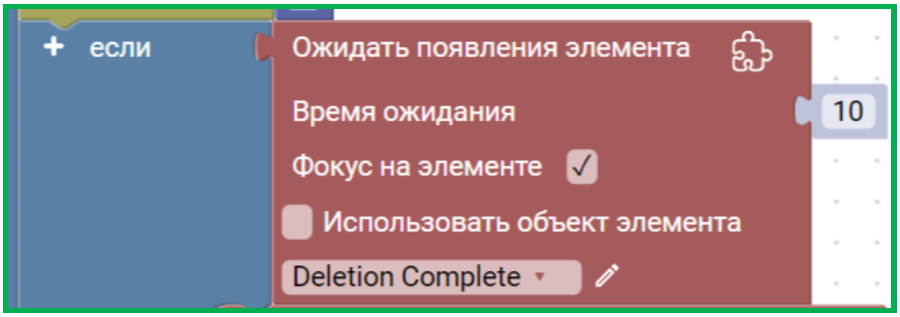
Шаг 9. Блок Клик по UI-элементу (кнопка "Да" в окне подтверждения)

- Элемент: Кнопка "Да" (Yes) в окне "Confirm Deletion"
- Тип: Левая кнопка
- Фокус:TRUE



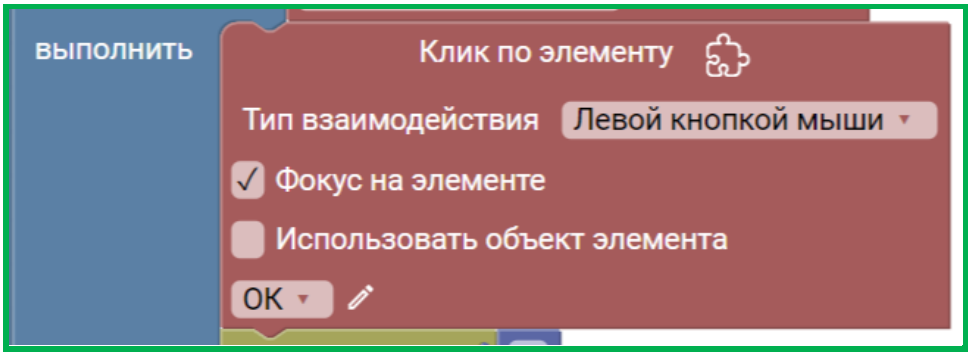
Шаг 10. Блок Задержка (ожидание закрытия окна)
Длительность: 2 секунды
Шаг 11. Блок Ожидать появления элемента (окно об успешном удалении)

- Элемент: Окно с заголовком "Deletion Complete"
- Таймаут: 10 секунд

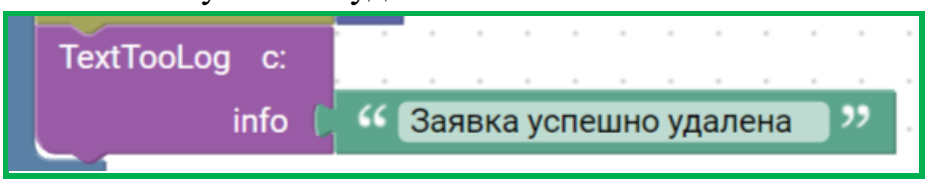


Шаг 12. Блок Клик по UI-элементу (кнопка "ОК")

- Элемент: Кнопка "ОК" в окне "Deletion Complete"
- Тип: Левая кнопка
- Фокус: TRUE

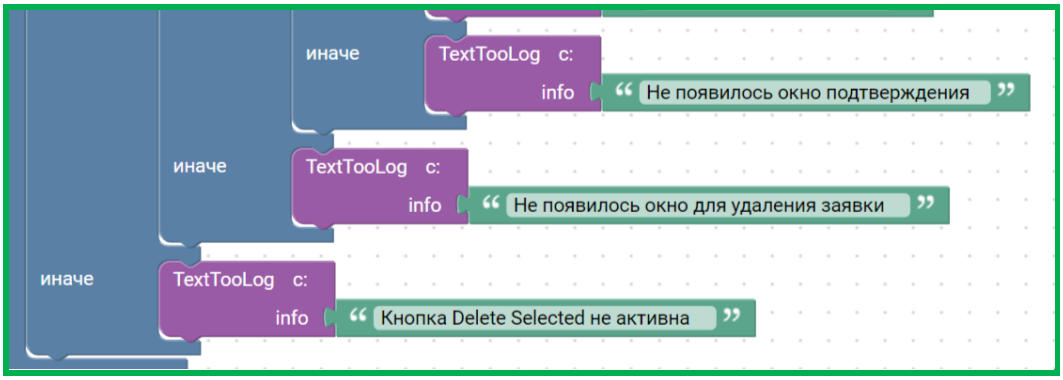


Шаг 13. Блок Задержка (ожидание закрытия окна)
Длительность: 2 секунды
Шаг 14. Блок Исполнить функцию (запись лога об успехе)
Функция: TextTooLog
Сообщение: 'Заявка успешно удалена'



Шаг 15. Блок Обработка ошибок (ветки "Нет" в условных операторах)

При отсутствии окон или неактивности кнопки вызывается TextTooLog с соответствующим сообщением об ошибке.



Главный процесс Desktop_2

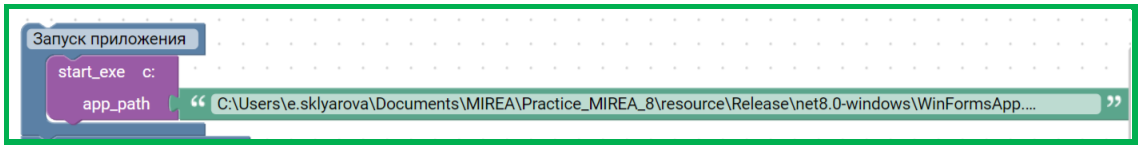
Главный процесс оркестратором всего RPA-сценария. Он последовательно вызывает функции, каждая из которых отвечает за отдельный этап автоматизации.

Шаг 1. Запуск приложения

Вызываем функцию start_exe

Пишем полный путь к файлу exe.

Приложения имеют зависимости – требует наличия других файлов в той же папке, то есть ожидает, что запуск будет из папки, где лежат все его компоненты



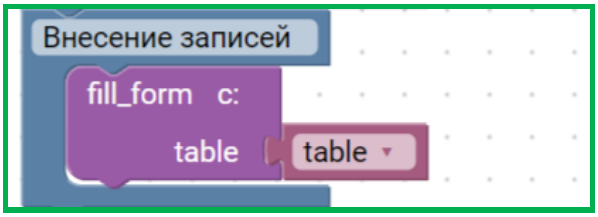
Шаг 2. Чтение Excel

Вызываем функцию read_excel('requests.xlsx') → получаем table



Шаг 3. Добавление заявок

Вызываем функцию fill_form(table) → добавляем 10 заявок



Шаг 4. Ищем строку по приоритету, выделяем её и изменяем статус на Done.

Вызываем `request_by_priority`

Даём значение переменной с приоритетом.

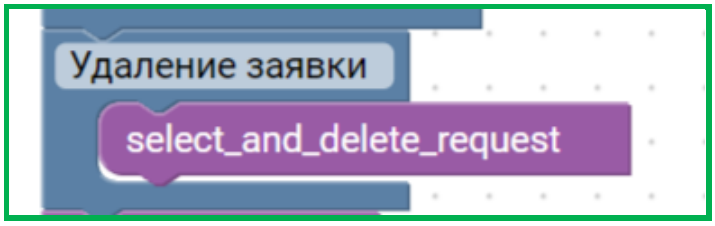


Подробнее о функции `request_by_priority`:

- Получает таблицу из приложения
- Перебирает строки в поисках Priority = "High"
- Находит индекс строки
- Кликает по найденной строке
- Проверяет наличие кнопки "Mark as Done"
- Кликает по кнопке
- Возвращает индекс найденной строки

Шаг 5. Удаление заявки.

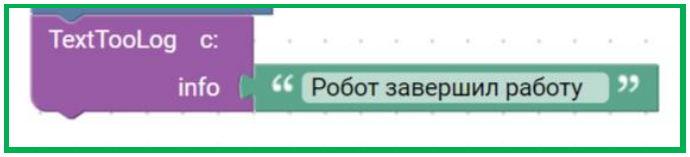
Вызываем `select_and_delete_request`

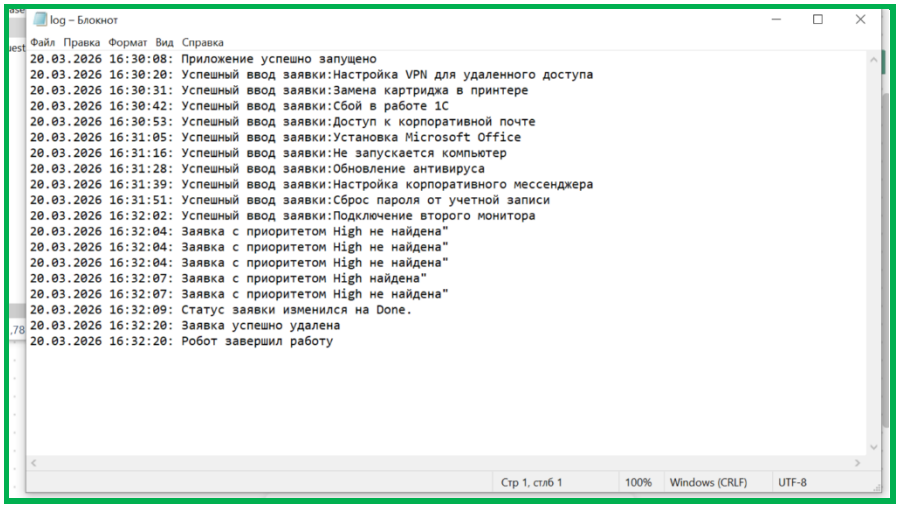


Подробнее о функции `select_and_delete_request`:

- Выбирает строку для удаления (первая строка)
- Проверяет, что кнопка "`Delete Selected`" активна
- Нажимает кнопку "`Delete Selected`"
- Ожидает окно подтверждения "`Confirm Deletion`"
- Нажимает кнопку "Да"
- Ожидает окно об успешном удалении "`Deletion Complete`"
- Нажимает кнопку "ОК"

Шаг 6. Отправляем сообщение об окончании работы и проверяем лог файлы





Практическая работа выполнена в полном объёме.

- Разработано десктоп-приложение для регистрации заявок с таблицей, кнопками управления и модальными окнами подтверждения.
- Создан модульный RPA-робот в среде Puzzle RPA, состоящий из функций запуска приложения, чтения Excel, массового добавления заявок, поиска за-

явки с приоритетом High с изменением статуса и удаления записи с обработкой модальных окон.

- Реализована система логирования для контроля выполнения всех этапов автоматизации.
- Выполнена проверка корректности добавления заявок, изменения статуса и удаления записей.

Робот полностью заменяет ручной труд при вводе заявок, исключает ошибки, связанные с человеческим фактором, и значительно ускоряет процесс обработки данных.

КОНТРОЛЬНЫЕ ЗАДАНИЯ

Общая часть задания практической работы

1. Ознакомьтесь со структурой и материалом методических указаний.
2. Ознакомьтесь с теоретическим материалом, изложенным в разделе «Теоретическая часть».
3. В случае отсутствия установленного ПО «Студия Puzzle RPA», скачайте по ссылке инсталляционный файл https://cloud.puzzle-rpa.ru/index.php/s/N87hb5sE2Lm0EQp (пароль для загрузки - `GDJoL9Yz`), и установите программный продукт. Для авторизации в студии введите

Логин - `mirea-student`
Пароль - `EEasSdDNKK5m`

Обратите внимание, что логин и пароль для доступа к полной версии продукта – временные и действуют до 05.07.2026 г.

4. Сделайте полную копию раздела «Теоретическая часть» из методички и поместите её в начало отчета.
5. Внимательно изучите и разберите материал раздела «Примеры задач».
6. Внимательно изучите условие контрольного примера (аналогичное задание для уникальной предметной области будет предложено Вам для самостоятельного выполнения).
7. Просмотрите видео-урок по выполнению задания контрольного примера.
8. **Поместите в отчет условие задания своего варианта, скопировав его из учебного репозитория**.
9. Для выполнения Задания 1 Вашего варианта создайте в любой удобной для Вас IDE Windows-приложение под именем ***PR8_Surname_RPA_Desktop_p2_ Problem1_varN***.
10. Для выполнения Задания 2 Вашего варианта создайте проект в программе *Puzzle RPA* под именем ***PR8_Surname_RPA_Desktop_p2_ Problem2_varN***, где *Surname* – Ваша фамилия, *N* – номер Вашего варианта.
11. Выполните 2 индивидуальных задания Вашего варианта (*не забудьте в Настройках активировать опцию Автосохранение*), последовательно оформляя отчет и размещая скрины с результатами задания, на которых **должны быть зафиксированы**, кроме схемы работы робота, имена проектов, содержащие **Вашу фамилию**.
12. **Поместите в конец отчета публичную ссылку** на чат Вашей защиты практической работы с помощью ИИ DeepSeek.

КОНТРОЛЬНЫЕ ЗАДАНИЯ

Контрольное задание 1

Задание 1. Расширение функциональности приложения

Необходимо доработать созданное на предыдущей практической работе приложение, добавив новые элементы управления и логику взаимодействия с таблицей доставок.

⭕ Новые элементы управления

🚚 Dispatch Delivery

Изменяет статус выбранной доставки:

{ "status": "Dispatched" }

👤 Assign Driver

Назначает водителя на выбранную доставку.

⚙️ Управление активностью элементов (IsEnabled)

Необходимо реализовать корректное управление доступностью кнопок:

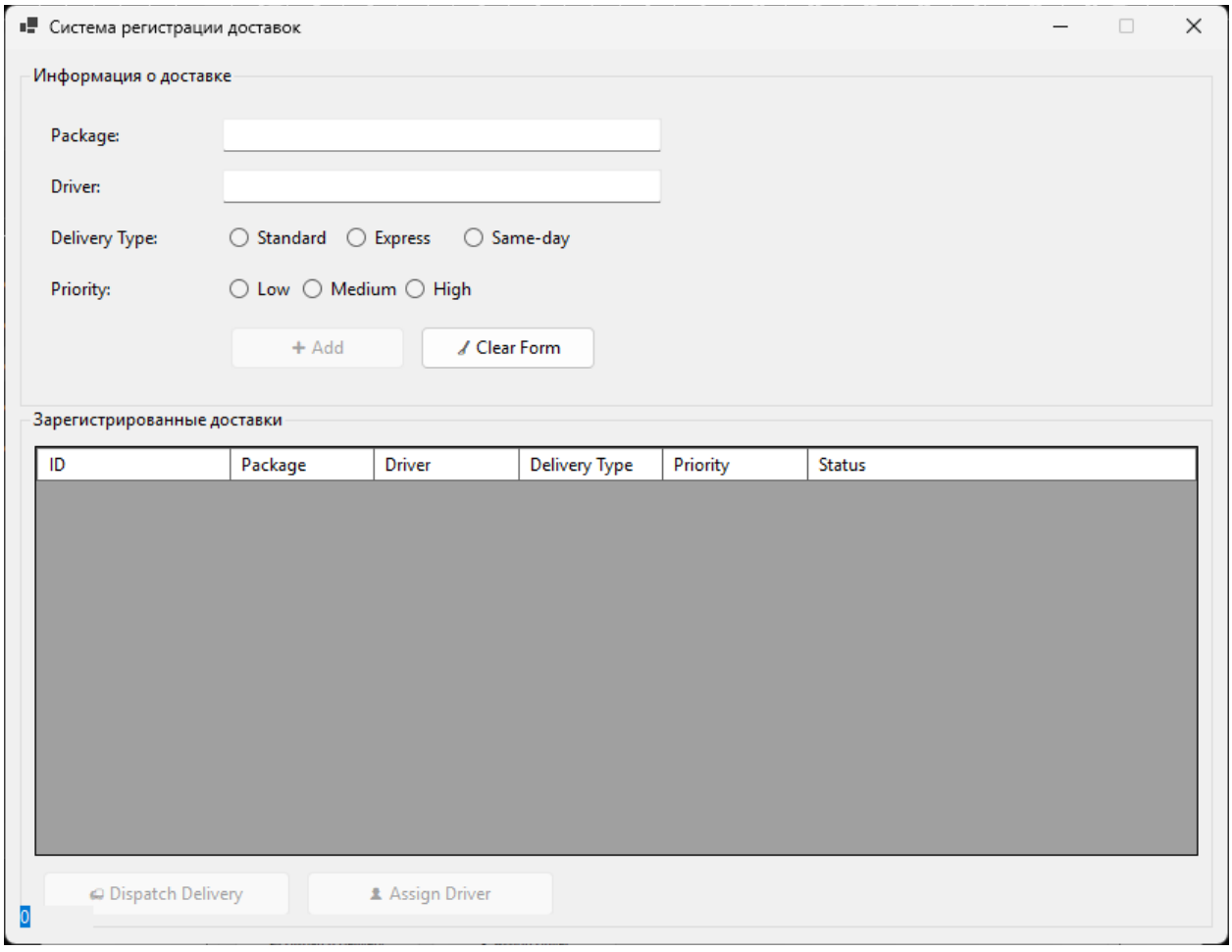
- **кнопки Dispatch Delivery и Assign Driver должны быть неактивны (IsEnabled = false), если строка таблицы не выбрана;**
- **при выборе строки кнопки становятся активными (IsEnabled = true);**
- **после выполнения действия кнопки могут оставаться активными или сбрасываться при снятии выделения;**
- **кнопка Dispatch Delivery должна быть неактивной, если статус доставки уже "Dispatched".**

📥 Входные данные (Excel)

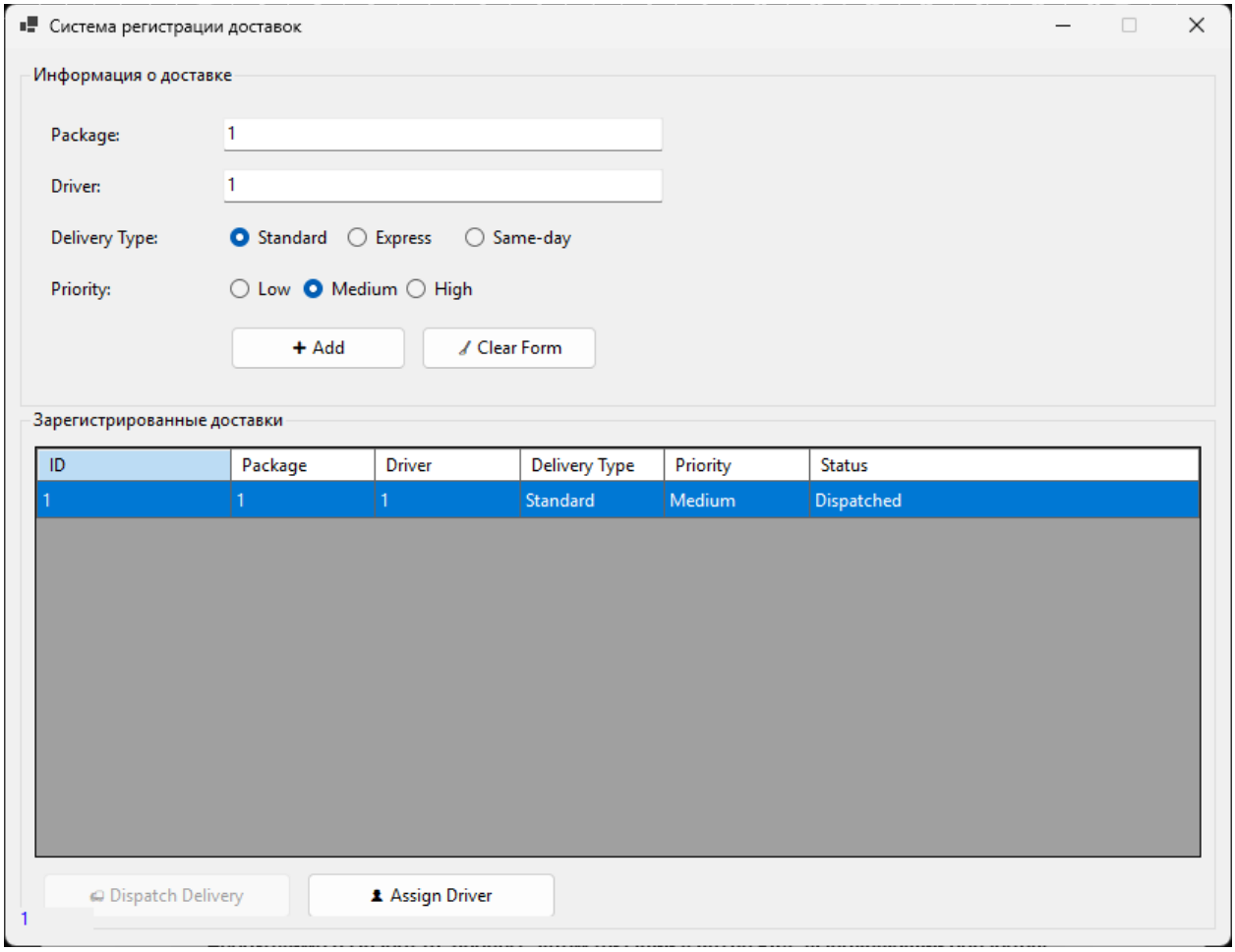
Для автоматизации используется внешний файл (например: deliveries.xlsx) со следующей структурой:

ID	Package	Driver	Delivery Type	Priority
1	Document Package	Ivan Petrov	Standard	High
2	Electronics Box	Anna Smirnova	Express	Medium
3	Food Delivery	Dmitry Orlov	Same-day	Low
4	Medical Supplies	Olga Ivanova	Express	High

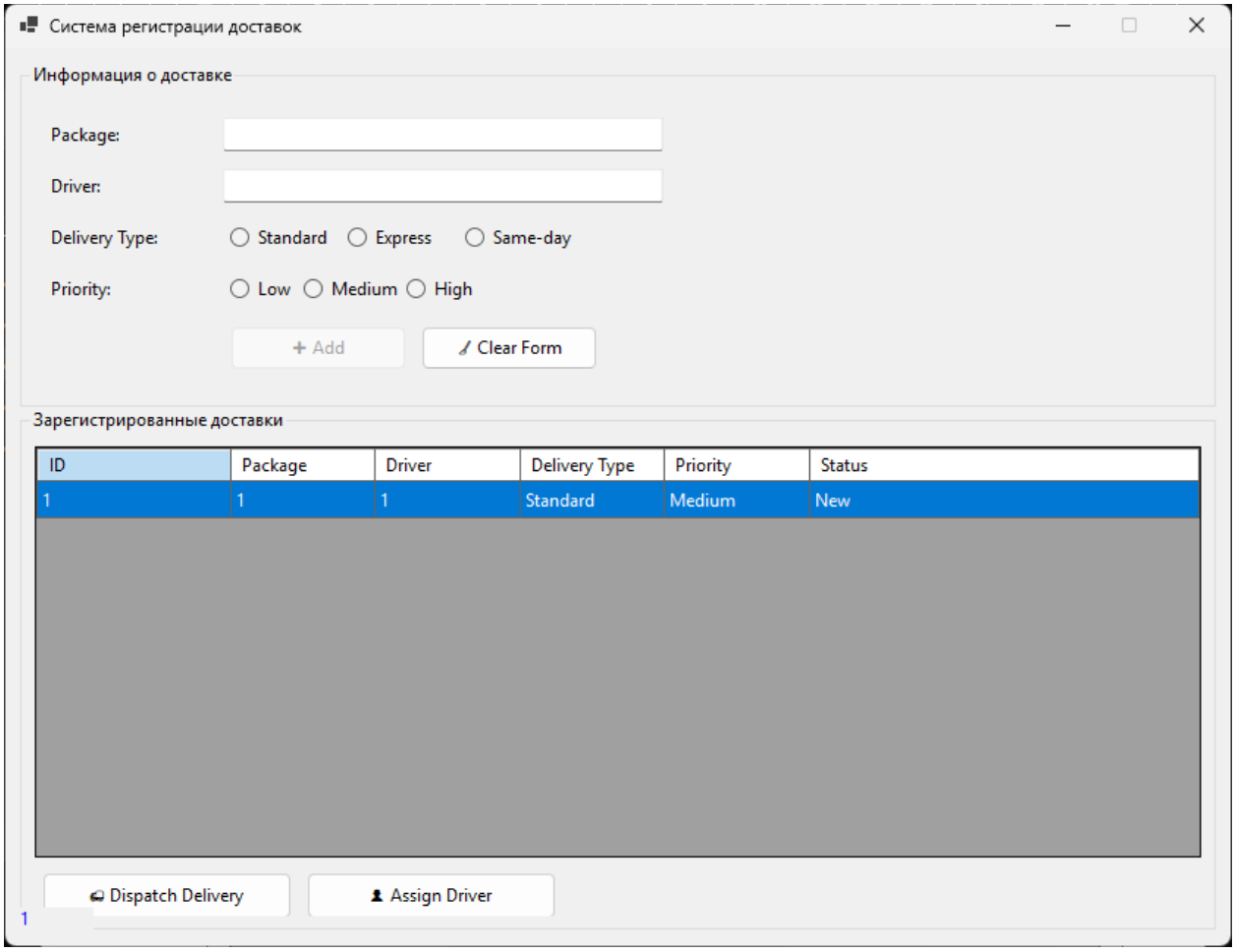
ID	Package	Driver	Delivery Type	Priority
5	Office Supplies	Sergey Kuznetsov	Standard	Medium
6	Furniture Set	Alex Sidorov	Standard	Low
7	Urgent Documents	Elena Volkova	Same-day	High
8	Retail Goods	Pavel Morozov	Express	Medium
9	Spare Parts	Irina Lebedeva	Standard	Low
10	Pharmaceutical	Kirill Fedorov	Express	High



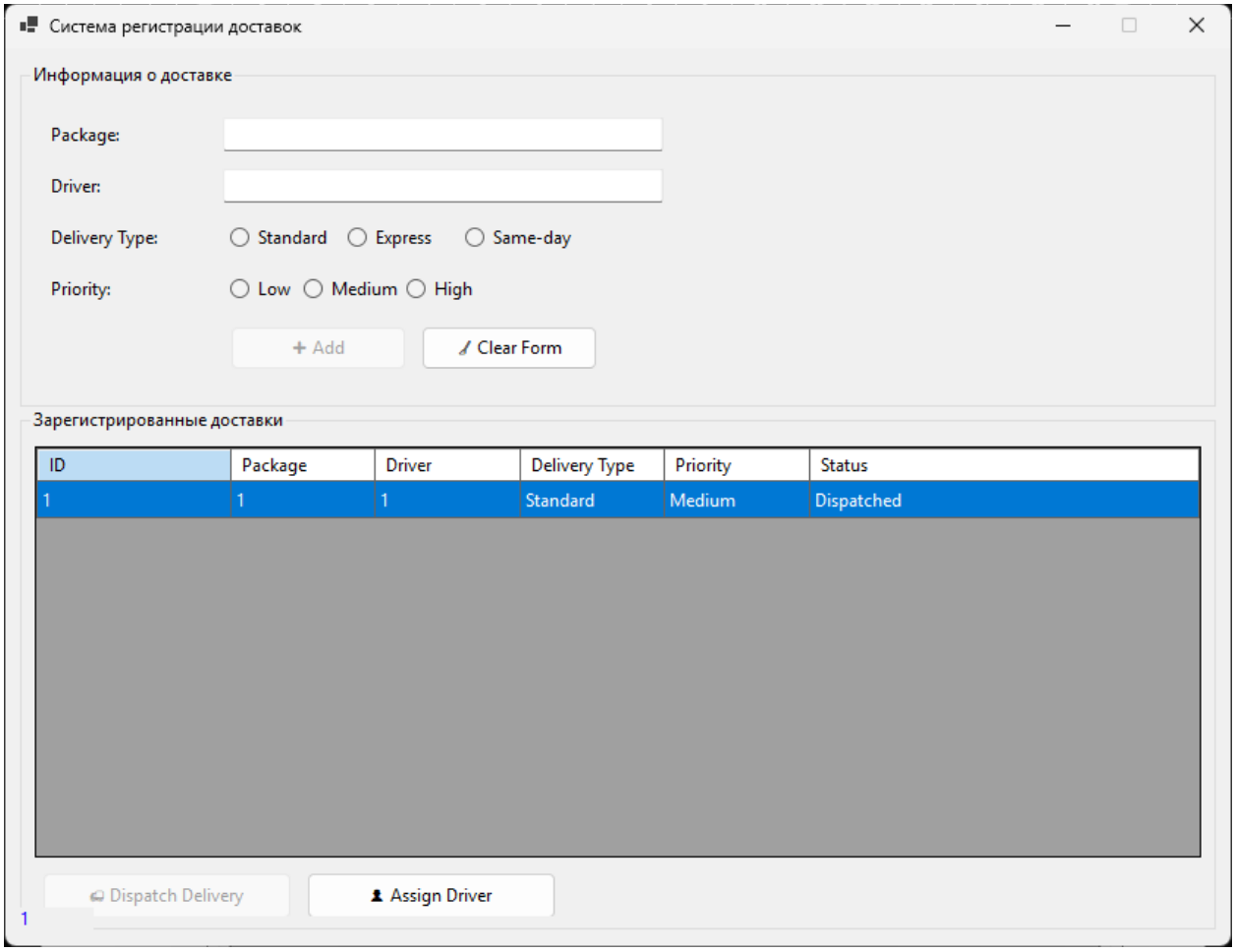
Кнопки неактивны тк нет данных о доставке



При заполненных полях о доставке, кнопка адд становится активной



При выделении строки в таблице кнопки становятся активными



Если статус диспатч, то кнопка диспатч деливери неактивна

Контрольное задание 2

Задание 2. Разработка расширенного RPA-сценария

Необходимо разработать процесс автоматизации в Puzzle RPA, выполняющий обработку доставок.

Алгоритм работы робота

1. **Запустить приложение.**
2. **Считать данные из Excel-файла.**
3. **Последовательно добавить не менее 10 доставок в систему.**

Для каждой записи:

- **заполнить поля формы;**
- **проверить состояние кнопки Add (IsEnabled);**
- **при необходимости скорректировать ввод;**

- добавить запись в таблицу.

4. Назначить водителя для каждой доставки (используя кнопку Assign Driver).
5. Найти все доставки с приоритетом High.
6. Для каждой такой доставки:

- выбрать строку в таблице;
- проверить, что кнопка Dispatch Delivery активна (IsEnabled = true);
- изменить статус на "Dispatched".

📊 Ожидаемый результат выполнения

После выполнения сценария в системе должно быть:

✓ Добавлено:

- 10 доставок из Excel

✓ Обновлено:

- для всех доставок выполнено действие Assign Driver
- все доставки с приоритетом High должны иметь статус:

{ "status": "Dispatched" }

📋 Пример итогового состояния таблицы

ID	Package	Driver	Delivery Type	Priority	Status
1	Document Package	Ivan Petrov	Standard	High	Dispatched
2	Electronics Box	Anna Smirnova	Express	Medium	New
3	Food Delivery	Dmitry Orlov	Same-day	Low	New
4	Medical Supplies	Olga Ivanova	Express	High	Dispatched
5	Office Supplies	Sergey Kuznetsov	Standard	Medium	New
...	...	...	...	...	...

🔎 Требования к корректности выполнения

Работа считается выполненной корректно, если:

✓ Интерфейс

- корректно реализовано управление IsEnabled;
- кнопки активируются только при выполнении условий;

- **корректно обрабатывается выбор строки таблицы;**

✓ Автоматизация

- **робот корректно читает данные из Excel;**
- **добавляет все записи;**
- **находит и обрабатывает доставки с приоритетом High;**
- **корректно работает с элементами интерфейса;**

✓ Данные

- **статусы обновлены корректно;**
- **данные не искажены;**

📜 Логирование выполнения процесса

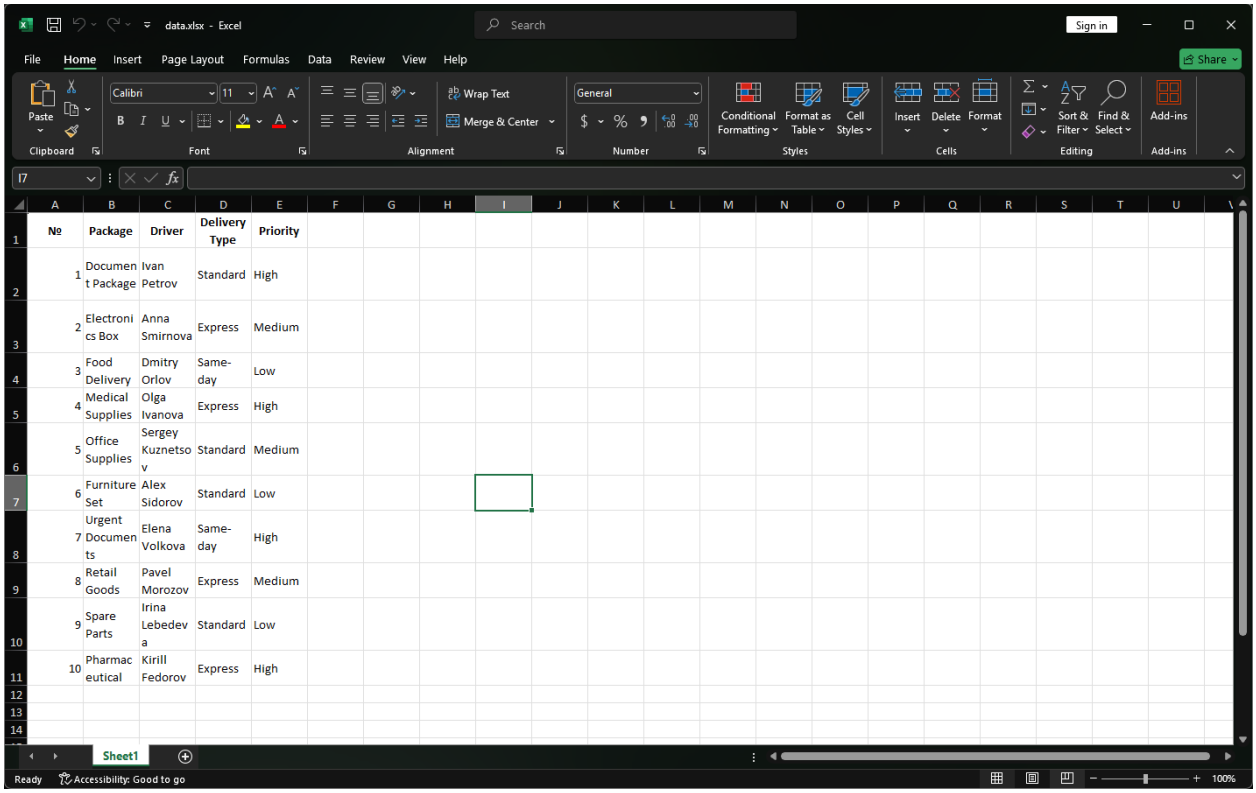
Необходимо реализовать подробное логирование.

В журнале должны фиксироваться:

- **запуск процесса;**
- **чтение данных из Excel;**
- **добавление каждой записи;**
- **изменение статусов;**
- **завершение процесса;**

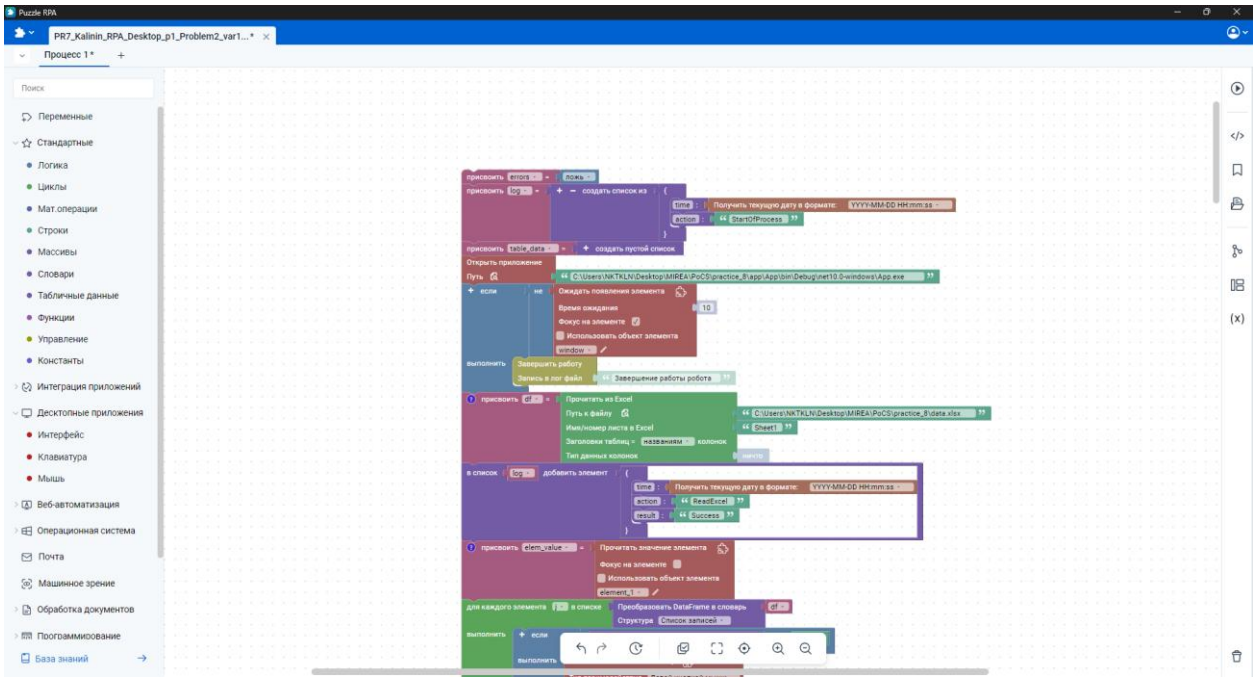
Пример:

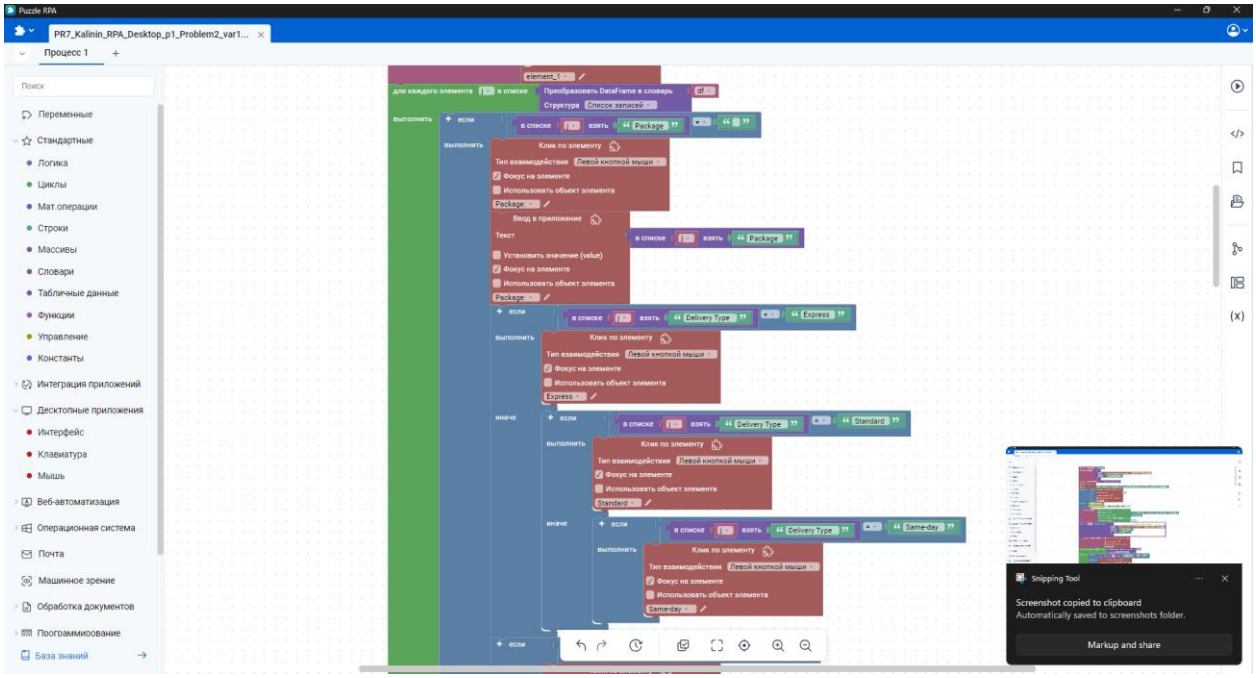

```
{
  "time": "2026-03-15 11:02:33",
  "step": "UpdateStatus",
  "delivery_id": 7,
  "new_status": "Dispatched",
  "result": "success"
}
```

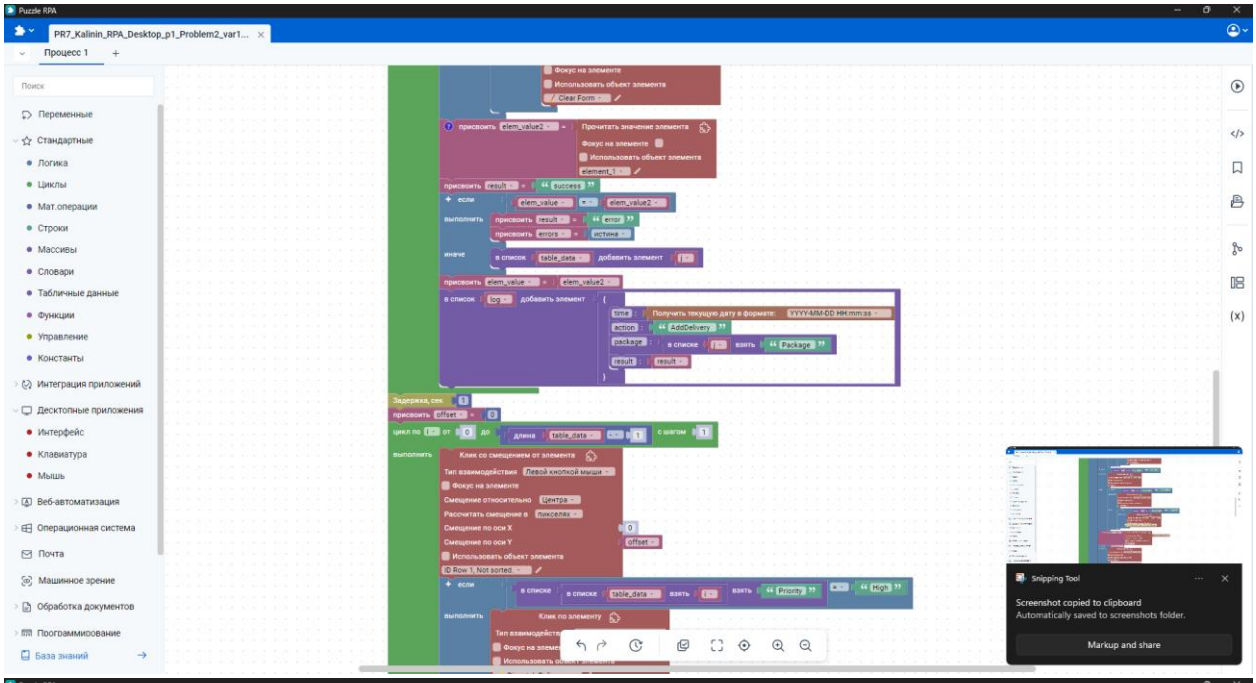


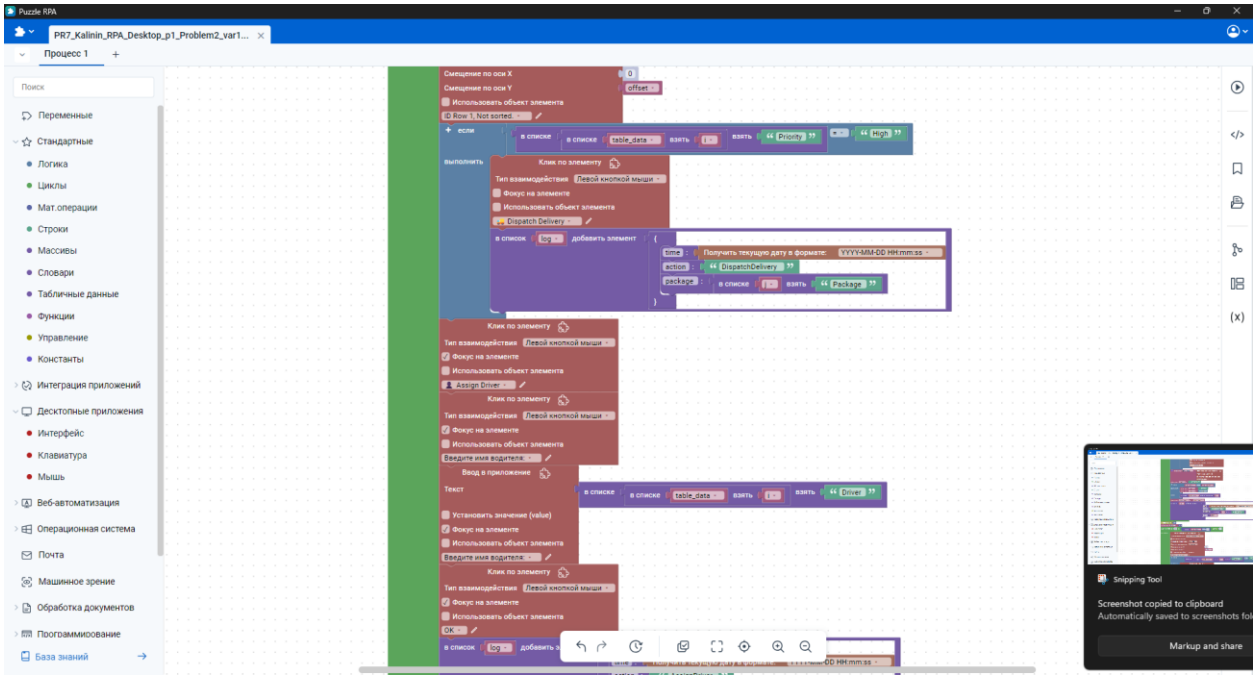
№	Package	Driver	Delivery Type	Priority
1	Document Package	Ivan Petrov	Standard	High
2	Electronics Box	Anna Smirnova	Express	Medium
3	Food Delivery	Dmitry Orlov	Same-day	Low
4	Medical Supplies	Olga Ivanova	Express	High
5	Office Supplies	Sergey Kuznetsov	Standard	Medium
6	Furniture Set	Alex Sidorov	Standard	Low
7	Urgent Documents	Elena Volkova	Same-day	High
8	Retail Goods	Pavel Morozov	Express	Medium
9	Spare Parts	Irina Lebedeva	Standard	Low
10	Pharmaceutical	Kirill Fedorov	Express	High

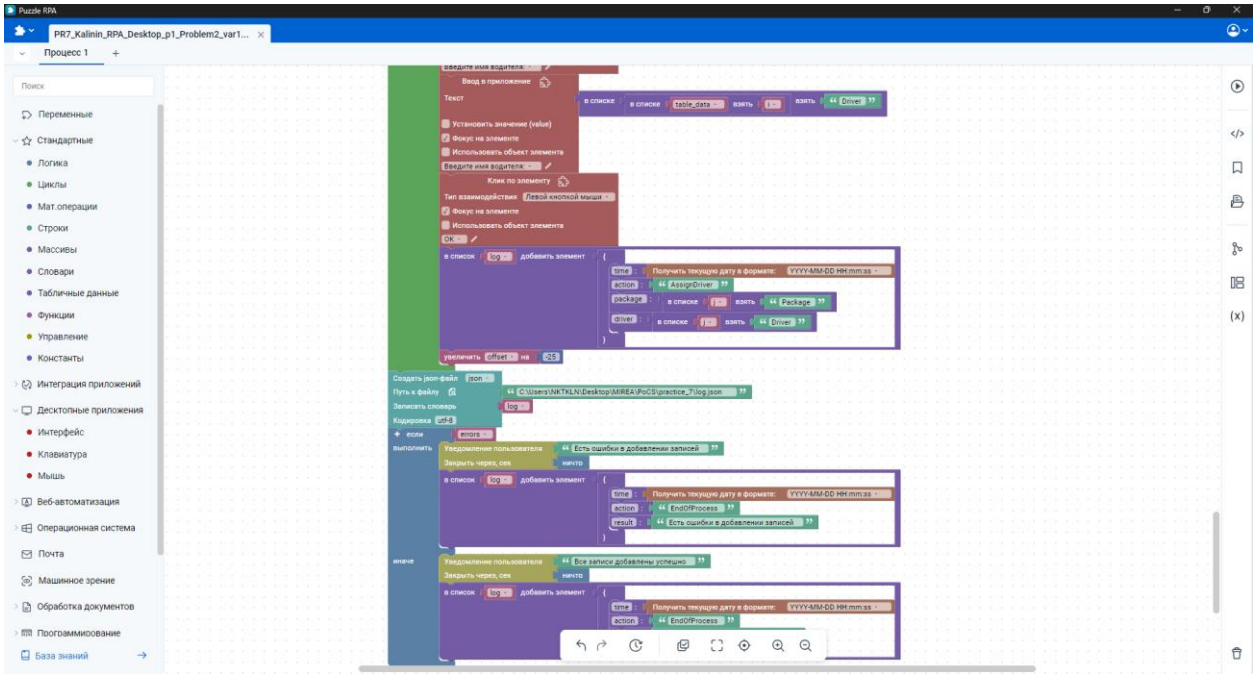
Успешное выполнение кода процесса

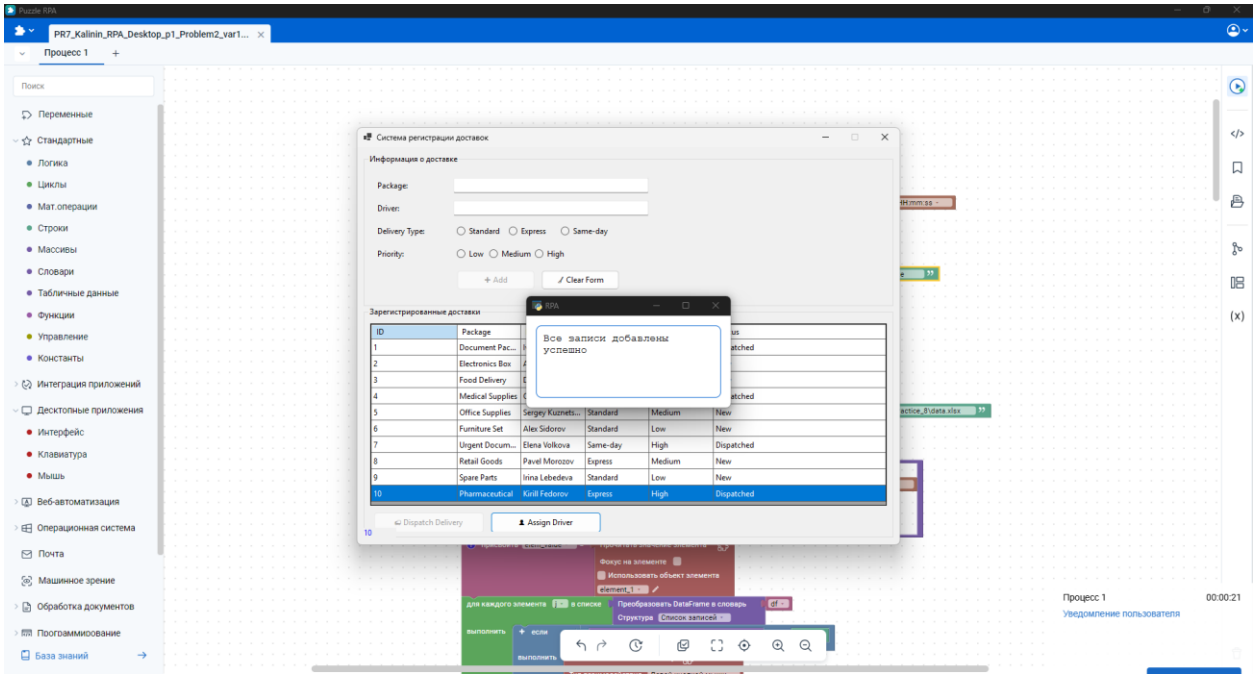


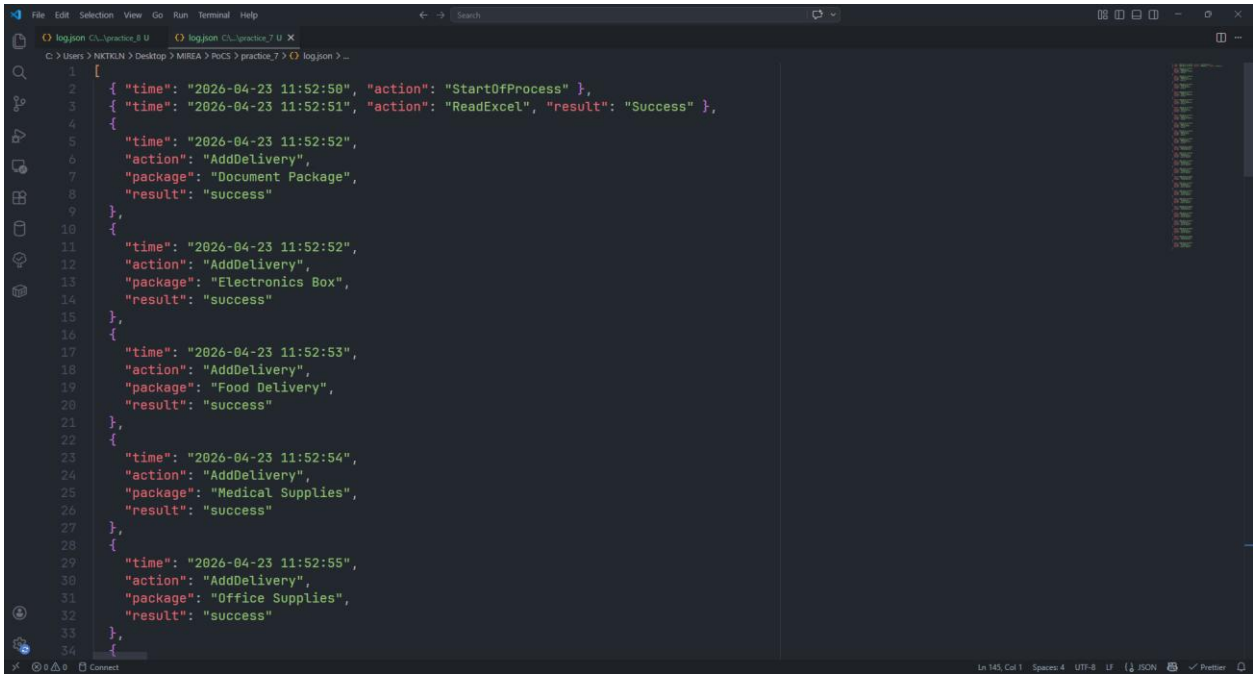

Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Поиск
Переменные
Стандартные
Логика
Циклы
Мат.операции
Строки
Массивы
Словари
Табличные данные
Функции
Управление
Константы
Интеграция приложений
Десктопные приложения
Интерфейс
Клавиатура
Мышь
Веб-автоматизация
Операционная система
Почта
Машинное зрение
Обработка документов
Программирование
База знаний
element_1
для каждого элемента
в списке
Преобразовать DataFrame в словарь
Структура
Список записей
выполнить
если
в списке
взять
Package
Клик по элементу
Тип взаимодействия
Левой кнопкой мыши
Фокус на элементе
Использовать объект элемента
Ввод в приложение
Текст
Установить значение (value)
Delivery Type
Express
Standard
Same-day
иначе
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Markup and share


Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Markup and share


Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Snipping Tool
Screenshot copied to clipboard
Markup and share


Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Поиск
Переменные
Стандартные
Логика
Циклы
Мат.операции
Строки
Массивы
Словари
Табличные данные
Функции
Управление
Константы
Интеграция приложений
Десктопные приложения
Интерфейс
Клавиатура
Мышь
Веб-автоматизация
Операционная система
Почта
Машинное зрение
Обработка документов
Программирование
База знаний


Puzzle RPA
PR7_Kalinin_RPA_Desktop_p1_Problem2_var1...
Процесс 1
Поиск
Переменные
Стандартные
Логика
Циклы
Мат.операции
Строки
Массивы
Словари
Табличные данные
Функции
Управление
Константы
Интеграция приложений
Десктопные приложения
Интерфейс
Клавиатура
Мышь
Веб-автоматизация
Операционная система
Почта
Машинное зрение
Обработка документов
Программирование
База знаний
Система регистрации доставок
Информация о доставке
Package:
Driver:
Delivery Type:
Standard
Express
Same-day
Priority:
Low
Medium
High
+ Add
Clear Form
Зарегистрированные доставки
RPA
Все записи добавлены успешно
Dispatch Delivery
Assign Driver
Процесс 1
00:00:21
Уведомление пользователя


File Edit Selection View Go Run Terminal Help
log.json C:\...\practice_8 U
log.json C:\...\practice_7 U
C: > Users > NIKTKLN > Desktop > MIREA > PoCS > practice_7 > log.json > ...
[
{ "time": "2026-04-23 11:52:50", "action": "StartOfProcess" },
{ "time": "2026-04-23 11:52:51", "action": "ReadExcel", "result": "Success" },
{
"time": "2026-04-23 11:52:52",
"action": "AddDelivery",
"package": "Document Package",
"result": "success"
},
{
"time": "2026-04-23 11:52:52",
"action": "AddDelivery",
"package": "Electronics Box",
"result": "success"
},
{
"time": "2026-04-23 11:52:53",
"action": "AddDelivery",
"package": "Food Delivery",
"result": "success"
},
{
"time": "2026-04-23 11:52:54",
"action": "AddDelivery",
"package": "Medical Supplies",
"result": "success"
},
{
"time": "2026-04-23 11:52:55",
"action": "AddDelivery",
"package": "Office Supplies",
"result": "success"
},
{
Connect
Ln 145, Col 1
Spaces: 4
UTF-8
LF
JSON
Prettier

Ссылка на репозиторий: https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_7

Ссылка на чат DeepSeek: https://chat.deepseek.com/share/z39ypk3ck7okfwxgqp